

# **Capstone II Final Report**

Smart Distributed Acoustic Calibration System (SDACS)

Kyle Dick (ID: 7206287), David Toma (ID: 8823349), Amy Wentzell (ID: 8820998)

Spring 2026 – Level 11

Bachelor of Applied Engineering – Electronic Systems Engineering, Conestoga  
College

EECE74135 - Capstone Project II

Professor Dr. So-Ra Chung, Professor Dr. Reda Fayek

August 17, 2026

## Table of Contents

|                                                                                                                              |    |
|------------------------------------------------------------------------------------------------------------------------------|----|
| ABSTRACT .....                                                                                                               | xi |
| 1. INTRODUCTION .....                                                                                                        | 1  |
| 1.1. Purpose.....                                                                                                            | 1  |
| 1.2. Project Definition.....                                                                                                 | 1  |
| 1.3. Background.....                                                                                                         | 1  |
| 1.4. Proposed Solution .....                                                                                                 | 2  |
| 2. LITERATURE REVIEW .....                                                                                                   | 4  |
| 2.1. Current Research.....                                                                                                   | 4  |
| 2.1.1. Three-dimensional Sound Source Diagnostic (2023).....                                                                 | 4  |
| 2.1.2. Spatial Audio System (2019).....                                                                                      | 6  |
| 2.2. Relevant References.....                                                                                                | 8  |
| 2.2.1. Two- and Three-dimensional Fast Fourier Transforms (FFTs).....                                                        | 8  |
| 2.2.2. FFT on the ESP-32.....                                                                                                | 10 |
| 2.2.3. Foundational Research and Standards for Bluetooth-Based Indoor Positioning and Distributed Acoustic Sensing [10]..... | 10 |
| 2.2.4. MATLAB Modelling Libraries.....                                                                                       | 12 |
| 2.2.5. Edge Impulse TinyML.....                                                                                              | 13 |
| 2.2.6. Market Research .....                                                                                                 | 15 |
| 3. PROJECT REQUIREMENTS.....                                                                                                 | 17 |
| 3.1. SMART Requirements.....                                                                                                 | 17 |
| 3.1.1. SMART Goal 1 – Multi-Node Embedded Acoustic System .....                                                              | 17 |
| 3.1.2. SMART Goal 2 – Reliable Embedded Firmware Operation.....                                                              | 17 |
| 3.1.3. SMART Goal 3 – Coordinated Multi-Node Recording.....                                                                  | 17 |
| 3.1.4. SMART Goal 4 – Wireless Data Aggregation and Processing .....                                                         | 17 |

|        |                                                                          |    |
|--------|--------------------------------------------------------------------------|----|
| 3.1.5. | SMART Goal 5 – Functional System Demonstration in Real Environment ..... | 17 |
| 3.1.6. | SMART Goal 6 – Verification, Validation, and Visualization.....          | 18 |
| 3.2.   | MoSCoW Requirements .....                                                | 19 |
| 4.     | SYSTEM DESIGN .....                                                      | 20 |
| 4.1.   | Functional Units.....                                                    | 21 |
| 4.1.1. | Recording Nodes.....                                                     | 21 |
| 4.1.2. | Server .....                                                             | 23 |
| 4.1.3. | Application.....                                                         | 25 |
| 4.2.   | Test Plans.....                                                          | 27 |
| 4.2.1. | Hardware Test Plan.....                                                  | 28 |
| 4.2.2. | Firmware Test Plan.....                                                  | 28 |
| 4.2.3. | Software Test Plan.....                                                  | 28 |
| 4.3.   | Off-the-shelf Components .....                                           | 30 |
| 4.3.1. | Node-RED.....                                                            | 30 |
| 4.3.2. | Microcontroller, Adafruit Metro S3.....                                  | 31 |
| 4.3.3. | Digital MEMS Microphone, ICS-43432.....                                  | 32 |
| 4.3.4. | LiPo Battery, 3.7V with MAX17048 Fuel Gauge .....                        | 33 |
| 4.3.5. | Temperature and Humidity Sensor, Sensirion SHT41 .....                   | 33 |
| 4.3.6. | PC, Raspberry Pi 5.....                                                  | 34 |
| 4.3.7. | M3 Screws and Standoffs (OEM).....                                       | 35 |
| 4.4.   | Safety, Societal and Environmental Issues .....                          | 36 |
| 4.4.1. | Safety Considerations .....                                              | 36 |
| 4.4.2. | Societal Considerations.....                                             | 36 |
| 4.4.3. | Environmental Considerations.....                                        | 37 |
| 5.     | DESIGN OF FUNCTIONAL UNITS .....                                         | 38 |

|        |                                                                   |    |
|--------|-------------------------------------------------------------------|----|
| 5.1.   | Hardware.....                                                     | 38 |
| 5.1.1. | Hardware Design .....                                             | 38 |
| 5.1.2. | Inputs and Outputs .....                                          | 39 |
| 5.1.3. | Schematics .....                                                  | 40 |
| 5.1.4. | Custom PCB Design .....                                           | 44 |
| 5.2.   | Firmware.....                                                     | 46 |
| 5.2.1. | Firmware Design.....                                              | 46 |
| 5.2.2. | Firmware Inputs and Outputs.....                                  | 48 |
| 5.2.3. | Firmware Process Flow.....                                        | 49 |
| 5.3.   | Middleware / Software.....                                        | 51 |
| 5.3.1. | Server Design.....                                                | 51 |
| 5.3.2. | Inputs and Outputs .....                                          | 52 |
| 5.3.3. | Node-RED Process Flow .....                                       | 54 |
| 5.3.4. | MATLAB Modelling.....                                             | 69 |
| 5.3.5. | Edge Impulse TinyML Model for Room Treatment Recommendations..... | 71 |
| 5.4.   | Enclosure Design .....                                            | 73 |
| 5.4.1. | Battery Port.....                                                 | 74 |
| 5.4.2. | MCU Housing .....                                                 | 75 |
| 5.4.3. | Lid .....                                                         | 76 |
| 6.     | DOCUMENTARY EVIDENCE OF SYSTEM DEVELOPMENT AND TESTING .....      | 77 |
| 6.1    | SDACS OSI Classification .....                                    | 77 |
| 6.2.   | Hardware.....                                                     | 80 |
| 6.2.1. | Embedded hardware and PCB .....                                   | 80 |
| 6.2.2. | Mechanical Enclosure.....                                         | 84 |
| 6.3.   | ESP-IDF Firmware and Embedded Signal Processing.....              | 88 |

|        |                                                   |     |
|--------|---------------------------------------------------|-----|
| 6.3.1. | Functional units.....                             | 88  |
| 6.3.2. | OSI classification.....                           | 88  |
| 6.3.3. | Development and Testing evidence .....            | 89  |
| 6.3.4. | System Integration and Performance evidence.....  | 92  |
| 6.4.   | Network and Communication.....                    | 95  |
| 6.4.1. | Functional units.....                             | 95  |
| 6.4.2. | OSI classification.....                           | 95  |
| 6.4.3. | Development and Testing evidence .....            | 95  |
| 6.4.4. | System Integration and Performance evidence.....  | 105 |
| 6.5.   | Middleware and Raspberry Pi Backend.....          | 107 |
| 6.5.1. | Functional units.....                             | 107 |
| 6.5.2. | OSI classification.....                           | 108 |
| 6.5.3. | Development and Testing evidence .....            | 108 |
| 6.5.4. | System Integration and Performance Evidence ..... | 109 |
| 6.6.   | Node-RED Data Collection Middleware .....         | 112 |
| 6.6.1. | Functional units.....                             | 112 |
| 6.6.2. | OSI classification.....                           | 112 |
| 6.6.3. | Development and Testing evidence .....            | 112 |
| 6.6.4. | System Integration and Performance evidence.....  | 112 |
| 6.7.   | Edge Impulse AI Classification .....              | 115 |
| 6.7.1. | Functional units.....                             | 115 |
| 6.7.2. | OSI classification.....                           | 115 |
| 6.7.3. | Development and Testing evidence .....            | 116 |
| 6.7.4. | System Integration and Performance evidence.....  | 118 |
| 6.8.   | MATLAB Analysis.....                              | 120 |

|        |                                                  |     |
|--------|--------------------------------------------------|-----|
| 6.8.1. | Functional units.....                            | 120 |
| 6.8.2. | OSI classification.....                          | 120 |
| 6.8.3. | Development and Testing evidence .....           | 120 |
| 6.8.4. | System Integration and Performance evidence..... | 121 |
| 6.9.   | Flutter/Dart GUI.....                            | 122 |
| 6.9.1. | Functional units.....                            | 122 |
| 6.9.2. | OSI classification.....                          | 122 |
| 6.9.3. | Development and Testing evidence .....           | 122 |
| 6.9.4. | System Integration and Performance evidence..... | 130 |
| 7.     | RESULTS AND ANALYSIS .....                       | 131 |
| 7.1.   | Node Frequency Detection Accuracy .....          | 131 |
| 7.2.   | TinyML Classifier Accuracy .....                 | 132 |
| 7.3.   | Spatial Acoustic Profile Accuracy .....          | 133 |
| 8.     | CONCLUSION AND RECOMMENDATIONS .....             | 135 |
| 8.1.   | Conclusion .....                                 | 135 |
| 8.2.   | Future Recommendations .....                     | 135 |
| 8.2.1. | Expand the Network .....                         | 135 |
| 8.2.1. | Detailed FFT Analysis .....                      | 135 |
| 8.2.2. | Minimize and Improve Hardware.....               | 135 |
|        | APPENDICES .....                                 | 136 |
|        | REFERENCES .....                                 | 137 |

## List of Figures

|                                                                                                                                                                                                                                                                    |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1: SDACS Conceptual Diagram.....                                                                                                                                                                                                                            | 3  |
| Figure 2: SFA and SFS System Block Diagram [8] .....                                                                                                                                                                                                               | 7  |
| Figure 3: N-Dimensional FFT [11].....                                                                                                                                                                                                                              | 8  |
| Figure 4: Storage Arrangement of the frequencies in the output FFT [11] .....                                                                                                                                                                                      | 9  |
| Figure 5: Bluetooth asset tracking based on RSSI and trilateration [10].....                                                                                                                                                                                       | 11 |
| Figure 6: SDACS System Block Diagram.....                                                                                                                                                                                                                          | 20 |
| Figure 7: SDACS Recording Node Block Diagram .....                                                                                                                                                                                                                 | 22 |
| Figure 8: SDACS Server Block Diagram .....                                                                                                                                                                                                                         | 24 |
| Figure 9: SDACS Application Block Diagram.....                                                                                                                                                                                                                     | 26 |
| Figure 10: MEMS Microphone Schematic .....                                                                                                                                                                                                                         | 40 |
| Figure 11: Temperature and Humidity Sensor Schematic.....                                                                                                                                                                                                          | 41 |
| Figure 12: Power LED Array Schematic .....                                                                                                                                                                                                                         | 42 |
| Figure 13: Metro Board Pinouts Schematic.....                                                                                                                                                                                                                      | 43 |
| Figure 14: PCB Design for Manufacturing, 3-D vs. 2-D Views in Altium.....                                                                                                                                                                                          | 45 |
| Figure 15: Audio Processing Flowchart Showing I2S to MQTT Pathway .....                                                                                                                                                                                            | 47 |
| Figure 16:SDACS firmware and Flutter GUI process flow with MQTT and REST API<br>communication.....                                                                                                                                                                 | 49 |
| Figure 17:Summarized SDACS software data-flow architecture with MQTT communication<br>between the distributed sensor nodes and Raspberry Pi, Node-RED telemetry processing and<br>storage, FastAPI backend communication, and the Flutter operator interface. .... | 51 |
| Figure 18:SDACS 0 Broker Smoke Test flow, showing simulated telemetry generation, MQTT<br>publication, and dashboard confirmation .....Live Telemetry: Flow and Dashboard Module<br>.....                                                                          | 54 |
| Figure 19: SDACS Live Telemetry Node-RED flow for receiving, normalizing, and displaying<br>real-time acoustic measurements and node-health information.....                                                                                                       | 55 |
| Figure 20: Live Telemetry dashboard module.....                                                                                                                                                                                                                    | 55 |
| Figure 21:SDACS Capture Control Node-RED flow for configuring, initiating, and monitoring<br>synchronized multi-node acoustic data collection. ....                                                                                                                | 56 |
| Figure 22: Capture Control Dashboard Module .....                                                                                                                                                                                                                  | 57 |

|                                                                                                                                                                                     |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 23: SDACS OTA and Commands Node-RED flow for issuing remote node commands and monitoring firmware-update status. ....                                                        | 58 |
| Figure 24: OTA + Commands Dashboard Module.....                                                                                                                                     | 59 |
| Figure 25:SDACS BLE Awareness Node-RED flow for triggering node advertisements, performing a Raspberry Pi gateway scan, and displaying node-presence and proximity information..... | 60 |
| Figure 26:BLE Awareness Dashboard Module Showing RSSI and Details .....                                                                                                             | 60 |
| Figure 27: SDACS Storage and Downloads Node-RED flow for normalizing MQTT records, logging data to SQLite and CSV files, and providing HTTP download access. ....                   | 61 |
| Figure 28: Node-RED Dashboard Widget Showing Data Logged to CSV and DB with Download Links .....                                                                                    | 62 |
| Figure 29:SDACS AI Dataset Collection Node-RED flow for label selection, feature validation, CSV generation, dataset reset, download handling, and MATLAB summary creation. ....    | 63 |
| Figure 30: SDACS AI Dataset Collection dashboard showing class selection, latest-row validation, CSV download, header repair, and dataset reset controls.....                       | 64 |
| Figure 31: SDACS MEMS Room Analysis Node-RED flow for receiving acoustic features, adding test metadata, displaying normalized results, and writing the MEMS testing CSV. ....      | 65 |
| Figure 32: MEMS Capture Summary dashboard showing the latest room-analysis measurements, 1 kHz detection metrics, audio diagnostics, and four-node comparison table.....            | 66 |
| Figure 33: MEMS CSV Controls and download interface used to assign test metadata, manage the room-analysis dataset, and confirm successful CSV storage.....                         | 67 |
| Figure 34: SDACS Runtime Capture Telemetry Node-RED flow for linking acoustic feature records to active captures and storing them in SQLi .....                                     | 68 |
| Figure 35: MATLAB Model, Sample Data from SDACS .....                                                                                                                               | 69 |
| Figure 36: SDACS Enclosure, Revision F.....                                                                                                                                         | 73 |
| Figure 37: SDACS Battery Port, Revision F .....                                                                                                                                     | 74 |
| Figure 38: SDACS MCU Housing, Revision F .....                                                                                                                                      | 75 |
| Figure 39: SDACS Lid, Revision F .....                                                                                                                                              | 76 |
| Figure 40: Visual Depiction of the OSI Model.....                                                                                                                                   | 77 |
| Figure 41:SDACS functional units and end-to-end system flo .....                                                                                                                    | 79 |
| Figure 42: Custom Manufactured PCB (within enclosure) .....                                                                                                                         | 81 |

|                                                                                                                                       |     |
|---------------------------------------------------------------------------------------------------------------------------------------|-----|
| Figure 43: SDACS Node Showing LED Array Lit .....                                                                                     | 82  |
| Figure 44: ESP-IDF monitor showing battery voltage and rate of charge .....                                                           | 82  |
| Figure 45: ESP-IDF Monitor Showing Capture Task.....                                                                                  | 82  |
| Figure 46: 4 Assembled Nodes .....                                                                                                    | 83  |
| Figure 47: Group Capture Dashboard Module .....                                                                                       | 83  |
| Figure 48: Custom Enclosure CAD Model.....                                                                                            | 84  |
| Figure 49: Custom Enclosure Progression (Rev B to Rev F) .....                                                                        | 85  |
| Figure 50: SDACS Lab Environment w/ Nodes Installed .....                                                                             | 86  |
| Figure 51: Custom Manufactured Enclosure, Fully Assembled Node .....                                                                  | 87  |
| Figure 52: Firmware Process Flow Diagram .....                                                                                        | 89  |
| Figure 53: ESP-IDF Monitor Showing Temperature Readings.....                                                                          | 91  |
| Figure 54: ESP-IDF firmware functional flow from audio acquisition to MQTT publication. ...                                           | 92  |
| Figure 55: Node-RED Dashboard "Node Health" Module showing 4 Nodes Capturing.....                                                     | 93  |
| Figure 56: Node-RED Dashboard "Capture Control" Module showing 4 Nodes Capturing,<br>Armed and Complete. ....                         | 94  |
| Figure 57: Warnings in SDACS Results Page .....                                                                                       | 94  |
| Figure 58: SDACS Network Architecture Diagram .....                                                                                   | 96  |
| Figure 59:MQTT Payload.....                                                                                                           | 99  |
| Figure 60: MQTT Payload Formatted as a JSON.....                                                                                      | 100 |
| Figure 61: Confirms Backend Endpoints Can be Reached at <BaseURL>.....                                                                | 101 |
| Figure 62: List of API Paths .....                                                                                                    | 101 |
| Figure 63: HTTP Methods.....                                                                                                          | 102 |
| Figure 64: Calibration Endpoints.....                                                                                                 | 103 |
| Figure 65: Capture Endpoints .....                                                                                                    | 103 |
| Figure 66: Node Endpoints .....                                                                                                       | 103 |
| Figure 67: Shows 4 Nodes Connected to Node-RED at Once.....                                                                           | 105 |
| Figure 68: MQTT Subscribe Listening Script Showing 4 Nodes are Active .....                                                           | 106 |
| Figure 69: Shows 1 Missing Node Offline (3 / 4 Nodes) .....                                                                           | 106 |
| Figure 70: Shows 3 / 4 Nodes Captured .....                                                                                           | 107 |
| Figure 71: Raspberry Pi backend receiving four-node MQTT telemetry: mosquitto_sub -h<br>localhost -t "sdacs/node/+/features" -v ..... | 109 |

|                                                                                                            |     |
|------------------------------------------------------------------------------------------------------------|-----|
| Figure 72: FastAPI endpoint returning the latest node feature record.....                                  | 110 |
| Figure 73: SQLite records produced during a synchronized capture.....                                      | 111 |
| Figure 74: First Iteration of AI Model.....                                                                | 117 |
| Figure 75: Final Iteration of AI Model .....                                                               | 118 |
| Figure 76: SDACS Node Layout, Actual (Real) vs. GUI (Setup) vs. Spatial Acoustic Profile<br>(Results)..... | 121 |
| Figure 77: Flutter Operation Interface.....                                                                | 124 |
| Figure 78: Summary of All Nodes, Target Frequency (Hz) vs. Measured Peak Frequency (Hz)<br>.....           | 131 |
| Figure 79: Ideal Lab Environment for Maximum Audio Data Quality .....                                      | 132 |

## List of Tables

|                                                          |    |
|----------------------------------------------------------|----|
| Table 1: MoSCoW Summary Table .....                      | 19 |
| Table 2: Hardware Inputs and Outputs .....               | 39 |
| Table 3: Firmware Inputs and Outputs .....               | 48 |
| Table 4: Node-RED Inputs and Outputs.....                | 52 |
| Table 5: Flutter Dart Inputs and Outputs .....           | 53 |
| Table 6: OSI Model Classification for SDACS Project..... | 78 |
| Table 7: Firmware OSI Classification Layers .....        | 88 |
| Table 8: Before and After Addition of Gain.....          | 90 |
| Table 9: Before and After High Pass Filter .....         | 90 |
| Table 10: Observed Performance of Applied Tones.....     | 90 |
| Table 11:Core Node Topics.....                           | 96 |
| Table 12:Command Topics.....                             | 97 |
| Table 13: AI and BLE-related topics.....                 | 97 |

## **ACKNOWLEDGEMENTS**

This project is possible due to the School of Engineering and Technology at Conestoga College, specifically the faculty of the Electronic Systems Engineering – Bachelor of Applied Engineering Degree program. Special thanks to Professor Dr. So-Ra Chung, Professor Dr. Reda Fayek, and Professor Dr. Michael Galle for their continued support and valuable consultation.

## **ABSTRACT**

The Smart Distributed Acoustic Calibration System (SDACS) is a portable, multi-node spatial audio calibration network developed to evaluate and model room acoustics for home studios and non-professional recording spaces. Traditional professional acoustic measurement tools are often expensive and rely on single-microphone configurations that fail to capture comprehensive room-wide variations. SDACS addresses this gap by deploying a distributed array of ESP32-S3 microcontroller nodes integrated with digital MEMS microphones, temperature and humidity sensors, and battery management systems to record localized sound pressure levels (SPL) and frequency responses. Collected data is transmitted wirelessly via MQTT to a central Raspberry Pi 5 server running Node-RED, where it is validated, logged, and processed. The system incorporates an automated TinyML classifier built with Edge Impulse to categorize room acoustic profiles and utilizes MATLAB-based spatial modeling to map frequency distributions. A Flutter cross-platform graphical user interface presents real-time telemetry, node status, and acoustic recommendations, enabling users to optimize recording positions and equalization settings without costly professional equipment.

# 1. INTRODUCTION

## *1.1. Purpose*

The purpose of this document is to present the design, testing, and implementation of the Smart Distributed Acoustic Calibration System (SDACS) project for Capstone I and II. This document was written for evaluation by the faculty of Conestoga College, including Professor Dr. So-Ra Chung and Professor Dr. Reda Fayek.

## *1.2. Project Definition*

SDACS is a spatial audio calibration network designed to model the acoustics of an internal space for the purpose of defining the best locations and/or conditions for recording. Inside a room, a source speaker performs a frequency sweep in the form of a logarithmic ping sequence to mimic the main audio source (i.e. a voice or other instrument) while a network of microphone nodes records the frequency response via Fast Fourier Transform (FFT) and Sound Pressure Level (SPL) at various locations around the room. Once calibrated, SDACS gathers the measurements from every node using Wi-Fi and Bluetooth, and outputs a three-dimensional model of the room acoustics to the user on an application/website.

## *1.3. Background*

Modern audio editing software such as Ableton [1], Logic Pro [2], and Audacity [3] offer clients the ability to produce complex music and audio recordings with a high degree of accuracy. While audio editing can enhance recordings, live acoustics still heavily impact the quality of recordings [4]. This is why professional producers still line their recording rooms with sound-proofing foam, purchase expensive high-quality microphones, and use gargantuan soundboards to enhance the audio quality in their studio while they record and mix. But what if someone does not want to be the next Justin Bieber? Many users do not have extensive capital to invest in sound-isolating equipment, but they still want to get the best out of their home studio – their bedroom, their office, or even their garage. How can a user maximize their audio quality when they cannot afford to build their own recording studio?

#### *1.4. Proposed Solution*

The Smart Distributed Acoustic Calibration System provides users with a fast and easy acoustics analysis of their chosen internal recording space. By setting up the nodes throughout the room, and playing a few seconds of audio, the user can view a complete three-dimensional model of its acoustics and get recommendations on the best places to record, and on the equalizer (EQ) profiles that will maximize audio quality without purchasing any new equipment.

Existing software solutions such as Room EQ Wizard provide the same analysis for no cost but use only one microphone as a reference. Using one audio source does not accurately map the whole room and does not aid in accurately identifying problem areas in the space like corners that produce poor-sounding reflections [5,4]. SDACS is a multiple-node system that records the acoustics of the entire room with an array of mics, identifying the best (and worst) areas to record, and recommended EQ settings for different areas of the room.

Existing embedded solutions such as ARC Studio and Sonarworks SoundID are more advanced and designed to provide the user with live EQ-adjustment [6,7]. These technologies, however, are still made primarily for producers. They are specifically made for mixing in recording studios, they use only one microphone, and they are expensive, requiring at minimum an investment of 500 Canadian Dollars (CAD) or more [6]. SDACS is designed to map the room acoustics as they are to the user, so they can make informed decisions about where they record and how to set the space up for their individual needs.

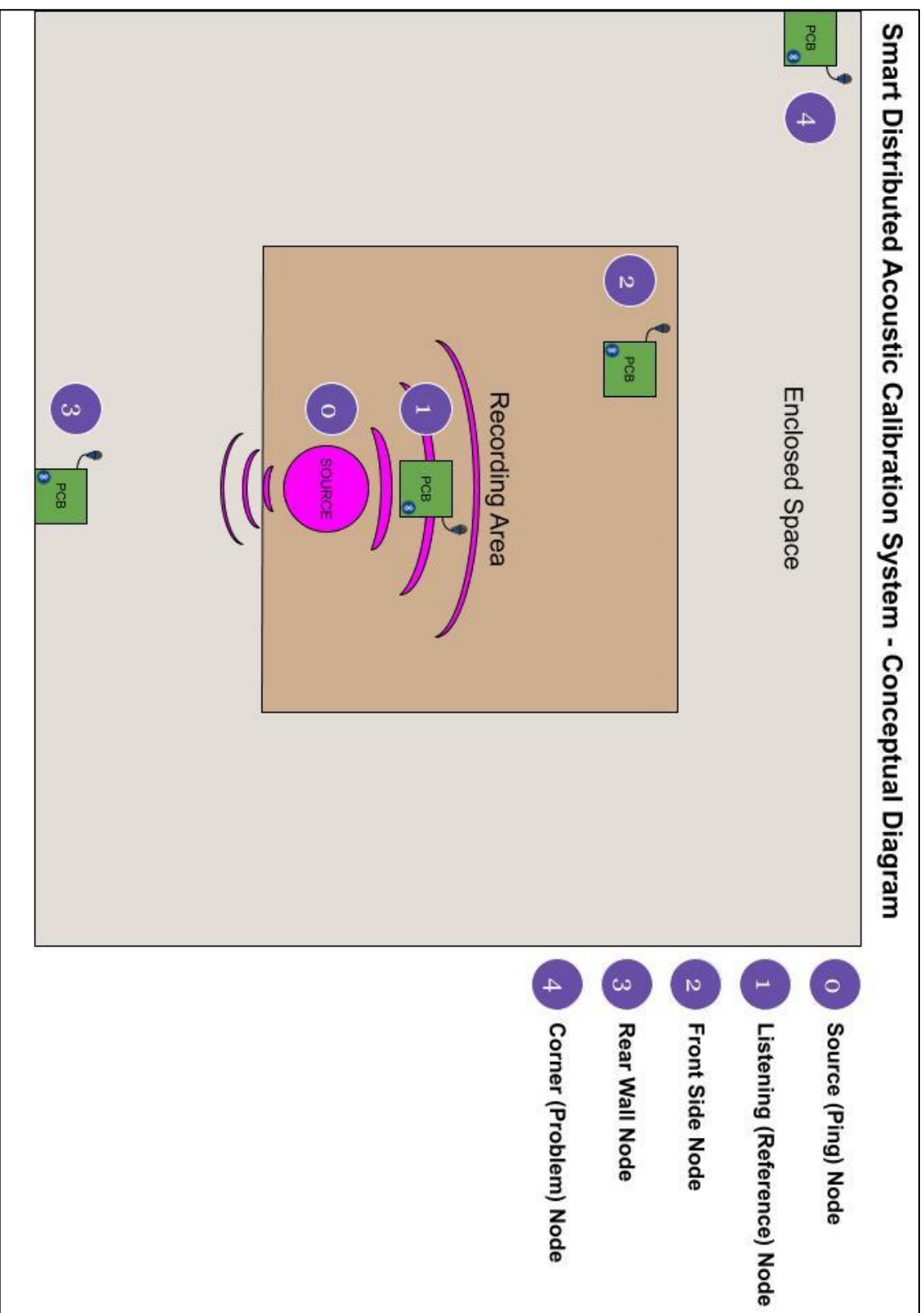


Figure 1: SDACS Conceptual Diagram

## 2. LITERATURE REVIEW

### 2.1. *Current Research*

#### 2.1.1. *Three-dimensional Sound Source Diagnostic (2023)*

In 2023, the Faculty of Engineering at Sherbrooke University and the Faculty of Music at McGill University used a spherical microphone array to produce a sound source diagnostic [9]. Inside a sound-proofed room, the array was placed in the center of multiple sound sources outputting at the same time. Using an Acoustic Imaging Structure From Motion (AISFM) framework, each individual acoustic source was identified and mapped by determined position and directivity from the spherical microphone array.



Figure 3: Three-dimensional Sound Source Diagnostic Infographic [9]

### *2.1.2. Spatial Audio System (2019)*

In 2019, the Department of Mechanical Engineering at National Tsing Hua University proposed a spatial audio system design consisting of a circular microphone array for Sound Field Analysis (SFA) and a logarithmically spaced loudspeaker array for Sound Field Synthesis (SFS) [8]. The system modelled the three-dimensional acoustics of a room in two steps:

1. Sample the environment using the SFA microphone array.
2. Mimic the environment using the SFS loudspeaker array.

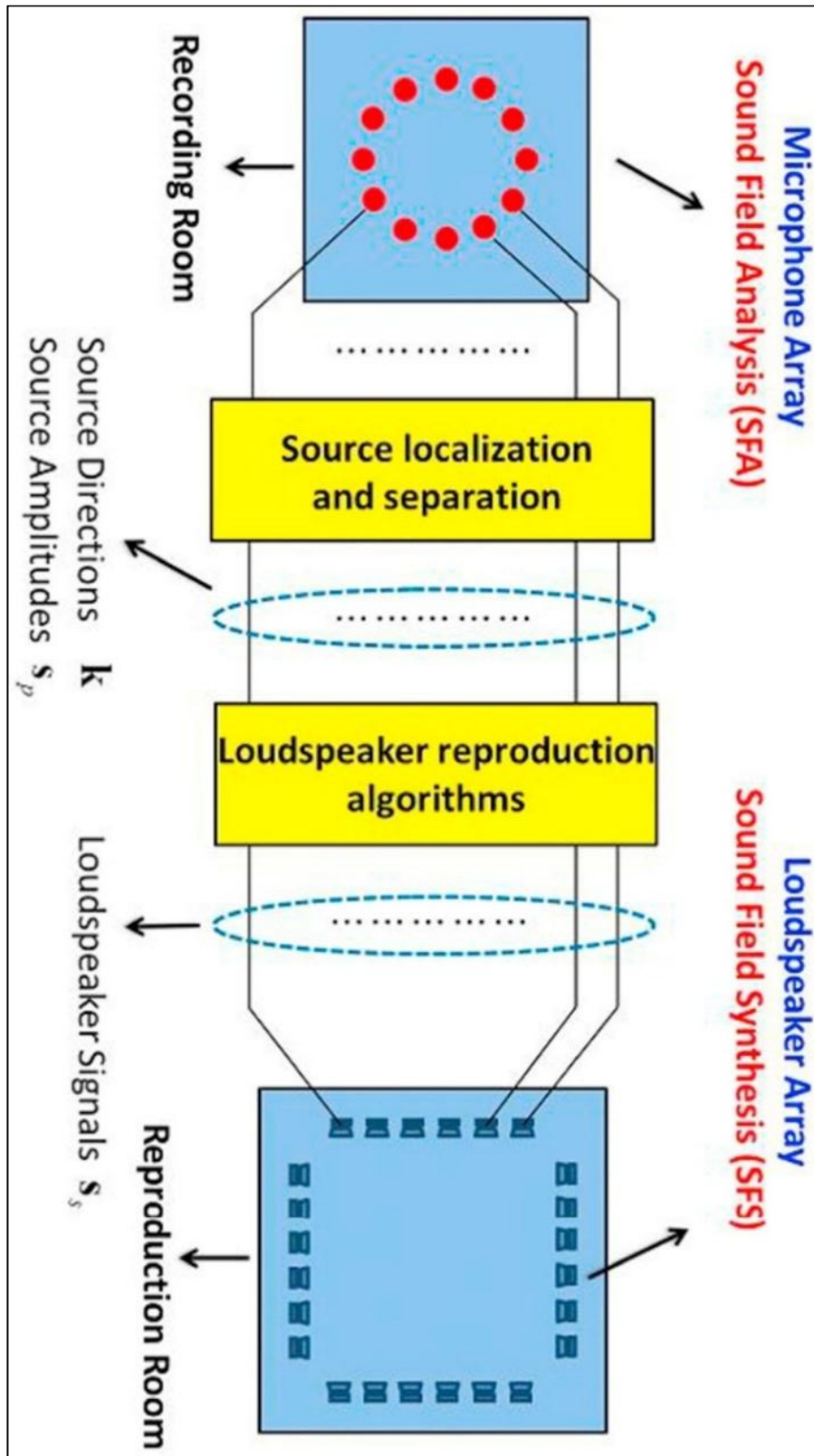


Figure 2: SFA and SFS System Block Diagram [8]

## 2.2. Relevant References

### 2.2.1. Two- and Three-dimensional Fast Fourier Transforms (FFTs)

The FFT of a signal in infinite dimensions is as follows:

$$H(n_1, \dots, n_L) \equiv \sum_{k_L=0}^{N_L-1} \cdots \sum_{k_1=0}^{N_1-1} \exp(2\pi i k_L n_L / N_L) \times \cdots \\ \times \exp(2\pi i k_1 n_1 / N_1) h(k_1, \dots, k_L)$$

Figure 3: *N*-Dimensional FFT [11]

The practical application for FFT in two dimensions is to run row-by-row in the two-dimensional space, and to pad around the border of the space with zeroes to ensure no discontinuities in the results [11].

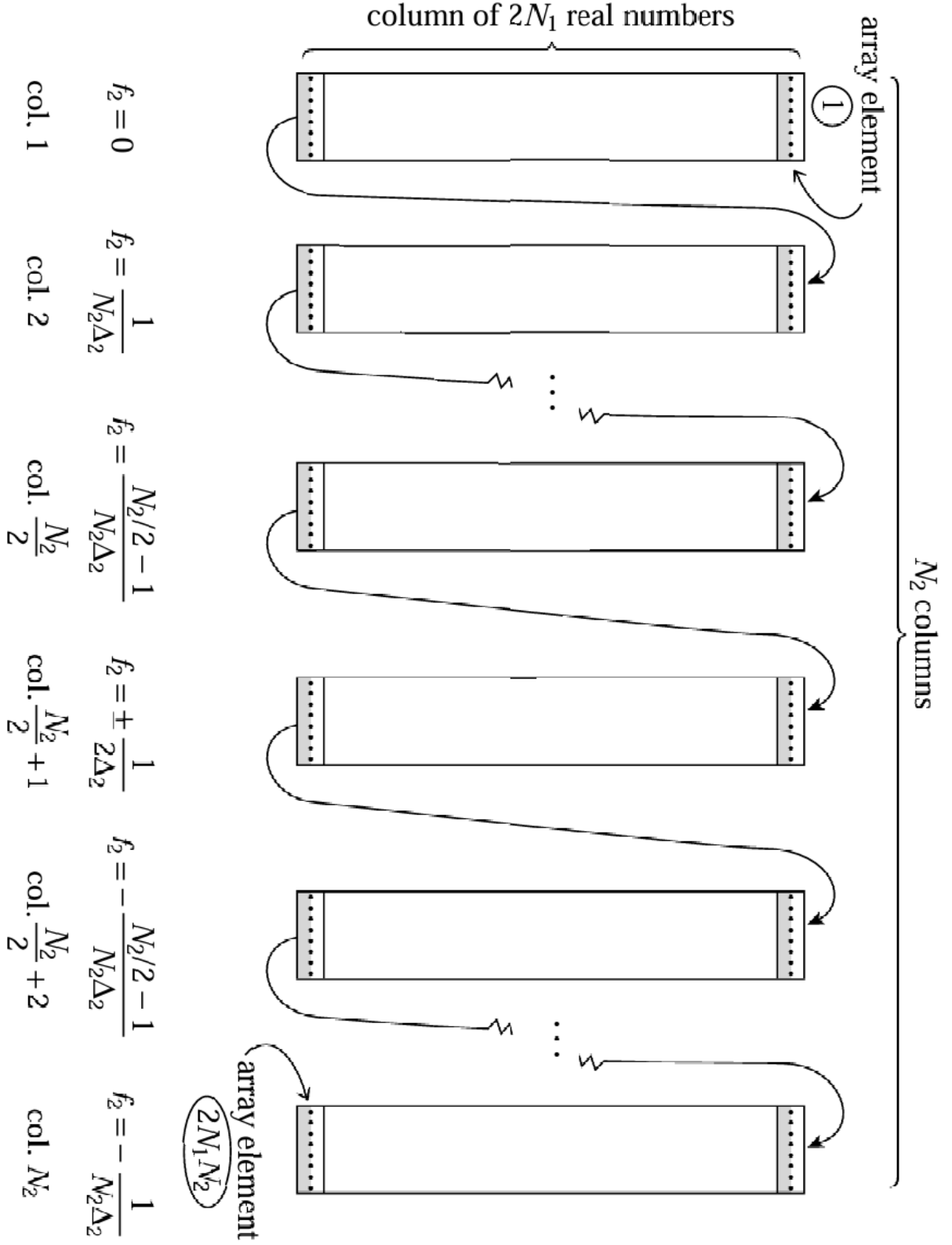


Figure 4: Storage Arrangement of the frequencies in the output FFT [11]

### *2.2.2.FFT on the ESP-32*

The ESP-32 microcontroller has pre-existing functions in the ESP-DSP library to perform FFT from a given signal [24]. This is accessible via the ESP-DSP library of functions.

### *2.2.3. Foundational Research and Standards for Bluetooth-Based Indoor Positioning and Distributed Acoustic Sensing [10]*

This project is informed by existing research, industry standards, and commercial tools related to indoor positioning systems, wireless sensor networks, and Bluetooth Low Energy (BLE)–based location services. The Bluetooth Special Interest Group (SIG) standardized Bluetooth Real-Time Locating System (RTLS) architectures that estimate device position using received signal strength indicator (RSSI) measurements and trilateration between multiple receivers [8]. While RSSI-based positioning is susceptible to multipath effects and variation in the environment, the literature identifies it as a low-cost, low-power approach suitable for indoor environments where global navigation of satellite systems is unavailable.

The document further identifies RSSI approaches that are more practical for embedded and educational applications. Similarly, existing acoustic measurement tools such as Room EQ Wizard [9] and Rational Acoustics Smart [10] demonstrate effective single-point acoustic analysis but rely on wired measurement setups and expert interpretation. These limitations motivate the exploration of distributed acoustic sensing systems that integrate spatial awareness with automated analysis and user-oriented guidance. Using Bluetooth in the way will enable us to map the SPF and frequency response of the room as well as estimate the position of the nodes. The following image shows one possible implementation of RSSI used in this project.

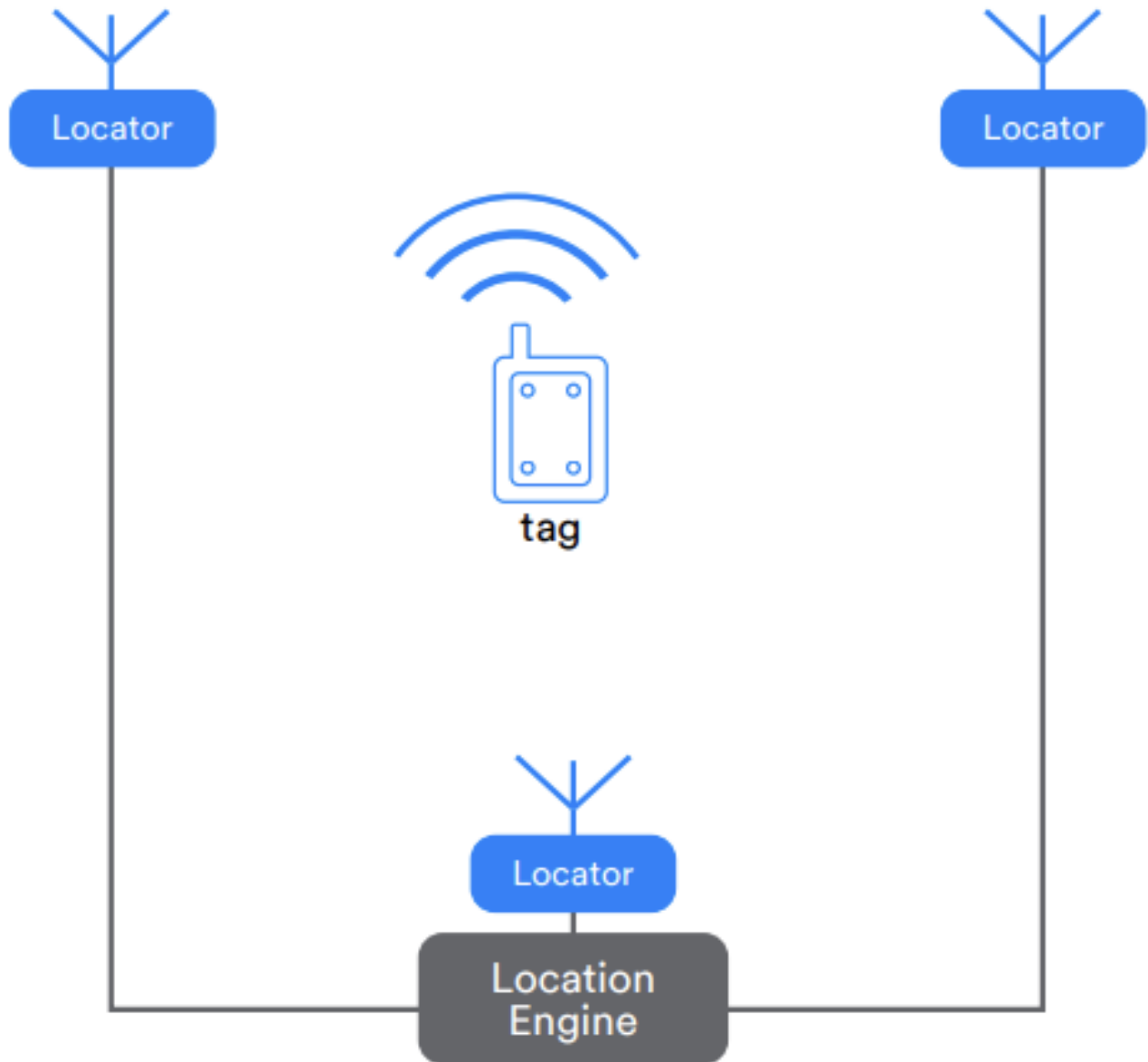


Figure 5: Bluetooth asset tracking based on RSSI and trilateration [10]

#### *2.2.4. MATLAB Modelling Libraries*

MATLAB is an application used for statistics, modelling, and other signal processing applications. MATLAB contains the following libraries and/or mathematical techniques suited for the 3-D modelling of acoustics for SDACS:

##### *2.2.4.1. 2-D Fast Fourier Transform*

The MATLAB “fft2” function operates under the assumption of a fully sampled, uniformly spaced 2-D signal and computes its discrete Fourier based on the available samples. To model the entire room, the 2-D FFT must be used with the locational data available for the nodes to determine the (incomplete) acoustics of the room.

##### *2.2.4.2. Flexible Statistics Data Analysis (FSDA) Toolbox*

The FSDA (Flexible Statistics Data Analysis) toolbox introduces a statistical framework centered on robust estimation techniques designed to operate effectively in the presence of incomplete, noisy, or contaminated data. Its methodology emphasizes identifying stable underlying structures through approaches such as robust regression and forward search, where models are iteratively built from small subsets of the data and expanded while monitoring consistency. In the context of sparse Fourier data, this corresponds to treating the available coefficients as partial observations of a broader model and using robust statistical procedures to estimate the parameters governing the full spectral or spatial representation, while reducing sensitivity to outliers or irregular sampling.

##### *2.2.4.3. Gaussian Distribution*

A Gaussian (normal) distribution model assumes that data are independent and symmetrically distributed about a mean, fully characterized by a mean and variance, with a unimodal, bell-shaped structure and no skewness or heavy tails. The primary advantages of this model are its analytical tractability, simple parameterization, and well-defined probabilistic properties, which enable efficient estimation, closed-form solutions, and straightforward confidence interval interpretation. However, its limitations arise from these same assumptions: it cannot capture skewed distributions, multimodal structures, or heavy-tailed behavior, and may perform poorly when real data deviate significantly from normality, particularly in the presence of outliers or non-symmetric patterns.

### *2.2.5. Edge Impulse TinyML*

Tiny Machine Learning (TinyML) is the deployment of machine learning models on embedded systems with limited computational resources, memory, and power consumption. Unlike conventional cloud-based machine learning systems, TinyML performs inference directly on edge devices, eliminating the need for continuous internet connectivity while significantly reducing latency and bandwidth requirements. This makes TinyML well suited for Internet of Things (IoT) applications where real-time decision making is required.

Edge Impulse is a cloud-based TinyML development platform that simplifies the creation, training, optimization, and deployment of machine learning models for embedded hardware. The platform provides an end-to-end workflow beginning with data collection and labeling, followed by signal processing, feature extraction, model training, validation, optimization, and deployment. It supports a wide variety of embedded processors including ESP32 microcontrollers, ARM Cortex-M devices, and Linux-based platforms such as the Raspberry Pi.

One of the primary advantages of Edge Impulse is its ability to automate much of the machine learning workflow while still allowing engineers to customize preprocessing blocks and neural network architectures. Rather than requiring extensive knowledge of machine learning frameworks such as TensorFlow or PyTorch, developers can construct optimized embedded models using a graphical interface. This significantly reduces development time while maintaining flexibility for custom datasets and application-specific feature engineering.

Within the SDACS project, Edge Impulse was selected to classify room acoustic environments using acoustic features generated from the distributed sensing nodes. Instead of transmitting raw audio recordings for classification, the embedded firmware extracts a set of statistical and spectral features from each recording window. These features include sound pressure level measurements, frequency-domain characteristics, spectral energy ratios, dominant frequency information, and other derived acoustic metrics. Feature extraction greatly reduces computational complexity while preserving the information required to distinguish between different acoustic environments.

The generated datasets are transmitted to the Raspberry Pi server through MQTT where Node-RED validates, labels, and stores the measurements before exporting them as CSV files. These datasets are then uploaded into Edge Impulse where they are divided into separate training and testing datasets. During model development, Edge Impulse automatically normalizes the input data, trains the neural network, evaluates classification accuracy, and produces confusion matrices and performance metrics used to assess the model.

The resulting classifier provides the SDACS system with the capability to automatically recognize different acoustic environments such as quiet rooms, rooms containing steady background noise, and environments containing speech activity. This information can later be incorporated into higher-level room analysis algorithms to recommend optimal recording locations or determine whether additional acoustic treatment may be beneficial.

Another important benefit of Edge Impulse is its support for optimized deployment packages. After training, the model can be exported as portable C++ libraries or Linux executables suitable for embedded deployment. In the SDACS architecture, the trained model is deployed to the Raspberry Pi 5 where inference is performed locally using acoustic features received from the distributed ESP32-S3 sensing nodes. Local inference minimizes latency, removes dependence on cloud connectivity, and allows the system to provide immediate feedback to the user while maintaining privacy since raw audio recordings do not need to leave the local network.

Although the final classifier achieved moderate classification accuracy, the work demonstrates the feasibility of integrating TinyML into a distributed acoustic sensing system. Future improvements such as collecting larger datasets, increasing environmental diversity, balancing class distributions, and exploring additional neural network architectures are expected to significantly improve classification performance while maintaining real-time execution on embedded hardware.

### *2.2.6. Market Research*

A big portion of the current market for acoustic measurement and calibration is split into 3 categories:

1. PC-based measurement/analysis tools
2. Calibrated measurement microphones and accessories
3. Commercial “correction” software

Our proposed smart acoustic calibration system incorporates all 3 of these categories, with the opportunity to differentiate through ease of use, guided workflows, portability, and connectivity.

A widely used reference tool is Room EQ Wizard (REW) [5]. REW supports core features that align closely with our project’s measurement goals, including generation of test signals and the ability to measure SPL, frequency response, impulse response, distortion and visualization outputs. This shows that the market already values these measurement modalities, however, REW’s workflow is largely PC-centric and it can be somewhat complex for beginners, which creates room for a system that has similar concepts but uses a simpler, easy to use UX.

On the hardware side, there is a calibrated USB measurement microphone called the miniDSP UMIK-1, which is plug-and-play for speaker and room acoustic measurements and is marketed for reliability/accuracy and avoiding OS friction [12]. For our project, this supports the idea that the end users expect measurement hardware to be simple, portable, and calibration-backed. It also highlights a key design expectation: our hardware should reduce setup complexity while still enabling accurate measurements.

For the last category, commercial correction software, Sonarworks SoundID Reference offers both a standalone app and a DAW plugin that calibrates speakers/headphones to deliver a more reliable reference sound for creators and engineers [13]. The pricing page also shows that it is available to consumers, demonstrates market demand for guided calibration improvements, and sets a quality bar for user experience and results reporting. Our project can differentiate by emphasizing education-first measurement outputs and by integrating hardware that makes data collection more approachable for beginners.

For the app layer, Flutter is an open-source framework for building natively compiled multi-platform applications from a single codebase [14], and Dart is a client-optimized language for developing fast apps across platforms. This stack aligns with market expectations for iOS and Android support without maintaining 2 separate codebases, and it enables rapid iteration on a UI that guides users through measurement steps and presents results clearly.

Overall, the market indicates clear demand for accurate measurement and calibration but also reveals a gap: many options either assume significant expertise or focus on applying correction without deeply teaching the measurement outputs. Our project can be completed by packaging professional measurement primitives into a guided, connected, beginner-friendly system while keeping workflows fast and hardware setups minimal.

### 3. PROJECT REQUIREMENTS

#### 3.1. SMART Requirements

##### 3.1.1. SMART Goal 1 – Multi-Node Embedded Acoustic System

By the end of Capstone 2, the team shall design and demonstrate a distributed acoustic measurement system consisting of a minimum of three ESP32-based nodes, each capable of simultaneously capturing digital audio at  $\geq 16$  kHz for at least 20 seconds per session, verified through controlled testing.

##### 3.1.2. SMART Goal 2 – Reliable Embedded Firmware Operation

Each node shall implement embedded firmware capable of managing audio acquisition, buffering, timing control, and wireless transmission, and operate successfully for 10 consecutive recording sessions without data loss, buffer overflow, or system reset, verified through repeated testing.

##### 3.1.3. SMART Goal 3 – Coordinated Multi-Node Recording

The system shall demonstrate coordinated multi-node recordings such that recorded audio streams can be aligned for comparative spatial analysis. Timing consistency shall be quantified and documented during validation testing.

##### 3.1.4. SMART Goal 4 – Wireless Data Aggregation and Processing

Each node shall successfully transmit recorded acoustic data wirelessly to a central Raspberry Pi 5, where received data shall be processed to generate frequency-domain analysis (FFT). Data integrity shall be verified by confirming that transmitted and received datasets are complete and consistent.

##### 3.1.5. SMART Goal 5 – Functional System Demonstration in Real Environment

A fully integrated prototype shall be demonstrated in a representative indoor environment, showing simultaneous multi-node operation and producing measurable and comparable acoustic results across spatially separated nodes during final evaluation.

#### *3.1.6. SMART Goal 6 – Verification, Validation, and Visualization*

A documented verification and validation plan shall be executed prior to project completion. All core system requirements shall be mapped to validation tests, and results shall be displayed through a web-based dashboard that presents frequency spectra and session-based comparisons during final demonstration

### 3.2. MoSCoW Requirements

Table 1: MoSCoW Summary Table

| Priority      | Requirement                                                                                         |
|---------------|-----------------------------------------------------------------------------------------------------|
| <b>MUST</b>   | Enable simultaneous acoustic measurements from multiple indoor locations.                           |
| <b>MUST</b>   | Utilize ESP32-based nodes with digital MEMS microphones for reliable audio capture.                 |
| <b>MUST</b>   | Support coordinated multi-node recording for spatial comparison.                                    |
| <b>MUST</b>   | Capture, buffer, and wirelessly transmit audio data to a Raspberry Pi 5 for centralized processing. |
| <b>MUST</b>   | Perform frequency-domain (FFT-based) analysis on recorded data.                                     |
| <b>MUST</b>   | Provide a web-based dashboard to display measurement results.                                       |
| <b>MUST</b>   | Demonstrate a functional prototype in a representative indoor environment.                          |
| <b>MUST</b>   | Execute structured verification and validation testing to confirm system performance.               |
| <b>SHOULD</b> | Allow remote configuration and control of measurement nodes.                                        |
| <b>SHOULD</b> | Store measurement data for post-processing and session comparison.                                  |
| <b>SHOULD</b> | Implement automated acoustic metrics such as RT60 estimation.                                       |
| <b>COULD</b>  | Include environmental sensing (e.g., temperature, humidity).                                        |
| <b>COULD</b>  | Perform limited on-node signal processing to reduce data transmission load.                         |
| <b>COULD</b>  | Support scalability to additional nodes and third-party tool integration.                           |
| <b>WON'T</b>  | Perform professional audio production or replace commercial acoustic systems.                       |
| <b>WON'T</b>  | Include active acoustic treatment hardware or support industrial-scale deployment.                  |

# 4. SYSTEM DESIGN

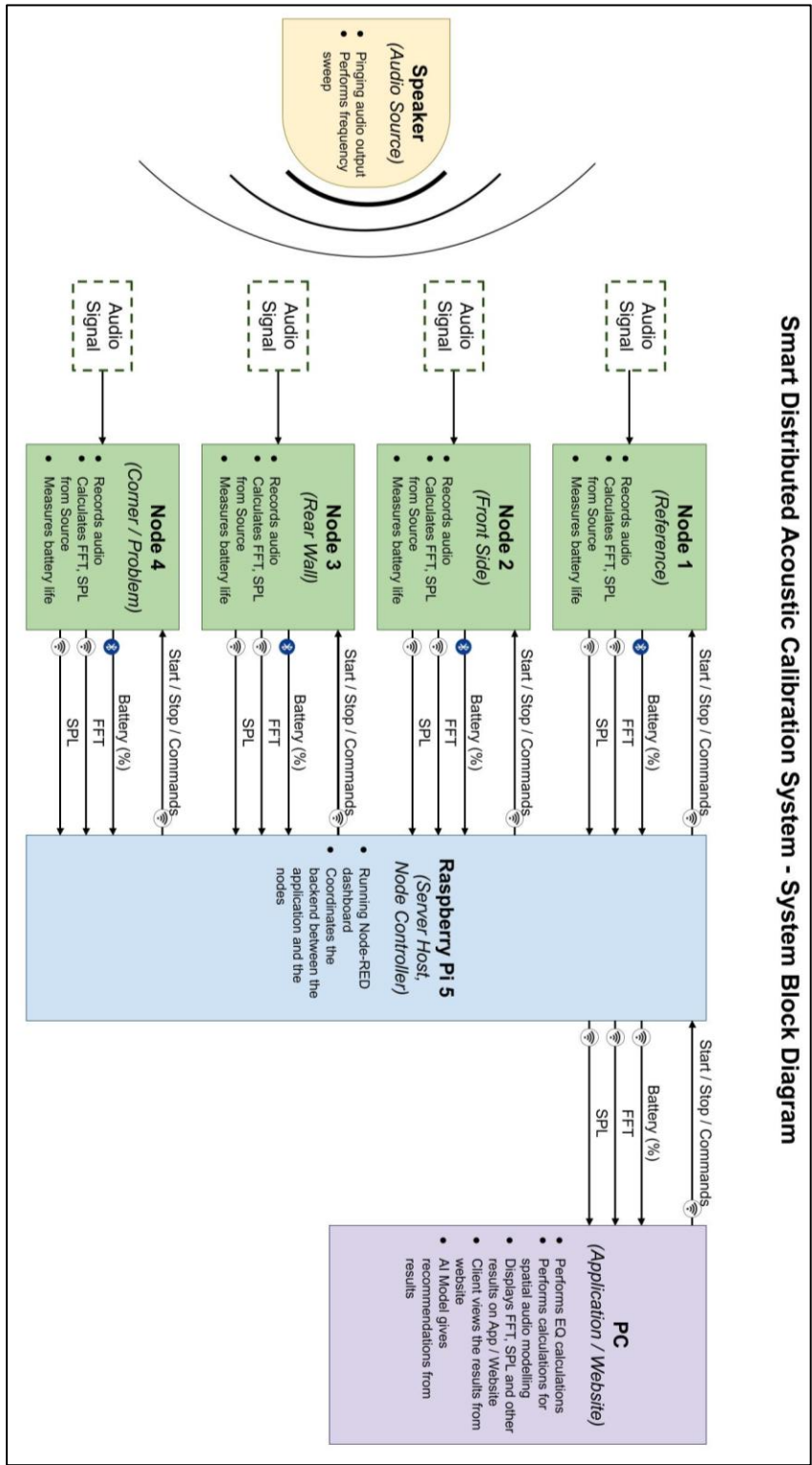


Figure 6: SDACS System Block Diagram

#### *4.1. Functional Units*

##### *4.1.1. Recording Nodes*

The recording nodes perform the following main functions:

- i. Records audio metrics, temperature and humidity, and battery life locally.
- ii. Stores all measured data on mounted SD Card.
- iii. Sends FFT, RT60, SPL, temperature/humidity and battery life to Server via Wi-Fi.

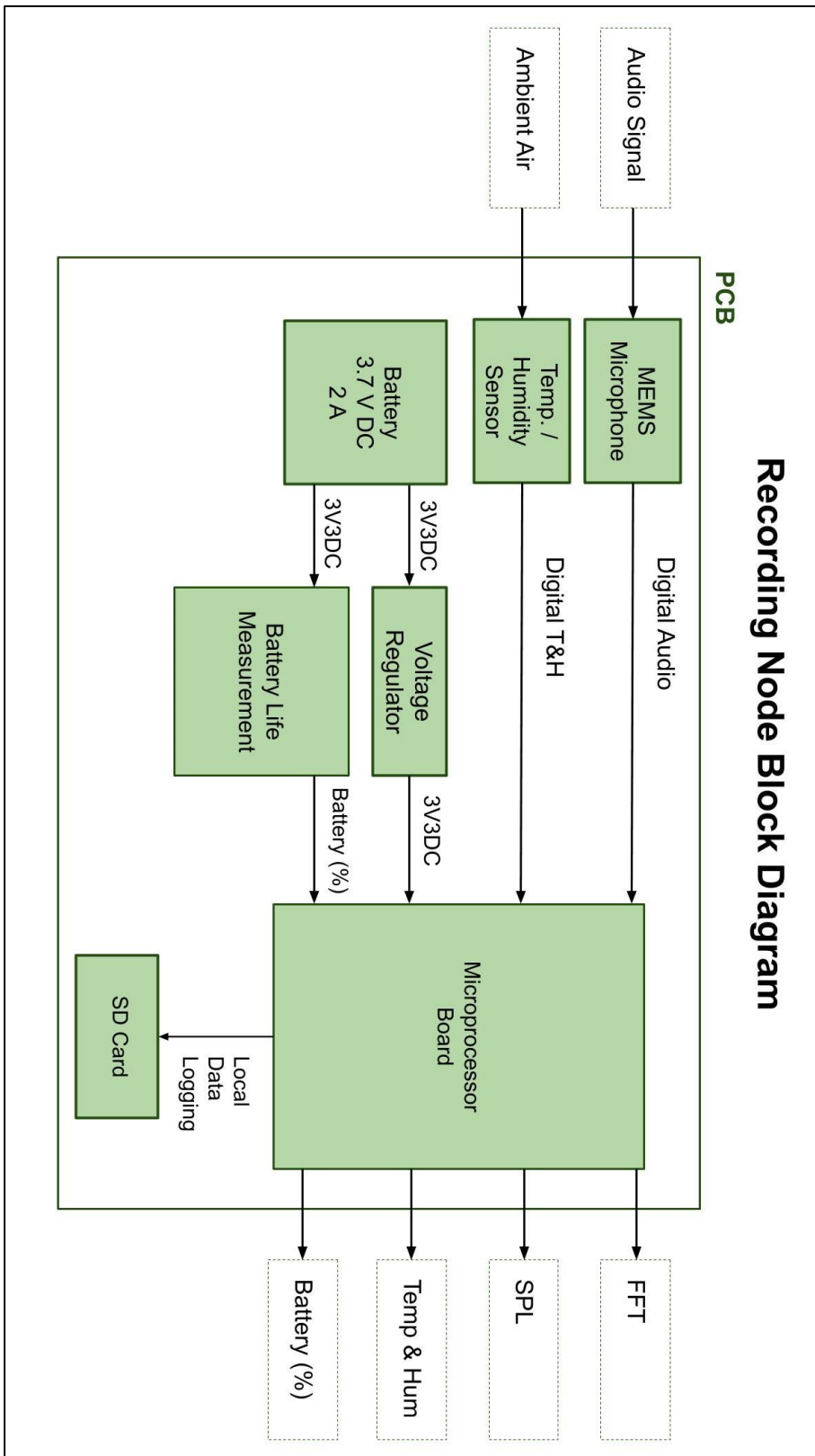


Figure 7: SDACS Recording Node Block Diagram

#### 4.1.2. *Server*

The server performs the following main functions:

- i. Hosts Node-RED at <broker\_ip>:port, and subscribes to recording nodes' MQTT topics.
- ii. Synchronizes the recording nodes to initiate simultaneous capture.
- iii. Sends commands including firmware updates and capture initialization to the recording nodes.
- iv. Receives data (i.e. RMS, FFT, temperature and humidity), heartbeats, OTA requests, etc. from recording nodes over Wi-Fi via MQTT topics.
- v. Performs analysis on the data received (converts from JSON to SQLite and CSV).
- vi. Sends updated data models, statistics, etc. to the Application and MATLAB.

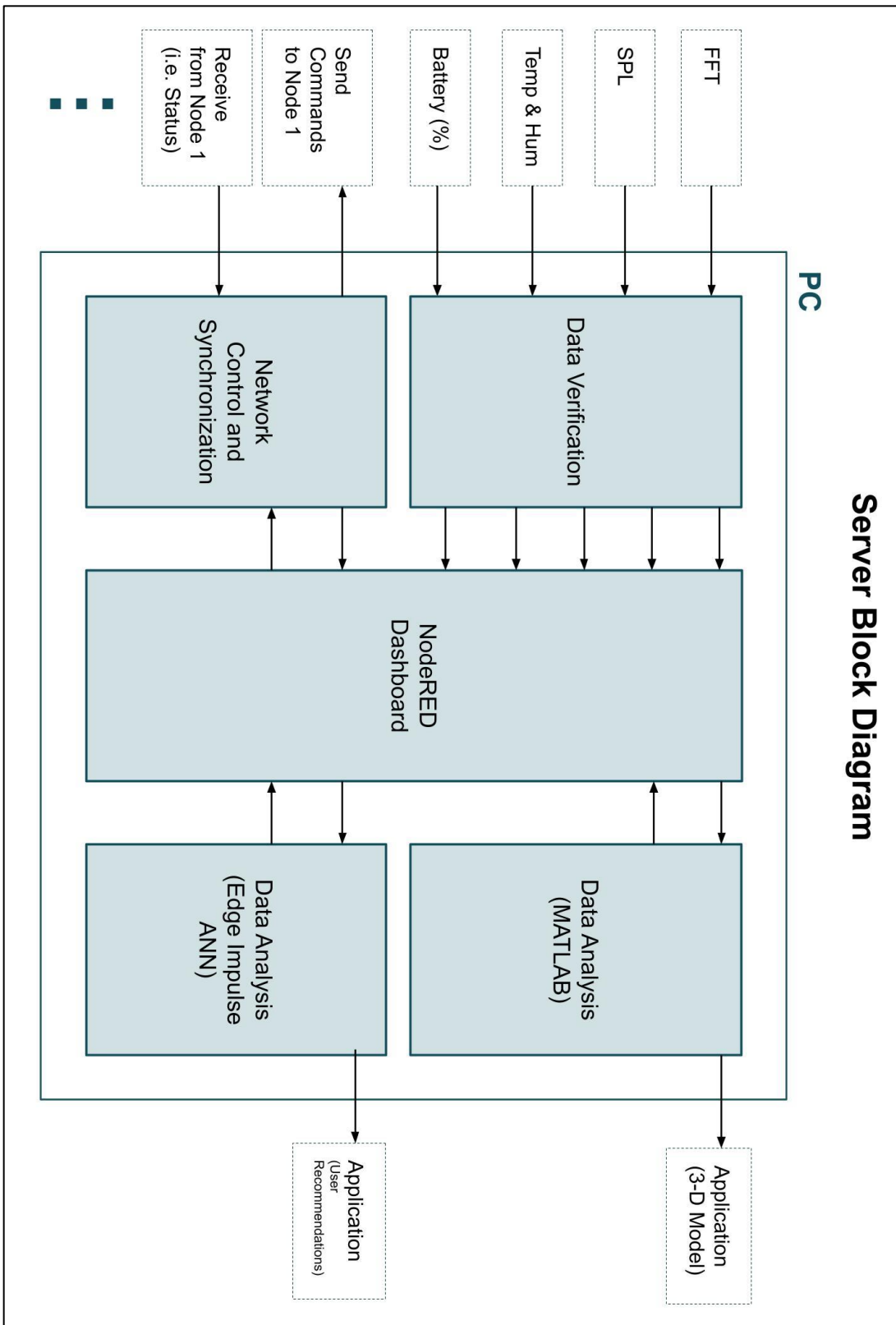


Figure 8: SDACS Server Block Diagram

#### *4.1.3. Application*

The application performs the following functions:

- i. User Interface
- ii. Initiates firmware updates
- iii. Sends user commands to the Server
- iv. Displays results of acoustic analysis using graphs, tables, etc.

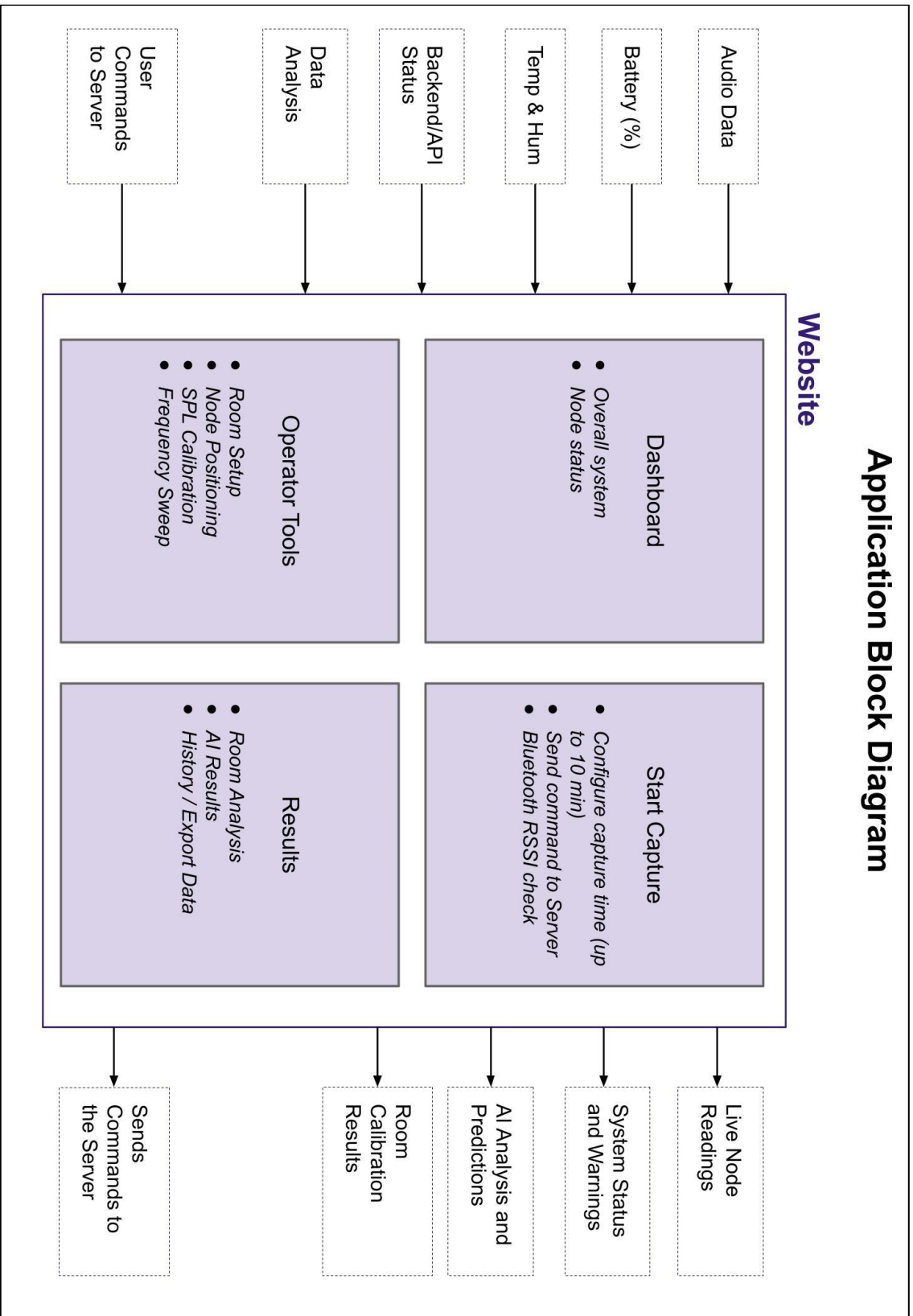


Figure 9: SDACS Application Block Diagram

#### *4.2. Test Plans*

The SDACS system was validated through a combination of calibration testing and function system testing, targeting both individual firmware branches and overall system integration.

The test plans were designed to verify the following SMART Goals:

- i. SMART Goal 1 – Multi-Node Embedded Acoustic System
- ii. SMART Goal 2 – Reliable Embedded Firmware Operation
- iii. SMART Goal 3 – Coordinated Multi-Node Recording
- iv. SMART Goal 4 – Wireless Data Aggregation and Processing
- v. SMART Goal 5 – Functional System Demonstration in Real Environment

Testing was divided into the following sections:

- i. Software
- ii. Firmware
- iii. Hardware

#### *4.2.1. Hardware Test Plan*

The Hardware Test Plan verifies the hardware connections on the custom PCB to the MCU. Using a multimeter and a PC, the following functionality will be tested to confirm the system meets the requirements outlined in Sections 3.1 and 3.2:

1. IC Pinouts
2. Power Verification

#### *4.2.2. Firmware Test Plan*

The Firmware Test Plan verifies the acoustic captures are running on the nodes, and that the data is being properly sent to the server backend (Node-RED). Using the custom PCB, the MCU, and a PC, the following functionality will be tested to confirm the system meets the requirements outlined in Sections 3.1 and 3.2:

1. Network / Node-RED Compatibility
2. Data Logging (Audio, Temperature and Humidity)
3. SD Card Logging (Local)
4. BLE-Scan (Awareness) on Node-RED
5. Start Group Capture on Node-RED
6. Download CSV on Node-RED

#### *4.2.3. Software Test Plan*

The Software Test Plan verifies the Frontend to Backend Integration for the application made using Flutter and FastAPI. Using the application website, the following functionality will be tested to confirm the system meets the requirements outlined in Sections 3.1 and 3.2:

1. Page navigation
2. Main dashboard layout
3. Node status display
4. Calibration control buttons

5. Sensor data display
6. Backend API communication
7. Error handling
8. User interface responsiveness

### *4.3. Off-the-shelf Components*

#### *4.3.1. Node-RED*

Node-RED was implemented on the Raspberry Pi 5 as the primary message-routing, control, data-logging, and operator-interface layer of SDACS. The Node-RED environment subscribes to telemetry published by the four ESP32 sensing nodes through the Mosquitto MQTT broker. Incoming messages are parsed, normalized, displayed through the Node-RED Dashboard, and stored in CSV or SQLite formats. The same environment also publishes synchronized capture commands, BLE discovery commands, and dataset labels. This approach provides a visual and modular middleware layer between the embedded sensing nodes and higher-level SDACS components such as the FastAPI backend, Flutter application, graphical analysis, and Edge Impulse classifier.

Key Features:

- i. Multi-node compatibility
- ii. Easily Scalable for Multiple Nodes (Devices)
- iii. Easy integration with Raspberry Pi Broker
- iv. Data logging for post processing in MATLAB

Engineering Justification:

Node-RED was chosen because it provides rapid prototyping for IoT devices, real-time MQTT integration, a simple and modular (expandable) server design, and a user-friendly visual interface.

#### *4.3.2. Microcontroller, Adafruit Metro S3*

The Adafruit Metro S3 is a development board based on the ESP32-S3 microcontroller, designed for embedded systems and IoT applications. It integrates Wi-Fi (2.4 GHz) and Bluetooth 5.0 connectivity, making it suitable for wireless communication in distributed sensor networks. The board includes built-in USB support, voltage regulation, battery charging circuitry, and a microSD card interface, allowing for both data acquisition and storage in a compact form factor.

The Metro S3 is used in the SDACS recording nodes to perform real-time acoustic signal acquisition, sensor integration, and wireless data transmission. Its processing capabilities allow for on-node signal processing, including RMS, dBFS, SPL estimation, and FFT calculations.

##### Key Features:

- i. ESP32-S3 Dual-Core Processor: Provides sufficient computational power for real-time signal processing and multitasking
- ii. Integrated Wi-Fi and Bluetooth: Enables wireless communication with the central server for data transmission and node synchronization
- iii. I2S Peripheral Support: Allows direct interfacing with digital MEMS microphones for accurate audio capture
- iv. I2C Interface: Supports environmental sensors such as temperature and humidity sensors on a shared bus
- v. MicroSD Card Support: Enables local storage of recorded data for redundancy and post-processing
- vi. Battery Management System: Includes onboard Li-Po charging and fuel gauge support for portable operation

##### Engineering Justification:

The Adafruit Metro S3 was selected due to its combination of wireless connectivity, processing capability, and integrated peripherals. Its compatibility with ESP-IDF and Arduino frameworks simplifies firmware development, while its compact design and built-in battery support make it ideal for distributed, portable sensing nodes.

#### *4.3.3. Digital MEMS Microphone, ICS-43432*

The ICS-43432 is a high-performance digital MEMS microphone designed for precision audio capture. It outputs a digital pulse-density modulated (PDM) signal that is converted internally to I2S-compatible data for direct interfacing with the ESP32-S3.

Key Features:

- i. High signal-to-noise ratio (~64 dBA) for accurate acoustic measurements
- ii. Wide frequency response suitable for audio analysis
- iii. Low distortion and stable sensitivity
- iv. Digital output reduces susceptibility to electromagnetic interference

Engineering Justification:

The use of a digital I2S microphone eliminates the need for analog front-end circuitry (e.g., amplifiers, ADCs), significantly simplifying PCB design while improving noise immunity. This is critical for accurate FFT and SPL calculations in electrically noisy embedded environments.

#### *4.3.4. LiPo Battery, 3.7V with MAX17048 Fuel Gauge*

Each node is powered by a 3.7V lithium-polymer (Li-Po) battery, enabling fully wireless and portable deployment. The MAX17048 Fuel Gauge IC monitors the battery life which can be extracted by the MCU in the form of a percentage.

Key Features:

- i. State-of-charge (SOC) estimation
- ii. Battery voltage monitoring
- iii. Charge/discharge rate tracking
- iv. I2C communication interface

Engineering Justification:

Accurate battery monitoring is critical for distributed systems to prevent node failure during measurement cycles. The MAX17048 enables predictive monitoring and reporting of battery status to the central server.

#### *4.3.5. Temperature and Humidity Sensor, Sensirion SHT41*

The SHT41 is a high-precision digital sensor used to measure ambient temperature and relative humidity.

Key Features:

- i. High accuracy ( $\pm 0.2^{\circ}\text{C}$  temperature,  $\pm 1.8\%$  RH typical)
- ii. Fast response time
- iii. Fully calibrated digital output
- iv. Low power consumption

Engineering Justification:

Environmental conditions such as temperature and humidity directly affect sound propagation and air density. Including the SHT40 enables contextual calibration of acoustic measurements and supports future compensation algorithms.

#### 4.3.6. *PC, Raspberry Pi 5*

The Raspberry Pi 5 is a single-board computer used as the central server in the SDACS system. It provides sufficient processing power and networking capability to manage communication between distributed nodes, perform backend data processing, and host the Node-RED server and web-based application.

The Raspberry Pi 5 operates on a Linux-based operating system and supports multiple communication protocols, including Wi-Fi, Ethernet, and Bluetooth. In this project, it functions as the MQTT broker host, data processing unit, and interface between the hardware system and the user-facing application.

Key Features:

- i. Quad-Core ARM Cortex Processor: Enables efficient handling of multiple processes, including data aggregation and analysis
- ii. Wi-Fi and Ethernet Connectivity: Provides reliable network communication with distributed nodes
- iii. GPIO and Peripheral Support: Allows integration with additional hardware if required
- iv. Large Memory Options: Supports efficient handling of data logging, database management, and real-time processing
- v. Linux OS Support: Enables use of Node-RED, MQTT broker services, and database tools such as SQLite
- vi. USB and Storage Interfaces: Supports external devices and storage for data logging and system expansion

Engineering Justification:

The Raspberry Pi 5 was selected due to its balance of performance, cost, and flexibility. It is widely supported, easy to configure, and capable of running the required backend services for SDACS, including MQTT communication, Node-RED dashboards, and data storage systems. Its ability to handle multiple networked nodes simultaneously makes it suitable for centralized system coordination.

#### *4.3.7. M3 Screws and Standoffs (OEM)*

A variety set of M3 screws at different lengths were chosen as a suitable screw for mounting the Adafruit Metro within the enclosure. A 5mm M3 screw insert was also chosen as a compatible insert for the screws within the enclosure.

#### *4.4. Safety, Societal and Environmental Issues*

##### *4.4.1. Safety Considerations*

The design of the system incorporates several safety considerations to ensure reliable and secure operation. The use of a 3.3 V low-voltage system minimizes the risk of electrical hazards to users and reduces the likelihood of component damage due to overvoltage conditions. Additionally, current-limiting resistors are implemented for all LED indicators to prevent excessive current draw and overheating.

The system is powered using a rechargeable lithium polymer battery (EEMB 3.7 V, 1100 mAh), which provides a stable and portable power source. Proper charging and protection circuitry must be used to prevent overcharging, deep discharge, and short-circuit conditions, which are critical for lithium-based batteries. The inclusion of a power switch also allows users to safely disconnect power when the device is not in use.

The use of digital sensors such as the MEMS microphone and temperature/humidity sensors reduces the need for high-voltage analog circuitry, improving overall system safety and robustness.

##### *4.4.2. Societal Considerations*

The primary societal impact of this project is its application in supporting music producers and audio engineers in studio environments. Accurate measurement and analysis of sound pressure levels (SPL) and room acoustics are essential for producing high-quality audio. However, professional acoustic measurement tools can be expensive and inaccessible to many users. This system provides a cost-effective and portable solution that enables users to analyze their acoustic environment, identify sound inconsistencies, and optimize their mixing and recording spaces. By improving access to acoustic measurement tools, the system supports creativity, productivity, and quality within the music production industry.

In addition to professional studio applications, the system can also benefit educational and community environments, including learning centers and spaces designed for individuals with sensory sensitivities or cognitive challenges. In these settings, excessive noise can negatively affect concentration, comfort, and overall well-being. By providing feedback on environmental

noise levels, the system can help maintain controlled and quieter environments, improving accessibility and inclusivity.

#### *4.4.3. Environmental Considerations*

Environmental impact was considered in the selection of components and system design. The use of a rechargeable lithium polymer battery significantly reduces electronic waste compared to disposable batteries, as it can be reused over many charge cycles. Additionally, the chosen components, including the MEMS microphone and digital sensors, are highly integrated, reducing the total number of discrete components required. This minimizes material usage and simplifies manufacturing, which contributes to a lower environmental footprint.

The system is also designed with power efficiency in mind. By using low-power components such as the ESP32-S3 microcontroller and efficient LED indicators, overall energy consumption is reduced, extending battery life and decreasing the frequency of charging.

## 5. DESIGN OF FUNCTIONAL UNITS

### 5.1. Hardware

The hardware system consists of a compact, battery-powered embedded platform designed for distributed acoustic data acquisition, environmental sensing, and wireless communication. Each SDACS recording node integrates sensing, processing, storage, and communication hardware into a single portable unit capable of synchronized operation across a network.

#### 5.1.1. Hardware Design

Each recording node consists of the following primary hardware components:

- i. Microcontroller – Adafruit Metro S3 (ESP32-S3)
- ii. Digital MEMS microphone – ICS-43434 MEMS Microphone (I2S interface)
- iii. Temperature and humidity sensor – Sensirion SHT40 (I2C interface)
- iv. Battery system – 3.7V Li-Po battery with MAX17048 Fuel Gauge IC
- v. MicroSD card interface for local data storage
- vi. Custom-designed PCB for system integration

These components form a self-contained sensing node capable of collecting, processing, and transmitting acoustic and environmental data.

### 5.1.2. Inputs and Outputs

Table 2: Hardware Inputs and Outputs

| Inputs                                                    | Outputs                                                                  |
|-----------------------------------------------------------|--------------------------------------------------------------------------|
| <b>3.7V Li-Po battery power</b>                           | Regulated power supplied to the ESP32-S3 and peripheral components       |
| <b>Acoustic pressure at the ICS-43432 MEMS microphone</b> | I2S digital audio signal to the ESP32-S3                                 |
| <b>Ambient temperature and humidity</b>                   | I2C temperature and relative-humidity data                               |
| <b>Battery voltage and state-of-charge</b>                | I <sup>2</sup> C battery voltage and percentage data from the fuel gauge |
| <b>ESP32-S3 GPIO control signals</b>                      | Battery-status LED indications                                           |
| <b>ESP32-S3 clock and word-select signals</b>             | Synchronized MEMS microphone audio output                                |
| <b>MicroSD card read/write commands</b>                   | Locally stored measurement and system data                               |
| <b>Wi-Fi and Bluetooth radio signals</b>                  | Wireless MQTT communication and BLE advertisements                       |
| <b>USB power / charging input</b>                         | Battery charging and wired programming/debugging connection              |

### 5.1.3. Schematics

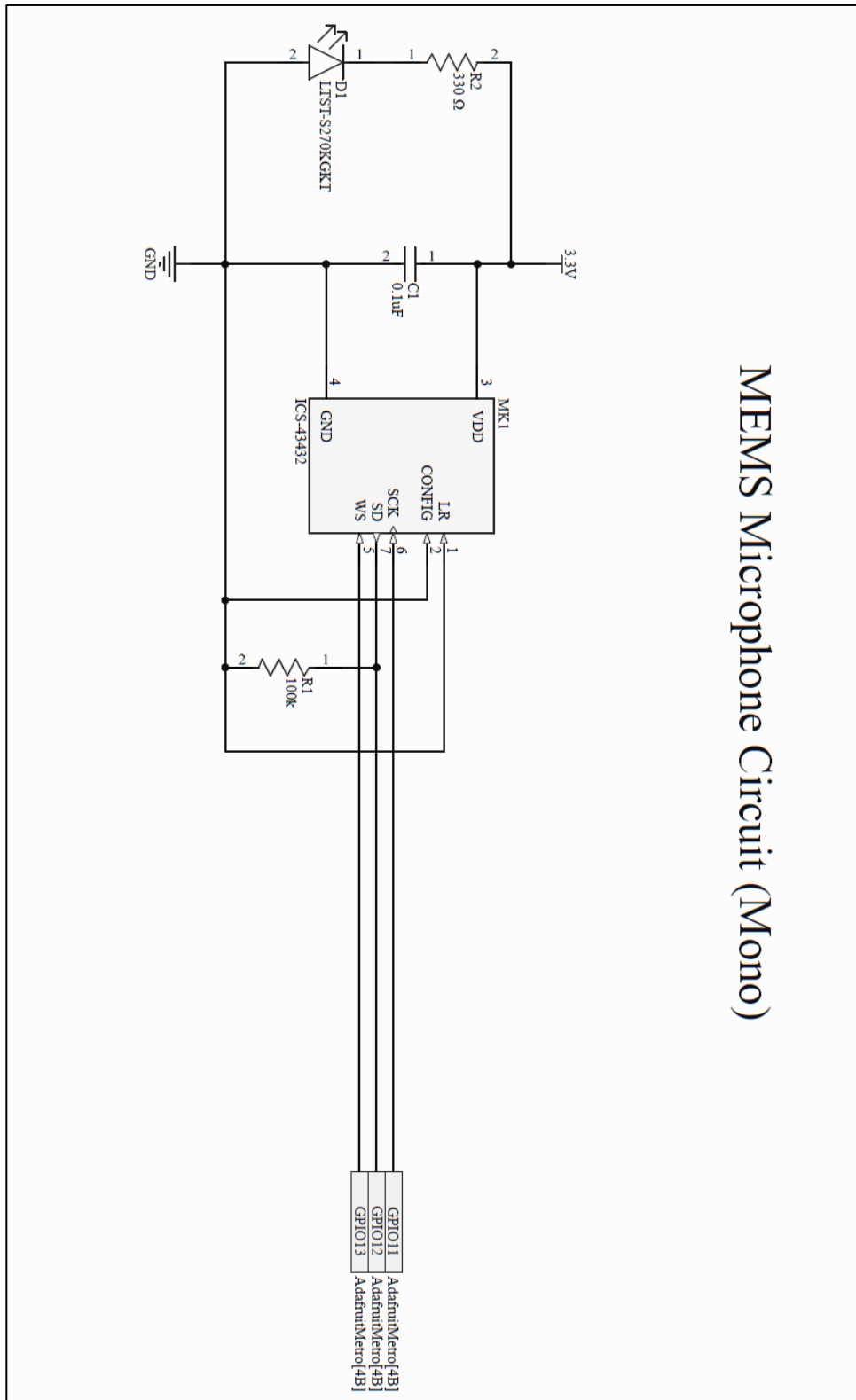


Figure 10: MEMS Microphone Schematic

# Temperature/Humidity Sensor

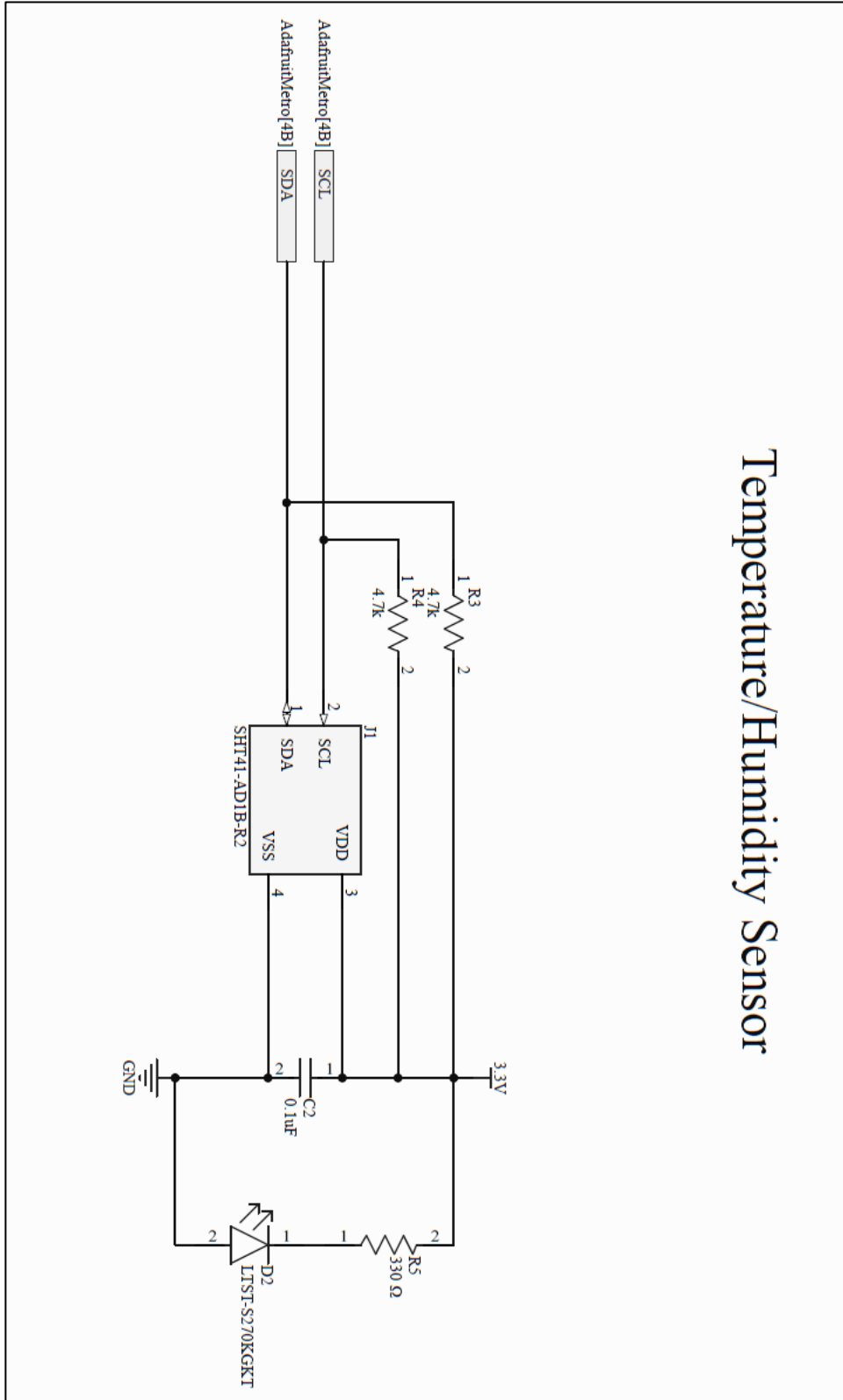


Figure 11: Temperature and Humidity Sensor Schematic

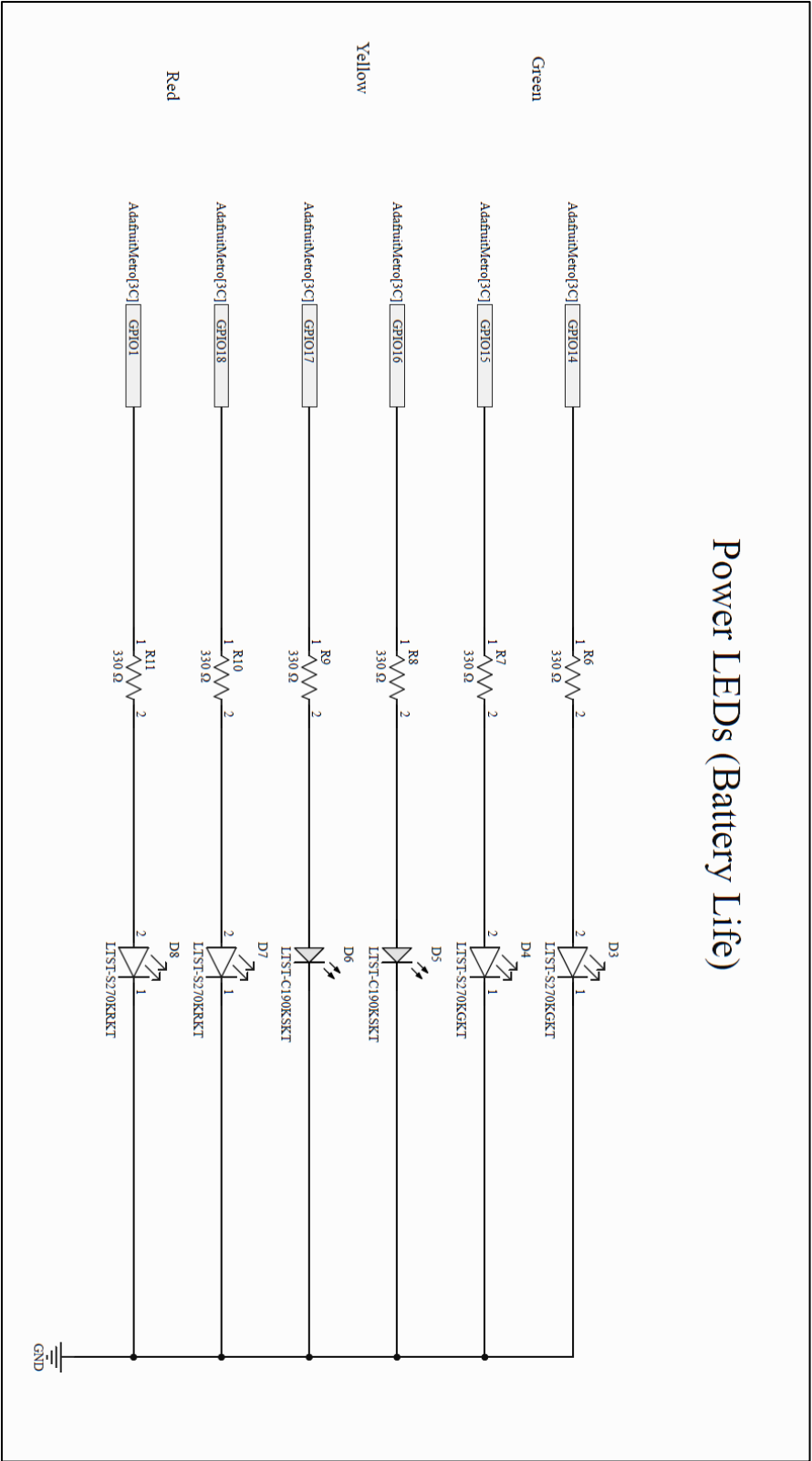
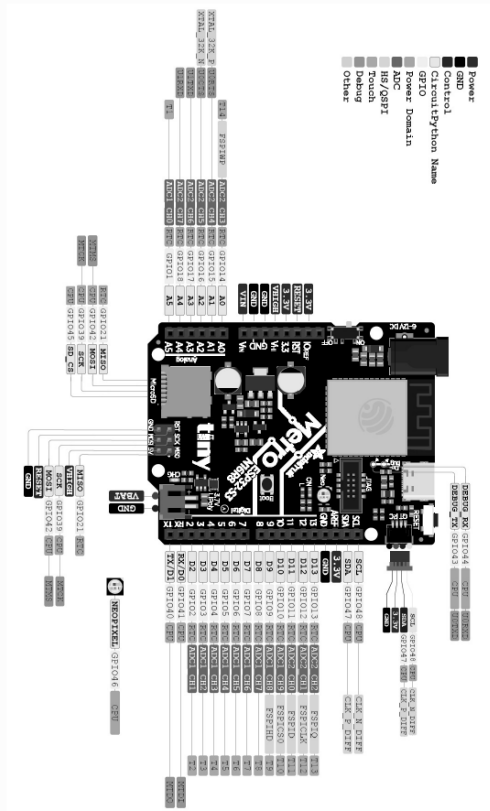


Figure 12: Power LED Array Schematic

## Metro Board Pinout (Headers)



#### *5.1.4. Custom PCB Design*

A custom PCB was designed to integrate all system components into a compact and reliable hardware platform.

##### PCB Functions:

- i. Distributes power from the battery to all components
- ii. Routes communication signals (I2S, I2C, SPI)
- iii. Provides mechanical mounting for sensors and connectors
- iv. Design Features:
  - v. Dedicated I2S trace routing for microphone signal integrity
  - vi. Shared I2C bus with proper pull-up resistors
  - vii. Decoupling capacitors placed near power pins for noise reduction
  - viii. Compact layout optimized for enclosure constraints

##### Engineering Considerations:

- i. Signal Integrity: Careful routing of high-speed I2S lines to minimize noise and jitter
- ii. Grounding Strategy: Common ground plane to reduce electromagnetic interference
- iii. Power Stability: Decoupling and filtering to ensure stable operation under dynamic loads
- iv. Modularity: Header pins included for debugging and expansion

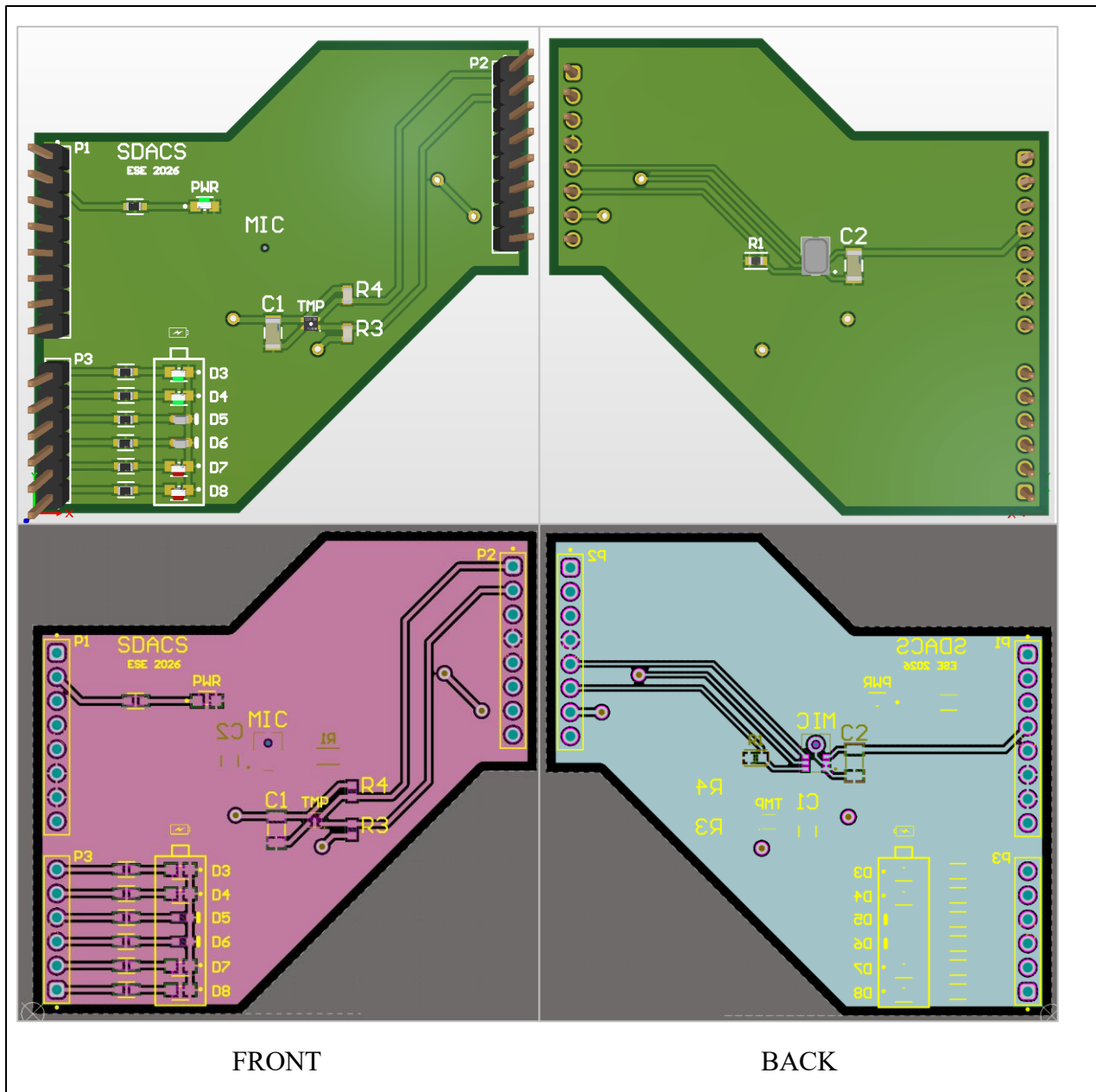


Figure 14: PCB Design for Manufacturing, 3-D vs. 2-D Views in Altium

## *5.2. Firmware*

### *5.2.1. Firmware Design*

The MEMS microphone audio is sampled at 48 kHz and stored in processing buffers. Software gain and a high-pass filter condition the signal before the firmware calculates SPL, FFT, peak frequency, and frequency-band energies. These results are packaged and published to the Raspberry Pi through MQTT, while backend commands can start synchronized captures and receive completion messages. This is shown in the following flow chart:

# SDACS Firmware Audio Processing Flow

HPF and Gain Added to Existing Feature Extraction and MQTT Streaming Pipeline

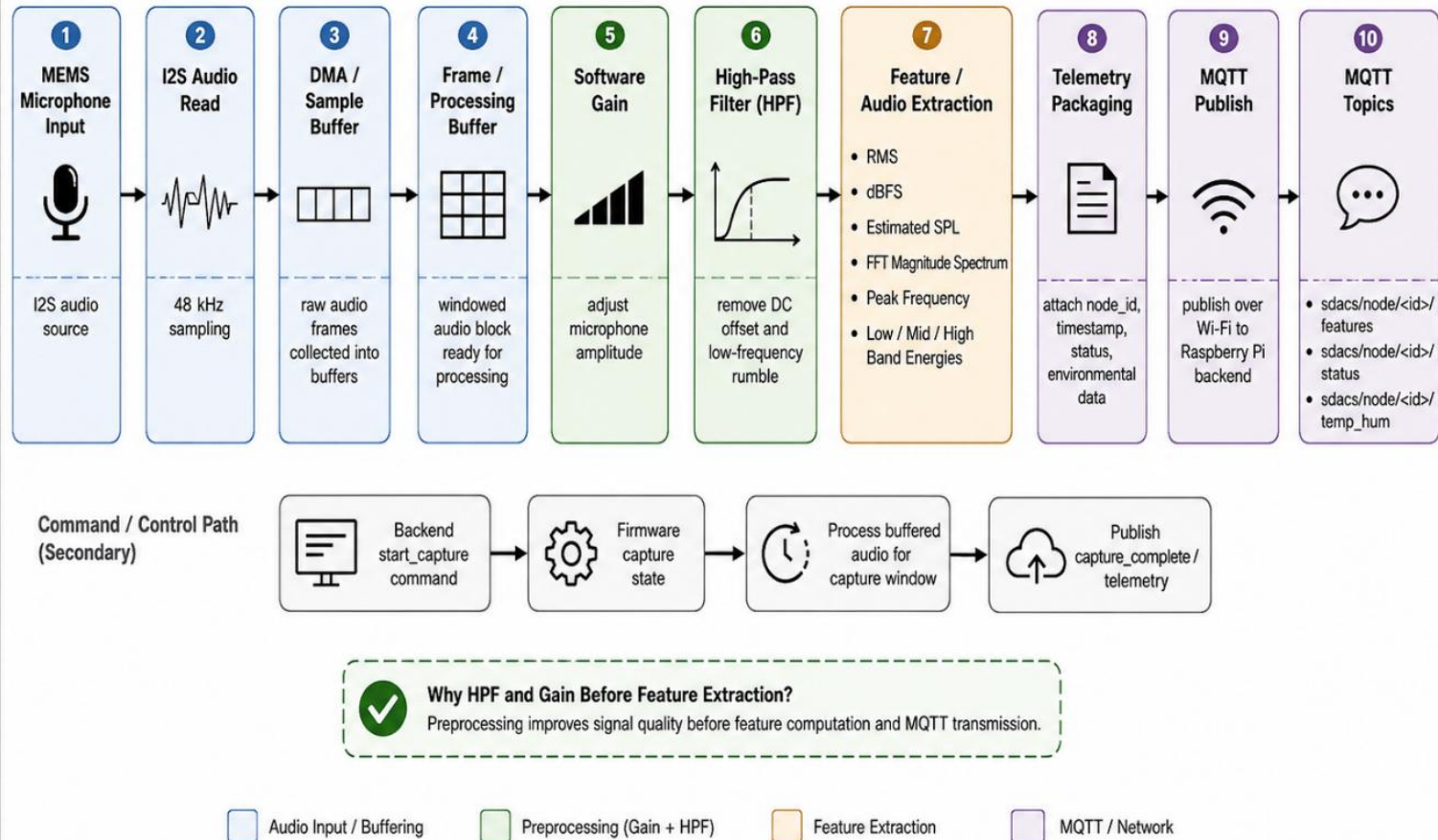


Figure 15: Audio Processing Flowchart Showing I2S to MQTT Pathway

### 5.2.2. Firmware Inputs and Outputs

Table 3: Firmware Inputs and Outputs

| Inputs                                          | Outputs                                                             |
|-------------------------------------------------|---------------------------------------------------------------------|
| I2S digital audio from the MEMS microphone      | Gain-adjusted and high-pass-filtered audio samples                  |
| Software gain and HPF configuration values      | RMS amplitude and dBFS                                              |
| SPL calibration reference and stored offset     | Estimated SPL                                                       |
| Audio sampling parameters and timing signals    | FFT peak frequency                                                  |
| Processed audio samples                         | Low-, mid-, and high-frequency band energies                        |
| PC temperature and humidity sensor data         | Temperature and humidity                                            |
| Battery/fuel-gauge sensor data                  | Battery voltage and percentage                                      |
| MQTT capture, calibration, and control commands | Capture status, acknowledgements, and completion messages           |
| Node configuration and firmware settings        | MQTT telemetry, node status, heartbeat, and firmware-version topics |

### 5.2.3. Firmware Process Flow

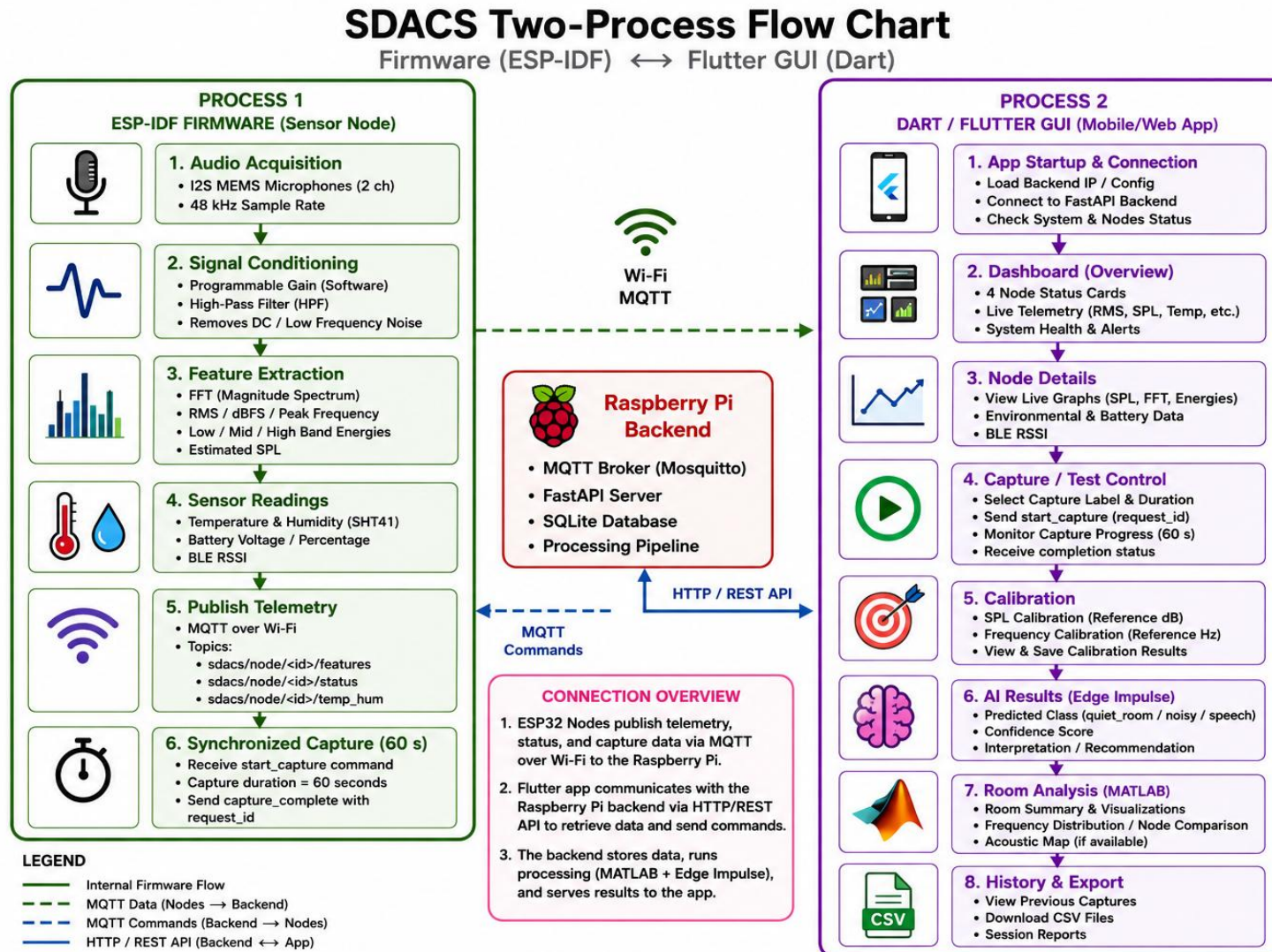
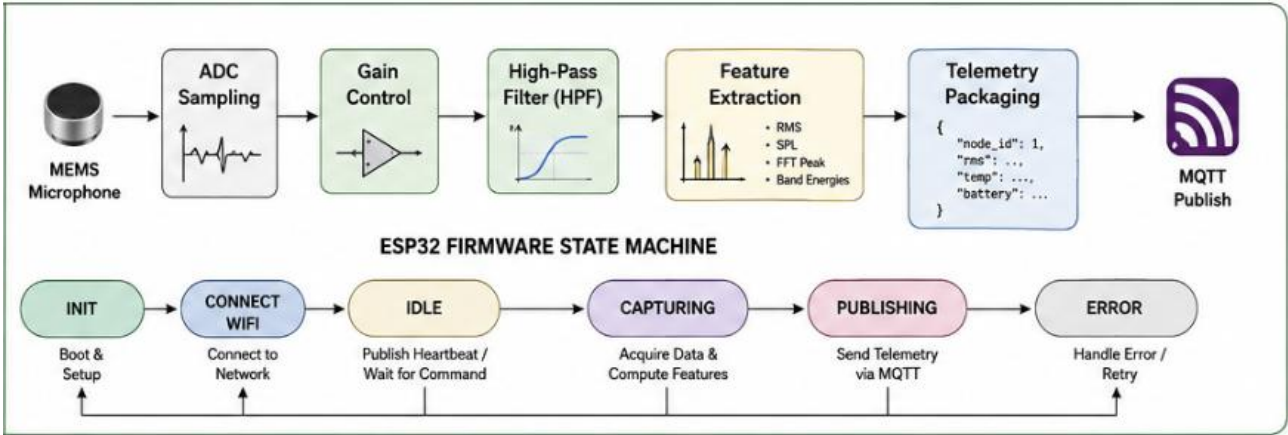
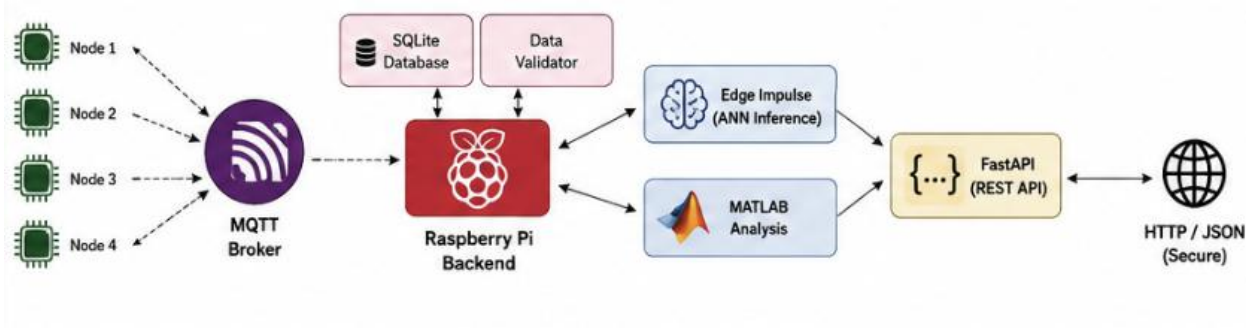


Figure 16: SDACS firmware and Flutter GUI process flow with MQTT and REST API communication.

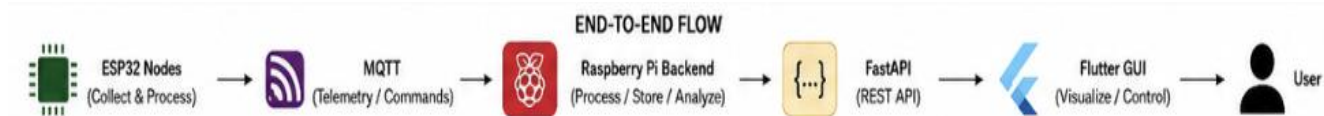
**Embedded processing:** The ESP32 firmware performs audio capture, gain adjustment, high-pass filtering, feature extraction, and sensor monitoring before sending compact telemetry. This reduces network traffic and ensures cleaner data is used for FFT, SPL, and frequency-band calculations.



**Backend communication:** MQTT connects the ESP32 nodes to the Raspberry Pi for telemetry, commands, and synchronized captures. The Raspberry Pi validates and stores the data, coordinates the four nodes, and provides Edge Impulse and MATLAB results through the FastAPI backend.



**GUI workflow:** Flutter communicates with the backend through HTTP/REST and presents the system in a logical sequence: connection, dashboard, node details, capture, calibration, AI results, room analysis, and export. This layered design keeps the firmware, backend, and GUI modular and easier to test, troubleshoot, and demonstrate



### 5.3. Middleware / Software

#### 5.3.1. Server Design

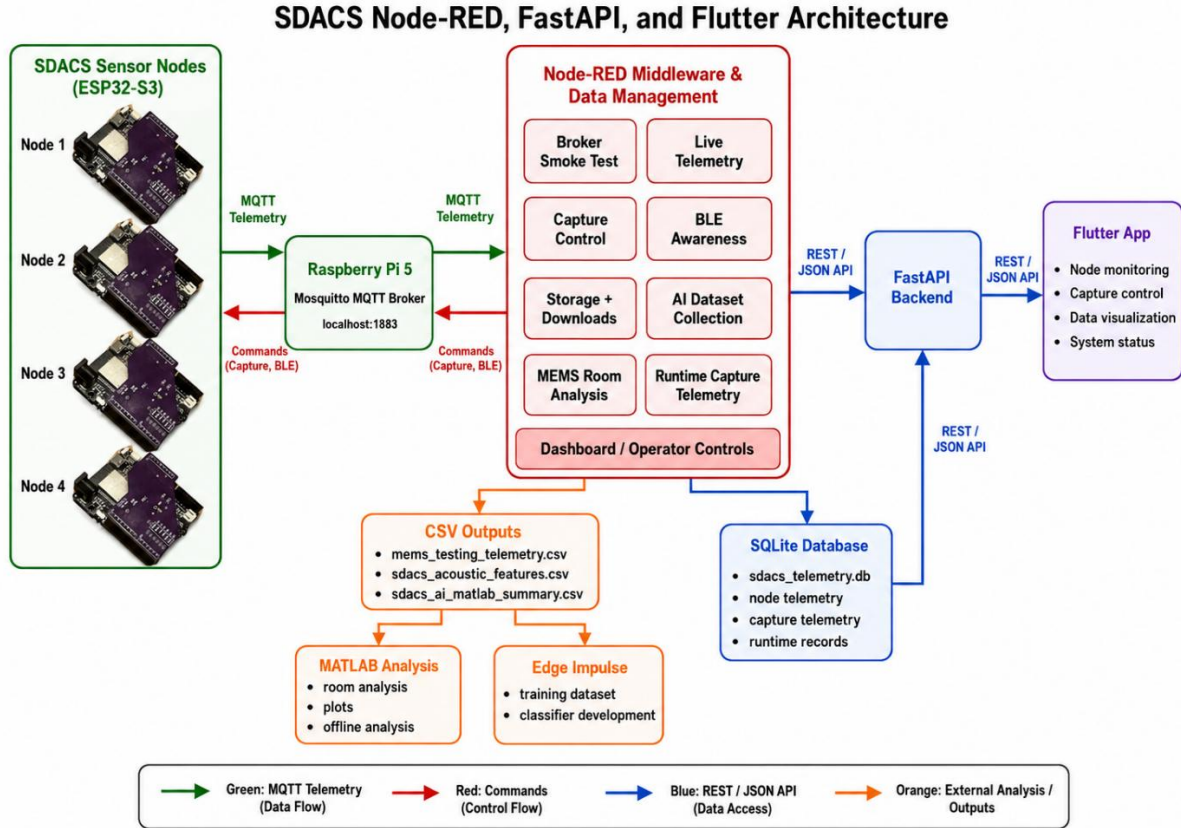


Figure 17: Summarized SDACS software data-flow architecture with MQTT communication between the distributed sensor nodes and Raspberry Pi, Node-RED telemetry processing and storage, FastAPI backend communication, and the Flutter operator interface.

Node-RED is responsible for receiving all the telemetry (sensor) data from the nodes via MQTT protocol. It processes and formats data into JSON, CSV and SQLite database content and provides real-time data through a visual dashboard interface.

### 5.3.2. Inputs and Outputs

Table 4: Node-RED Inputs and Outputs

| Type          | Data or function                                                                            | MQTT topic / interface                                   | Node-RED output or destination                                                           |
|---------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>Input</b>  | Acoustic features, including dBFS, estimated SPL, peak frequency, and frequency-band values | sdacs/node/+/features                                    | Live dashboard, AI/MEMS CSV files, SQLite database, and backend data                     |
| <b>Input</b>  | Temperature and relative humidity                                                           | sdacs/node/+/temp_humidity                               | Live dashboard and persistent telemetry storage                                          |
| <b>Input</b>  | Battery percentage, voltage, and fuel-gauge status                                          | sdacs/node/+/fuel_gauge                                  | Node-health dashboard and SQLite telemetry records                                       |
| <b>Input</b>  | Node heartbeat and connection status                                                        | sdacs/node/+/status/heartbeat                            | Node-health display, last-seen time, and storage record                                  |
| <b>Input</b>  | Capture state and capture-complete events                                                   | sdacs/node/+/status and<br>sdacs/node/+/capture_complete | Capture-status tables and capture-linked SQLite records                                  |
| <b>Input</b>  | BLE presence and topology information (RSSI)                                                | sdacs/node/+/presence and<br>sdacs/site/topology         | BLE Awareness dashboard and node-presence records                                        |
| <b>Output</b> | Synchronized capture command                                                                | sdacs/group/all/cmd                                      | Starts a coordinated recording on all four nodes                                         |
| <b>Output</b> | BLE advertising and gateway scan commands                                                   | sdacs/group/all/cmd and<br>sdacs/ble/gateway/rpi5/cmd    | Initiates node advertising and Raspberry Pi BLE scanning                                 |
| <b>Output</b> | Downloadable datasets                                                                       | HTTP download endpoints                                  | mems_testing_telemetry.csv, sdacs_acoustic_features.csv, and sdacs_ai_matlab_summary.csv |
| <b>Output</b> | Persistent system data                                                                      | SQLite database                                          | Makes node telemetry, status, and capture records available to the FastAPI backend       |

Table 5: Flutter Dart Inputs and Outputs

| Inputs                                         | Outputs                                               |
|------------------------------------------------|-------------------------------------------------------|
| Backend API address and connection settings    | Backend connection status                             |
| Node telemetry received through the REST API   | Live node status cards and sensor readings            |
| RMS, dBFS, SPL, FFT, and frequency-band data   | Acoustic values, graphs, and room summaries           |
| Temperature, humidity, and battery data        | Environmental and battery displays                    |
| Heartbeat, firmware version, and capture state | Online/offline indicators and system alerts           |
| User-selected capture label and duration       | Capture command sent to the backend                   |
| User calibration settings and reference values | Calibration request and displayed calibration results |
| Edge Impulse prediction data                   | Predicted class and confidence score                  |
| MATLAB analysis data                           | Room-analysis visualizations and recommendations      |
| Historical records from the backend database   | Capture history, reports, and CSV downloads           |

### 5.3.3. Node-RED Process Flow

#### The Broker Smoke Test

The Broker Smoke Test is a quick system-health check that confirms Node-RED and the Mosquitto MQTT broker can successfully publish, receive, and display SDACS messages before the physical sensor nodes are used. It sends simulated heartbeat and acoustic-feature data through the same MQTT topics as the real system, helping isolate communication or backend problems early.

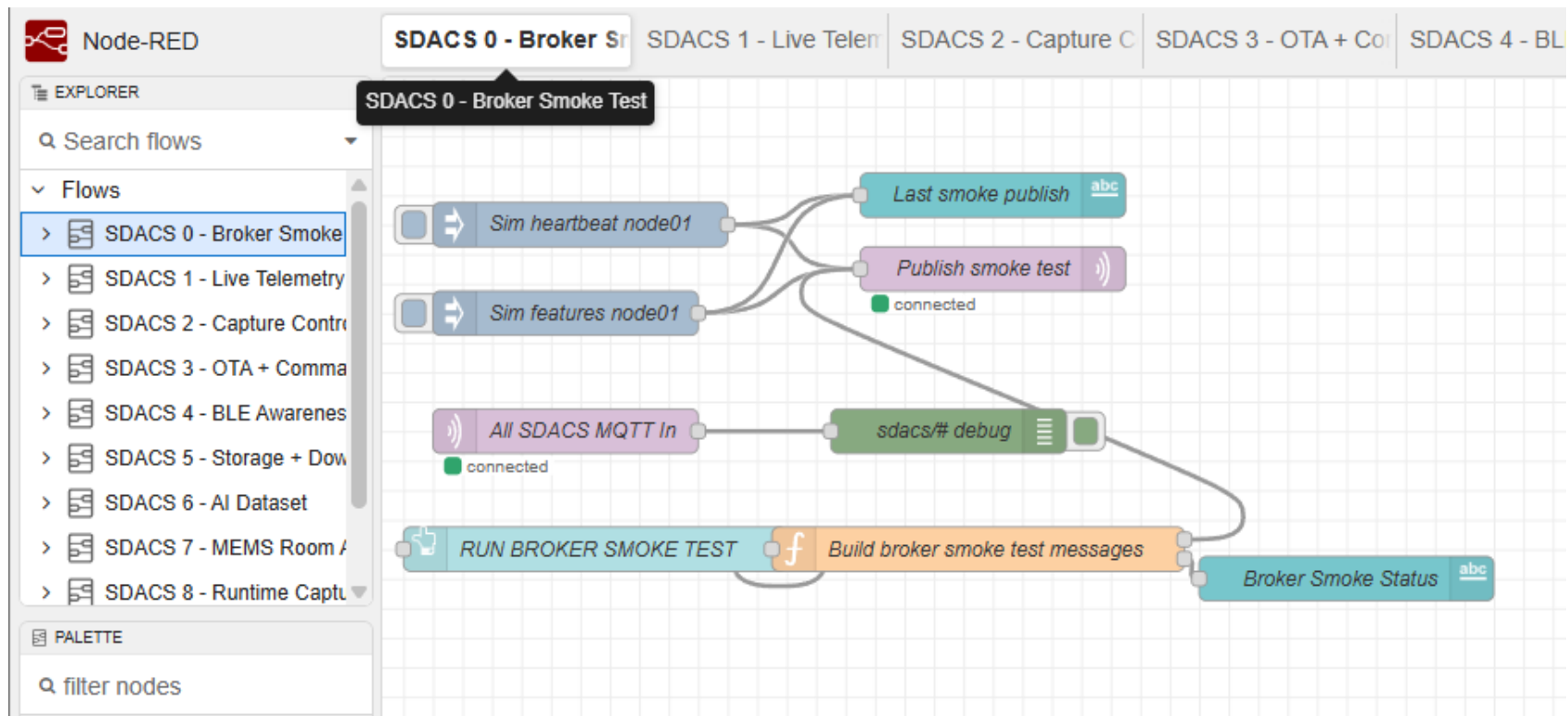


Figure 18:SDACS 0 Broker Smoke Test flow, showing simulated telemetry generation, MQTT publication, and dashboard confirmation      Live Telemetry: Flow and Dashboard Module

The SDACS 1 – Live Telemetry flow receives real-time acoustic, environmental, battery, and node-health data from all four sensor nodes through MQTT. It safely parses and normalizes the incoming messages, then updates the Node-RED dashboard with the latest measurements and a consolidated node-status table. This gives the operator one place to confirm that every node is connected, reporting valid data, and operating normally.

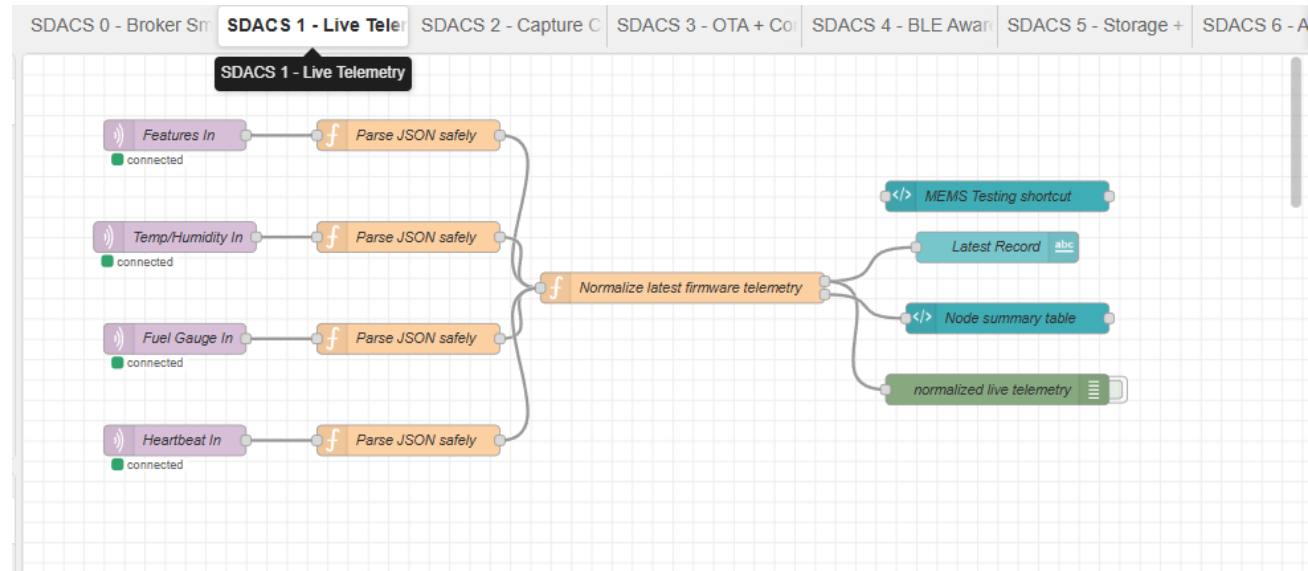


Figure 19: SDACS Live Telemetry Node-RED flow for receiving, normalizing, and displaying real-time acoustic measurements and node-health information.



Figure 20: Live Telemetry dashboard module

### Capture Control Flow and Dashboard Module

The SDACS 2 – Capture Control flow coordinates synchronized recording across all four sensor nodes. The operator selects the capture delay, recording duration, and dataset label, then starts one group command that is published to every node through MQTT. A shared request ID links the resulting status messages and completed capture files, allowing the dashboard to show each node's progress and confirm that the distributed recording completed successfully.

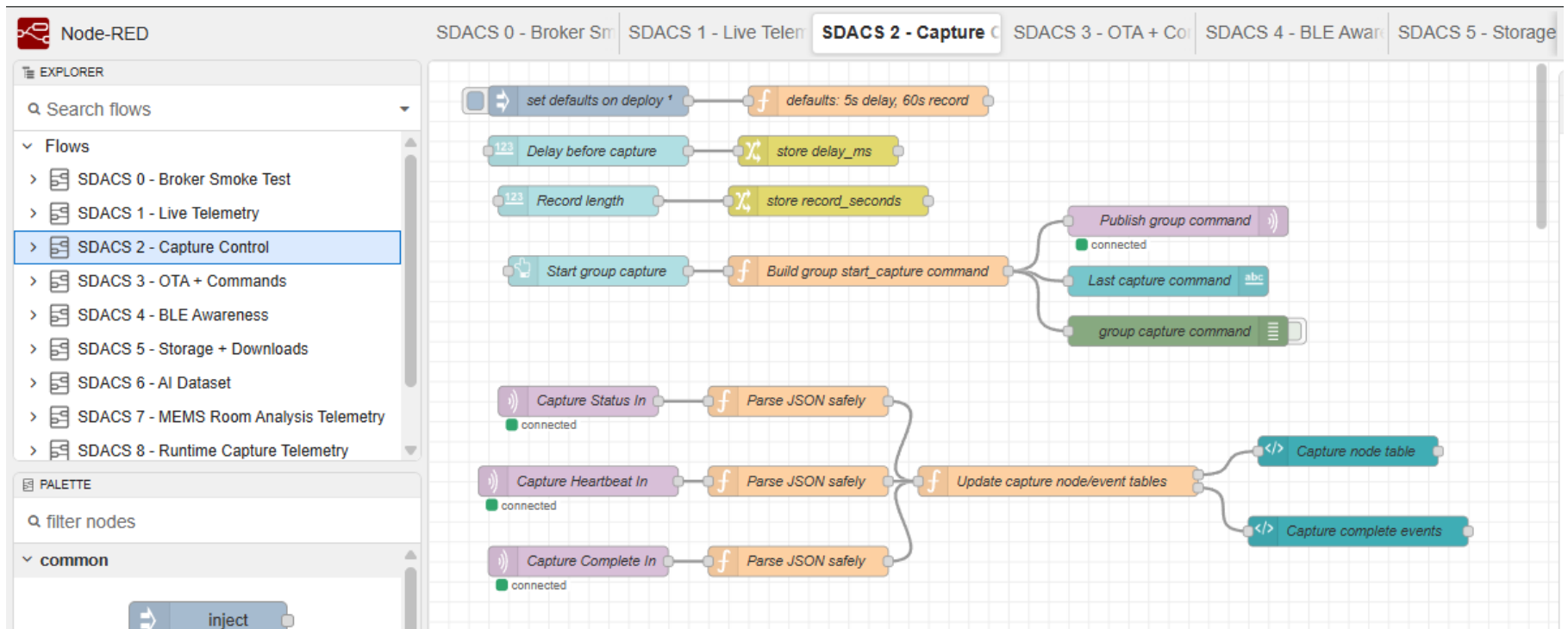


Figure 21:SDACS Capture Control Node-RED flow for configuring, initiating, and monitoring synchronized multi-node acoustic data collection.

This dashboard section allows the operator to configure and start a synchronized capture across all SDACS nodes. The Delay before capture setting provides preparation time before recording begins, while Record length defines the duration of the capture. Selecting START GROUP CAPTURE publishes the configured command to all nodes, and the Last command field displays the most recent capture settings for confirmation.

The screenshot shows a dashboard titled "Capture Control" in blue text. Below the title, there are two configuration fields: "Delay before capture" and "Record length". The "Delay before capture" field has a dropdown arrow, the value "0 ms", and an up arrow. The "Record length" field has a dropdown arrow, the value "1 s", and an up arrow. To the right of these fields is a large blue button with a white play icon and the text "START GROUP CAPTURE". Below the configuration fields, there is a "Last command" label followed by a right-pointing arrow and the text "| delay ms | record s".

Figure 22: Capture Control Dashboard Module

#### Over The Air (OTA) Flow and Dashboard Module

The SDACS 3 – OTA + Commands flow provides centralized remote maintenance and command control for the four sensor nodes. The operator selects a target node and can issue commands such as ping, reboot, or firmware update through MQTT, while an OTA status table reports each node's progress, firmware version, request ID, and any resulting messages. This allows node availability and firmware deployment to be managed without physically reconnecting each device. This flow is not working.

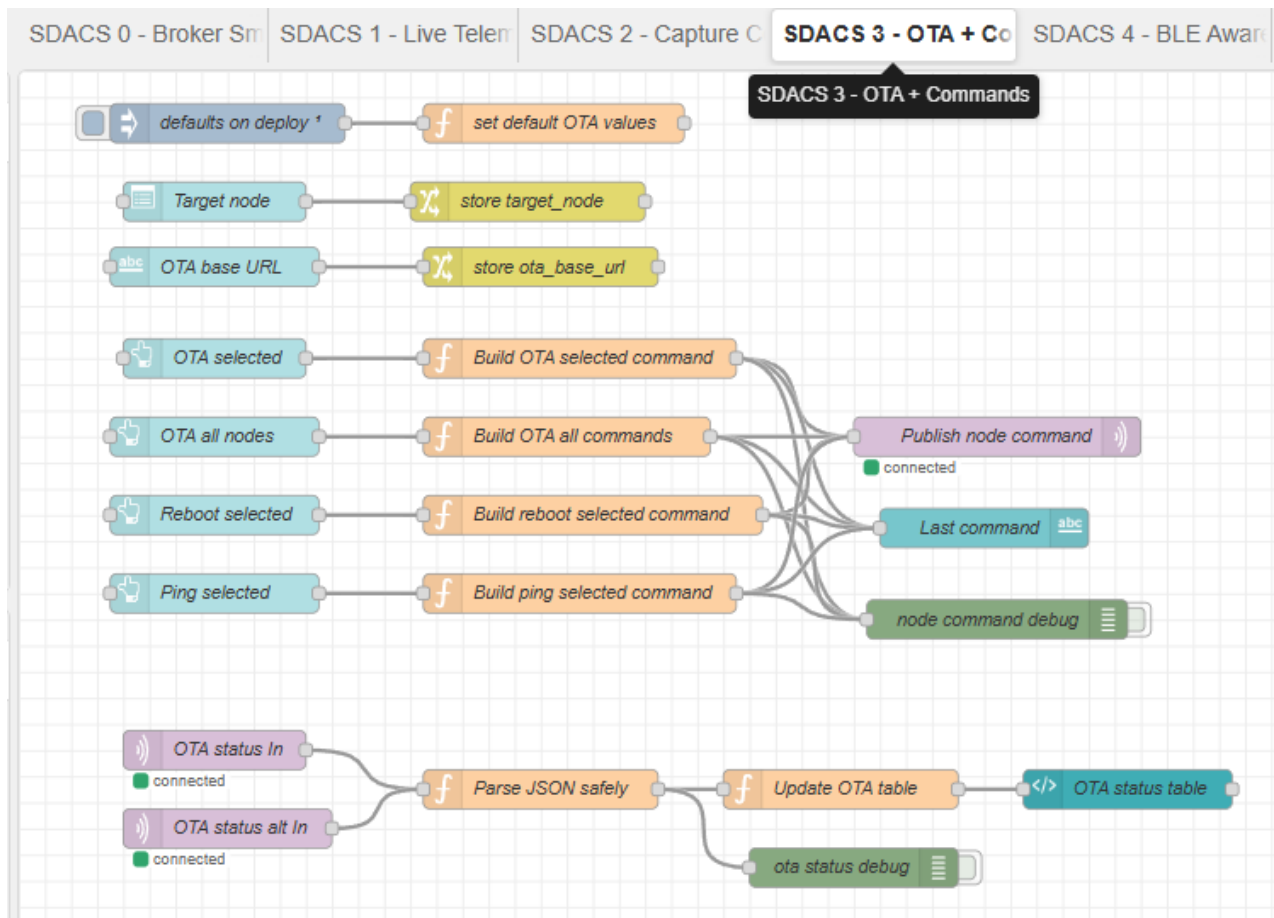


Figure 23: SDACS OTA and Commands Node-RED flow for issuing remote node commands and monitoring firmware-update status.

This dashboard section provides centralized remote control and maintenance for the SDACS sensor nodes. The operator selects a target node, enters the firmware server URL, and can then initiate an update for one node or all four nodes, reboot the selected device, or send a ping command to confirm connectivity. The lower status table displays each node's command state, request identifier, update progress, firmware version, response message, and most recent activity, allowing the operator to verify that commands were received and completed successfully. This will be implemented in the future if the project ever goes to market.

OTA + Commands

Target

node01

OTA base URL

Example: http://192.168.68.56:8080/firmware

OTA SELECTED NODE

OTA ALL NODES

REBOOT SELECTED

PING SELECTED

Last command

→

| Node | State | Request | Progress | FW | Message | Last seen |
|------|-------|---------|----------|----|---------|-----------|
|------|-------|---------|----------|----|---------|-----------|

Figure 24: OTA + Commands Dashboard Module

### Bluetooth Awareness (BLE) Flow

The SDACS 4 – BLE Awareness flow provides a secondary method of confirming that the four sensor nodes are physically present and detectable near the Raspberry Pi gateway. Selecting Scan Now first commands all nodes to advertise over Bluetooth for 15 seconds, waits briefly for the advertisements to begin, and then triggers a 15-second BLE scan on the Raspberry Pi. The resulting dashboard displays each detected node, its BLE signal strength, estimated proximity, detection count, address, and last-seen time

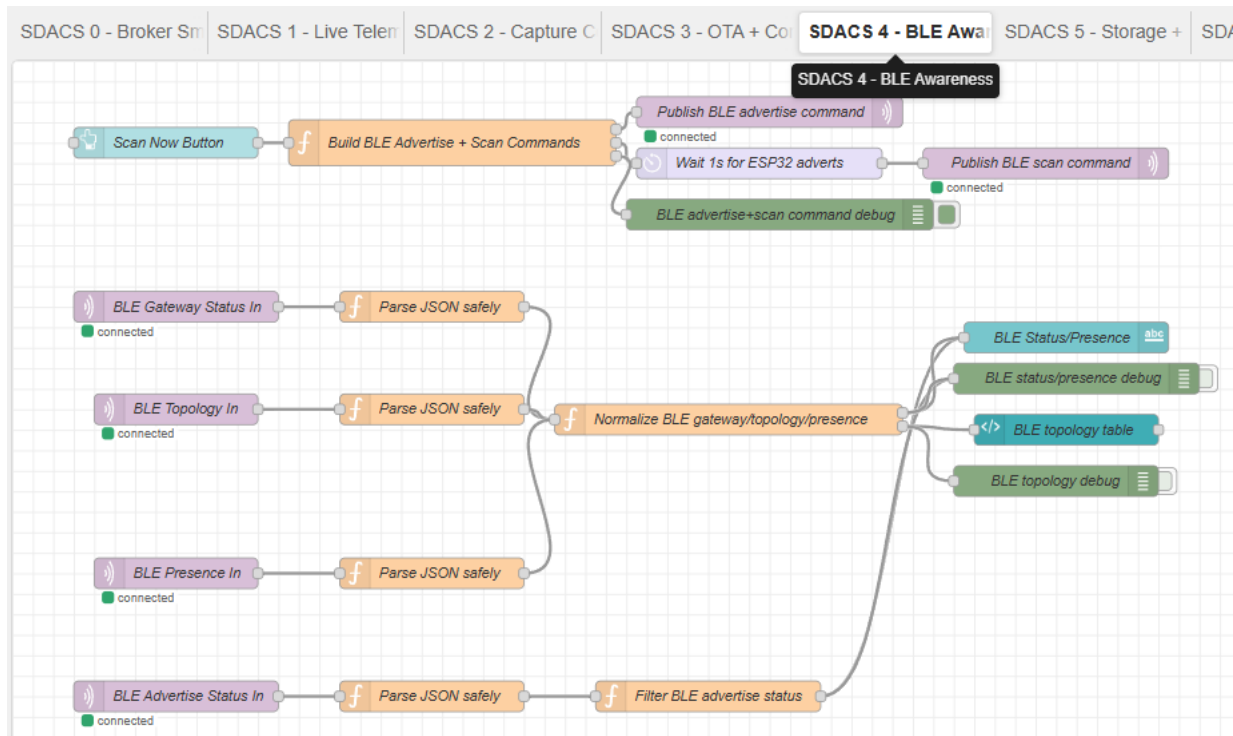


Figure 25:SDACS BLE Awareness Node-RED flow for triggering node advertisements, performing a Raspberry Pi gateway scan, and displaying node-presence and proximity information.

This following dashboard screenshot shows a completed BLE awareness scan initiated from Node-RED. The gateway reports that the scan finished successfully and detected one SDACS node. The results table identifies node04 as being nearby, with a BLE RSSI of -53 dBm, one detection during the scan, a recorded last-seen timestamp, and its Bluetooth device address. This confirms that the Raspberry Pi BLE gateway can trigger a scan, detect advertising SDACS nodes, and display their presence information on the dashboard.

### BLE Awareness

**SCAN NOW**

**BLE gateway | complete | | 2026-07-28T20:53:15.818310+00:00 |**  
 Status/Presence **ble\_scan\_20260728205259**

**Last scan:** 2026-07-28T20:53:15.818122+00:00 | **Request:** ble\_scan\_20260728205259 |  
**Trigger:** node-red | **Nodes:** 1

| Node   | Proximity | BLE RSSI | Seen | Last Seen                        | Address           |
|--------|-----------|----------|------|----------------------------------|-------------------|
| node04 | near      | -53      | 1    | 2026-07-28T20:53:15.818101+00:00 | 94:A9:90:28:5A:5A |

Figure 26:BLE Awareness Dashboard Module Showing RSSI and Details

### Storage + Downloads Flow

The SDACS 5 – Storage + Downloads flow converts incoming MQTT records into a consistent format and stores them for later testing and analysis. It logs system telemetry and capture events to the Raspberry Pi SQLite database, writes the required CSV datasets, filters unnecessary periodic records, and provides HTTP download endpoints for the files used by MATLAB and Edge Impulse. This gives the project a permanent, traceable record of system operation beyond the live dashboard.

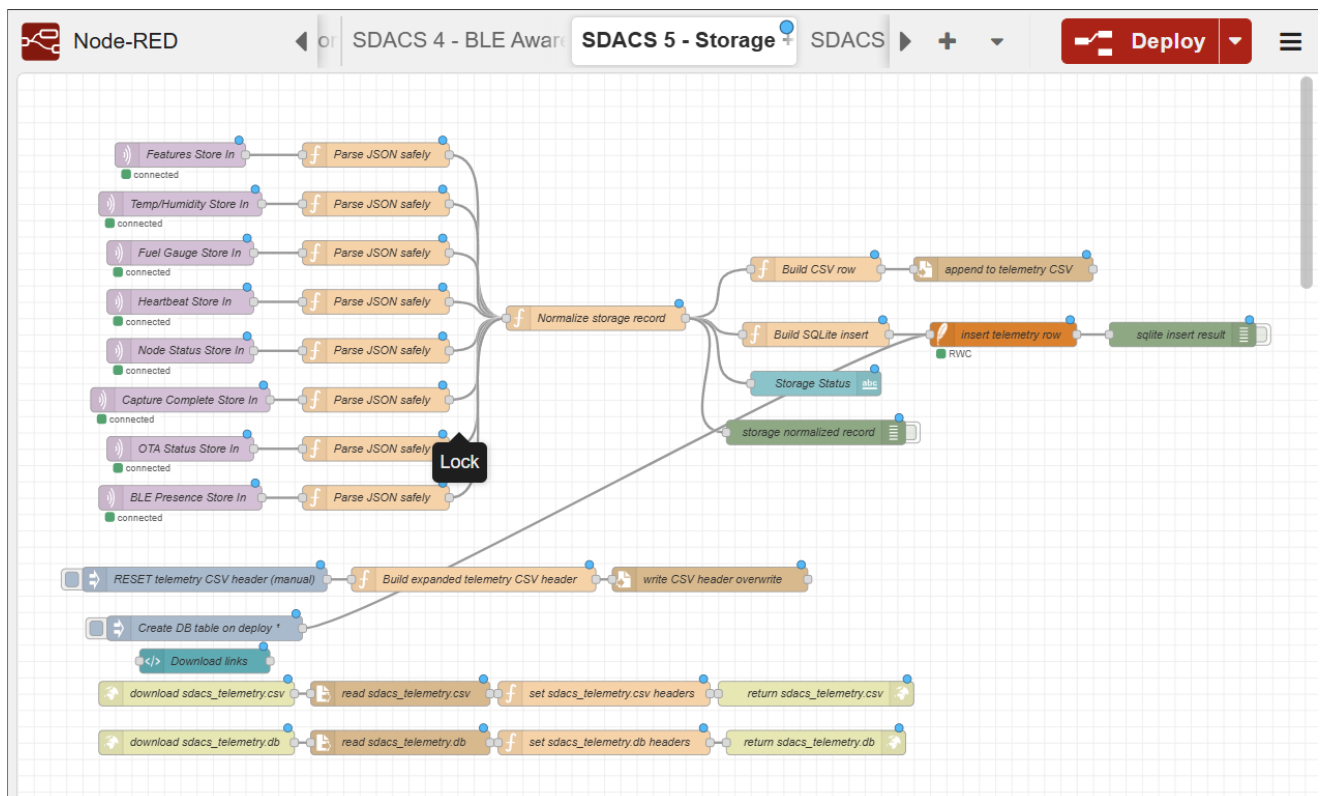


Figure 27: SDACS Storage and Downloads Node-RED flow for normalizing MQTT records, logging data to SQLite and CSV files, and providing HTTP download access.

This dashboard section confirms that SDACS telemetry is being saved successfully. The status message shows that a heartbeat record from node03 was written to both the CSV log and the SQLite database. The two links allow the operator to download either the CSV file for inspection and analysis or the complete SQLite database for detailed review and backend use.

## Storage + Downloads

Storage

**Logged heartbeat from node03 to CSV + SQLite**

[Download CSV](#) | [Download SQLite DB](#)

*Figure 28: Node-RED Dashboard Widget Showing Data Logged to CSV and DB with Download Links*

### AI Dataset Collection Flow

The SDACS 6 – AI Dataset Collection flow manages labelled acoustic data collection for Edge Impulse training. The operator selects one of six approved sound classes before starting a capture, and only feature records linked to a valid labelled capture are written to the AI dataset CSV. The flow also validates incoming records, displays the latest accepted row, maintains the required CSV header, and produces MATLAB summary data while keeping unlabelled runtime captures separate from the training dataset.

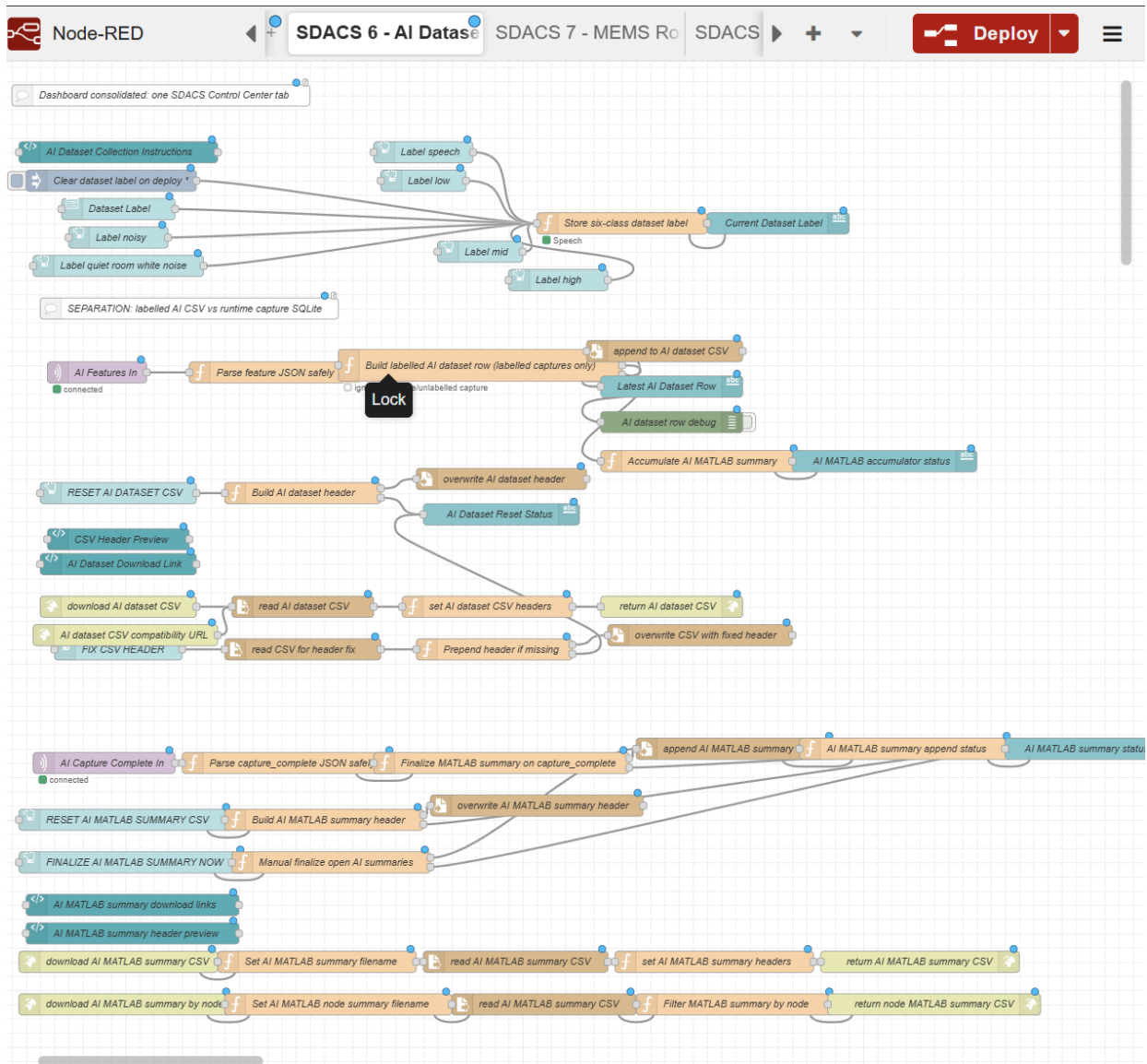


Figure 29:SDACS AI Dataset Collection Node-RED flow for label selection, feature validation, CSV generation, dataset reset, download handling, and MATLAB summary creation.

The dashboard screenshot shows the operator interface for selecting the ground-truth class used during acoustic data collection. Six buttons represent the available labels: noisy, speech, low, mid, high, and quiet-room white noise. The current label is displayed beside the selector, while the Latest row field confirms whether the newest feature record was accepted and shows its node, sequence number, label, signal levels, clipping status, and warnings. The interface also provides controls to download the AI dataset, repair the CSV header, or reset the file before starting a new collection session.

## AI Dataset Collection

Dataset label Select one of the six labels Current **Speech**

NOISY

SPEECH

LOW (40–500 HZ)

MID (400 HZ–4 KHZ)

HIGH (4–20 KHZ)

QUIET ROOM (WHITE NOISE)

Latest **VALID | node03 seq 20 | speech | raw -68 dBFS | scene -64.26 dBFS |**  
row **clipped 0 | ready; warning=i2s\_timeouts**

[Download AI Dataset CSV](#)

**AI CSV header — raw spectral path + 150 Hz HPF scene path:**

timestamp\_iso,run\_id,node\_id,seq,sample\_rate\_ok,effective\_sample\_rate\_hz,audio\_error,target\_freq\_hz,target\_band,suggested\_z\_metric,rms,dbfs,db\_spl,f\_peak\_hz,f\_peak\_full\_hz,f\_peak\_acoustic\_hz,tone\_1khz\_ratio,tone\_1khz\_peak\_hz,tone\_1khz\_ratio\_hit,tone\_1khz\_peak\_hit,tone\_1khz\_detected,tone\_1khz\_local\_total\_energy,tone\_1khz\_local\_noise\_energy,tone\_1khz\_local\_noise\_avg\_energy,tone\_1khz\_local\_ratio,tone\_1khz\_contrast\_db,tone\_1khz\_contrast\_hit,low\_rumble\_ratio,room\_band\_total\_energy,band\_sub\_ratio,band\_bass\_ratio,band\_low\_mid\_ratio,band\_mid\_ratio,band\_presence

FIX CSV HEADER

RESET CSV

CSV **CSV reset: dual-path raw + HPF scene header**  
status **written; old rows cleared**

Figure 30: SDACS AI Dataset Collection dashboard showing class selection, latest-row validation, CSV download, header repair, and dataset reset controls.

### MEMS Room Analysis Telemetry Flow

The SDACS 7 – MEMS Room Analysis Telemetry flow prepares acoustic measurements for detailed room and frequency-band analysis. Incoming MEMS feature records are parsed, normalized, displayed on the dashboard, and appended to `mems_testing_telemetry.csv`. The flow records test metadata such as the selected label, reference SPL, and operator notes, while also preserving room-analysis features including sub, bass, low-mid, mid, presence, and high-band ratios, dominant-band results, 1 kHz detection metrics, acoustic peak frequency, and I2S diagnostic information. It also provides controls to reset and download the MEMS CSV for offline MATLAB analysis

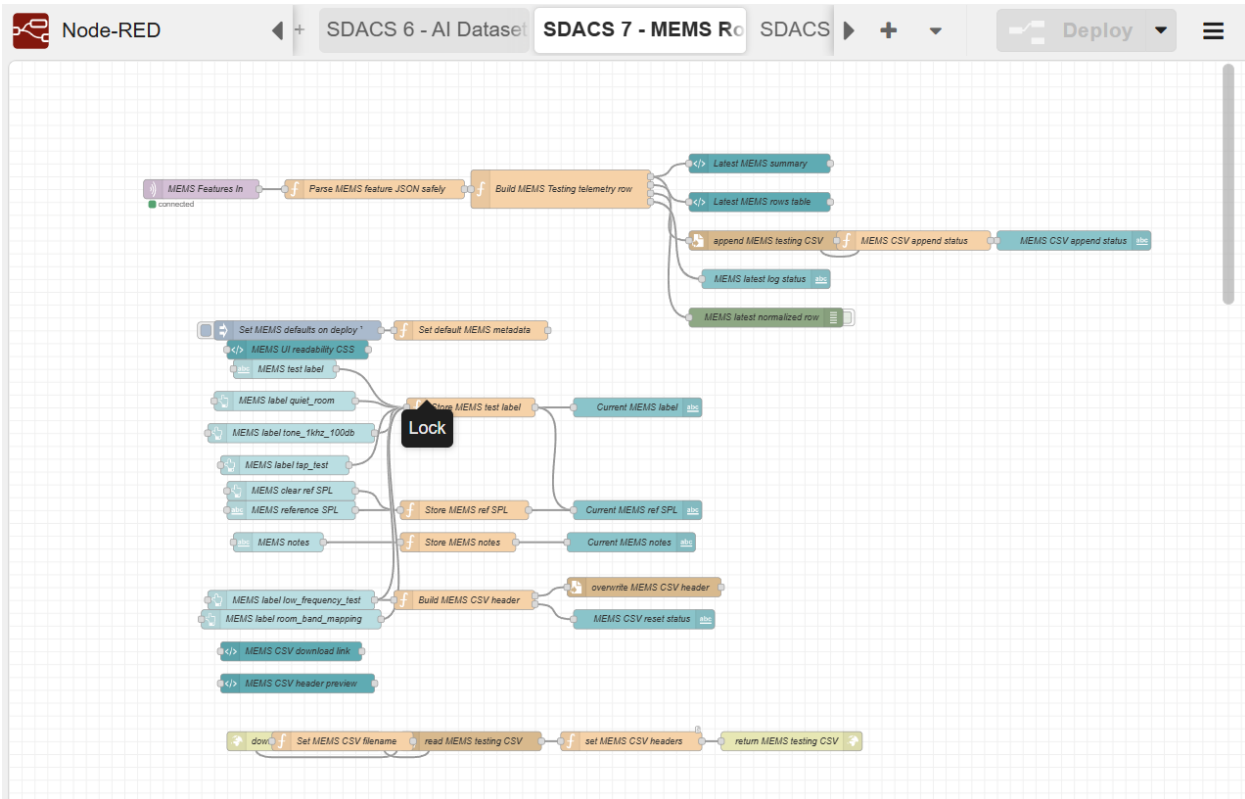


Figure 31: SDACS MEMS Room Analysis Node-RED flow for receiving acoustic features, adding test metadata, displaying normalized results, and writing the MEMS testing CSV.

The MEMS Capture Summary presents the most recent acoustic record in a readable format. It reports the source node, test label, I2S configuration, sample count, sampling status, dominant frequency band, individual band-energy ratios, 1 kHz tone-detection results, peak frequencies, signal level, and diagnostic warnings. The table beneath the summary compares the latest measurements from all four nodes, allowing differences in dominant band, dominant-band ratio, and peak frequency to be identified across the room.

## MEMS Capture Summary

### Latest MEMS Room/Band Row

#### Test

Node: **node02** | Label: **unlabeled** | Ref SPL: **N/A** | Notes:

#### I2S

Slot: **left** | RX Mode: **stereo\_raw** | Conv: **shift8** | n: **47872** | Eff SR: **47872** | SR OK: **true**  
Timeouts: **97** | Audio Error: **i2s\_timeouts**

#### Room Analysis

Dominant Band: **bass** | Ratio: **0.316755** | Peak: **46.9** Hz

Sub: **0** | Bass: **0.316755** | Low-mid: **0.22662** | Mid: **0.201563** | Presence: **0.132347** | High: **0.122715**

#### 1 kHz Tone Detection

Detected: **false** | Ratio: **0.031378** | Ratio hit: **false** | Peak hit: **true** | 1 kHz Peak: **1031.2** Hz  
Local ratio: **0.135605** | Contrast: **-8.68** dB | Contrast hit: **false** | Local noise: **0.01090445**

#### Audio

Full Peak: **46.9** Hz | Acoustic Peak: **281.2** Hz | dBFS: **-68.59** | p2p: **25120**

current dBFS: **-68.59** | shift8 dBFS: **-120** | low24 dBFS: **-120** | Rumble Ratio: **0.33851** | Room total: **0.06925817**

Latest MEMS logged node02 seq 59 | dominant=bass (0.316755 @ 46.9 Hz) |  
MEMS 1k\_ratio=0.031378 | local=0.135605 | contrast=-8.68 dB | 1k=no |  
loq label=unlabeled

## Raw I2S Scaling Diagnostics

| Node   | Label     | Ref | Seq | Dominant | Dom Ratio | Dom Peak | Sub | E |
|--------|-----------|-----|-----|----------|-----------|----------|-----|---|
| node01 | unlabeled | N/A | 59  | low_mid  | 0.37588   | 234.4    | 0   | C |
| node02 | unlabeled | N/A | 59  | bass     | 0.316755  | 46.9     | 0   | C |
| node03 | unlabeled | N/A | 59  | low_mid  | 0.317547  | 234.4    | 0   | C |
| node04 | unlabeled | N/A | 59  | bass     | 0.377016  | 46.9     | 0   | C |

Figure 32: MEMS Capture Summary dashboard showing the latest room-analysis measurements, 1 kHz detection metrics, audio diagnostics, and four-node comparison table.

The **MEMS CSV Controls** dashboard allows the operator to assign a descriptive test label, enter a reference sound-pressure level, and record notes before a capture. Quick-selection buttons are

provided for common tests such as quiet-room measurements, 1 kHz reference tones, tap tests, low-frequency tests, and room-mapping sessions. The lower section confirms the CSV file path and appends status and provides controls to reset or download the current MEMS dataset.

MEMS CSV Controls

Test label

Current label

QUIET

1 KHZ / 100 DB

TAP TEST

Ref SPL dB

LOW FREQ

CLEAR REF

ROOM MAP

Current ref

Notes

Current notes

Use quick labels before each capture. This MEMS panel is on the main SDACS dashboard tab and now logs room-analysis bands: sub, bass, low-mid, mid, presence, high, dominant band, and 1 kHz detector hit fields plus local contrast metrics.

MEMS Downloads

RESET MEMS CSV

CSV status

Download MEMS Testing CSV

CSV now includes room-analysis band ratios/peaks, dominant\_band fields, and local 1 kHz contrast metrics. Path resolves from SDACS\_MEMS\_CSV\_PATH or HOME/sdacs/mems/mems\_testing\_telemetry.csv

CSV

MEMS CSV append OK: 3:42:59 PM →

append /home/kyledavid36/sdacs/mems/mems\_testing\_telemetry.csv

MEMS CSV header:

current room-analysis + local-contrast header is 129 columns and includes tone hit fields, local 1 kHz contrast fields, sub/bass/low-mid/mid/presence/high band ratios, band peaks, and dominant\_band fields.

Key fields:

tone\_1khz\_local\_ratio, tone\_1khz\_contrast\_db, tone\_1khz\_contrast\_hit, tone\_1khz\_ratio\_hit, tone\_1khz\_peak\_hit, room\_band\_total\_energy, band\_bass\_ratio, band\_low\_mid\_ratio, band\_mid\_ratio, band\_presence\_ratio, band\_high\_ratio, dominant\_band\_name, dominant\_band\_ratio, dominant\_band\_peak\_hz.

Figure 33: MEMS CSV Controls and download interface used to assign test metadata, manage the room-analysis dataset, and confirm successful CSV storage.

Runtime Capture Telemetry Flow

The SDACS 8 dashboard component provides basic confirmation that runtime acoustic records have been linked to an active capture and written to the SQLite database. Unlike the AI Dataset

67

Collection interface, it does not provide label-selection or dataset-management controls because its primary role is to preserve labelled and unlabeled runtime captures for backend processing.

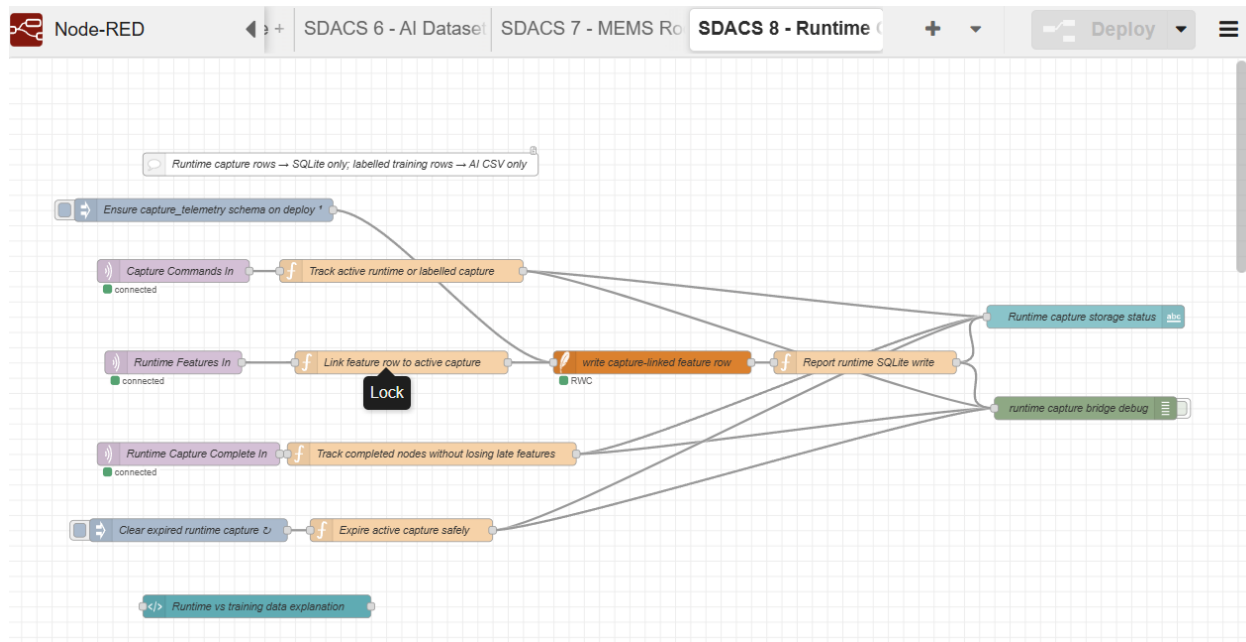


Figure 34: SDACS Runtime Capture Telemetry Node-RED flow for linking acoustic feature records to active captures and storing them in SQLite

#### 5.3.4. MATLAB Modelling

MATLAB was used to model the physical position of the nodes and the data received by each node.

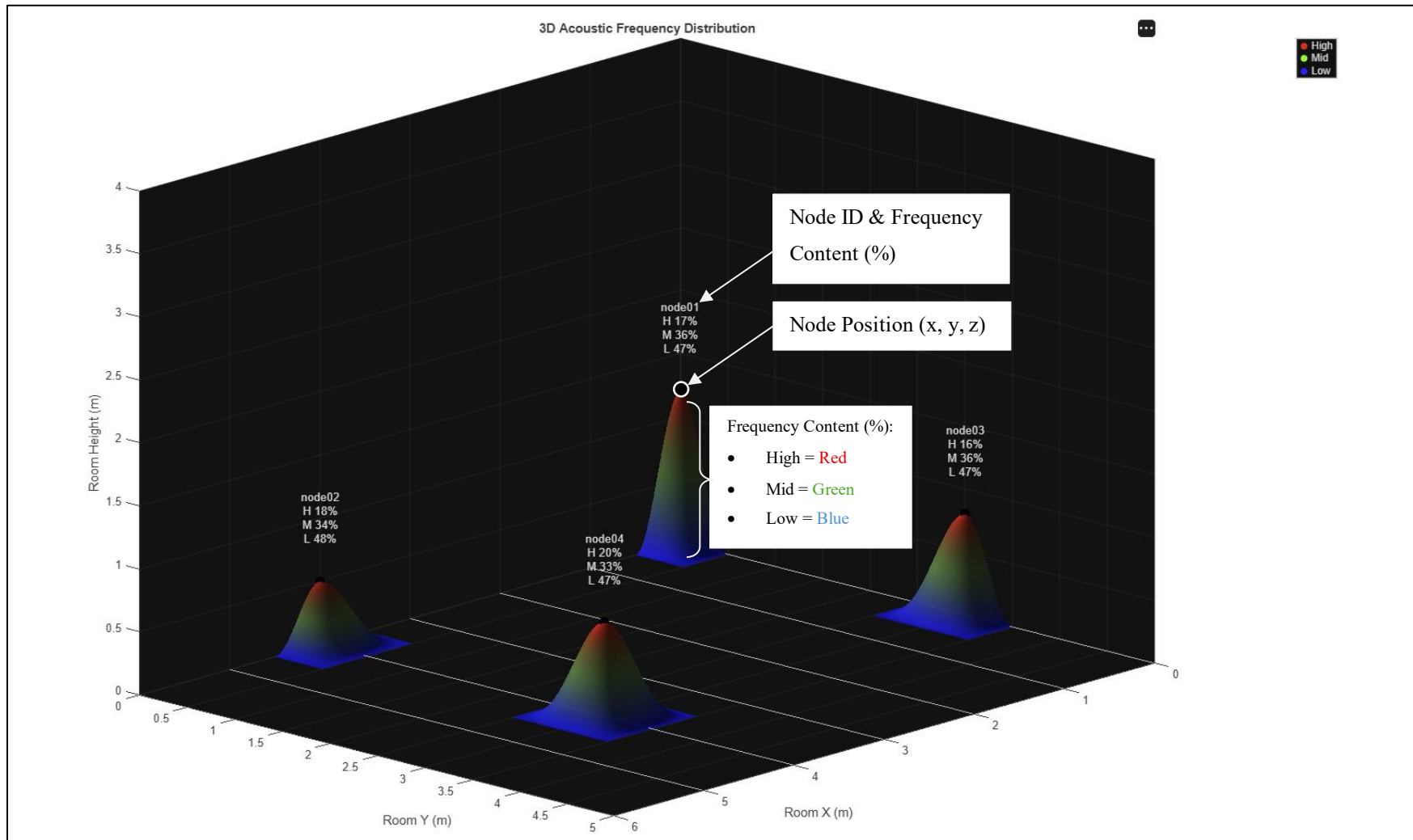


Figure 35: MATLAB Model, Sample Data from SDACS

The MATLAB Model uses frequency ratios from the entire capture to determine the frequency profile at the node. The ratios are represented as a percentage of all the frequencies detected by the node during the capture. For example, a 20-second recording of a full sweep from 0Hz to 20kHz should detect all frequencies equally, and the ratios of low, mid, and high frequency calculated post-recording should be equal at one-third or 33% of the whole capture. Any deviations from this expected equality reflect a characteristic of the recording space, i.e. node01 in the figure above has a bass-heavy profile because the low frequency ratio is 47%, about 15% higher than the expected 33%.

The MATLAB model provides the user a visual aide to understand both where the data is being collected, and the type of data that is being collected to determine the frequency bias of their space. Additionally, the TinyML model uses this data to recommend treatment plans to the user depending on the results.

#### 5.3.5. *Edge Impulse TinyML Model for Room Treatment Recommendations*

Edge Impulse was implemented as the machine-learning development platform for the SDACS acoustic classification system. Rather than processing raw audio directly, the model uses acoustic features generated by the recording nodes and collected through the Node-RED middleware.

These measurements represent characteristics of the captured audio including signal level, peak-frequency information, frequency-band ratios, dominant-frequency information, and FFT-derived energy measurements.

The final classifier uses 31 engineered acoustic input features. Removing the raw audio from the machine-learning input significantly reduces the amount of information that must be processed while allowing the classifier to operate using measurements already produced by the SDACS firmware. The resulting feature vectors are validated and stored by Node-RED before being exported as labelled CSV datasets for model development.

The final model was simplified to classify three acoustic conditions:

1. Noisy
2. Quiet Room with Noise
3. Speech

A feedforward Artificial Neural Network (ANN) was selected for classification. The 31 input features are passed through fully connected dense layers, followed by a Softmax output layer that produces a confidence value for each of the three acoustic classes. The model is non-recurrent, meaning that each feature vector is classified independently without requiring information from previous captures.

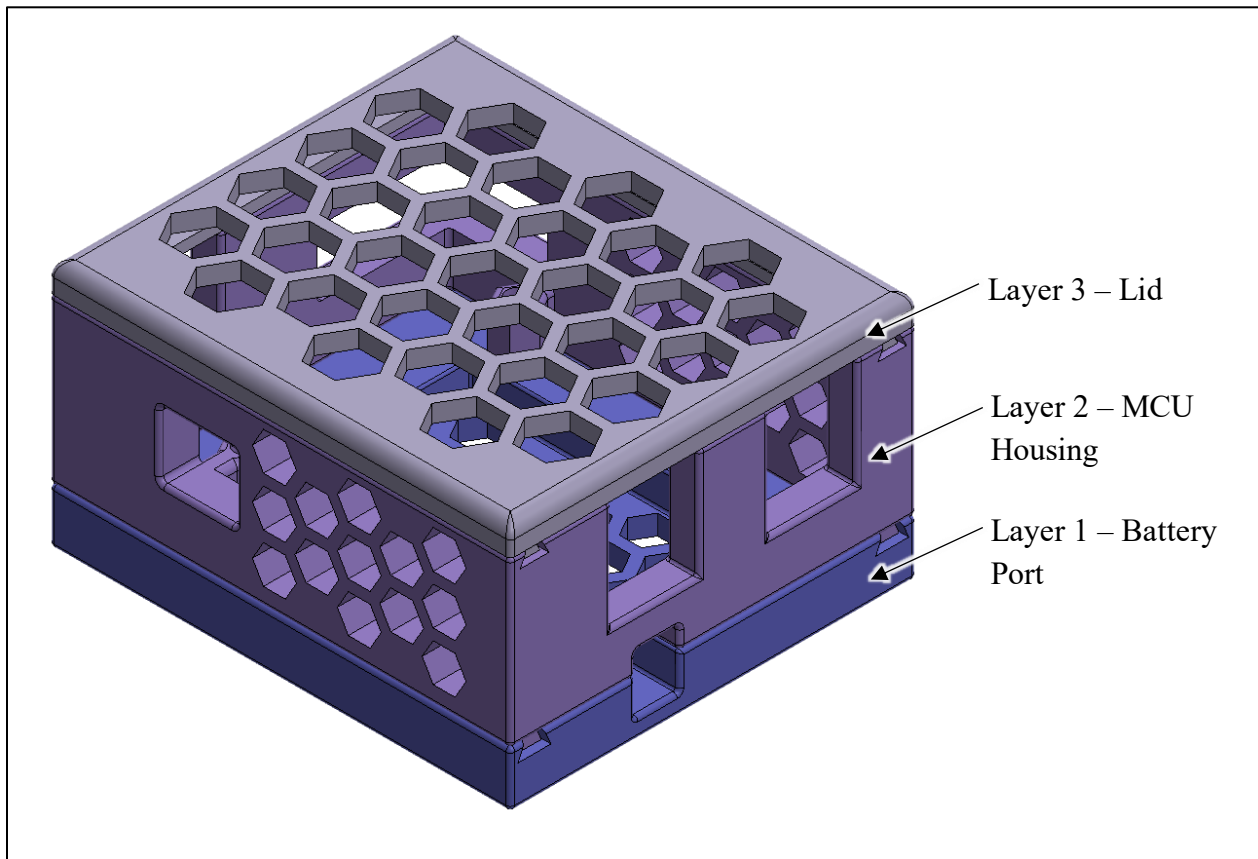
Development was performed iteratively. The initial model used a smaller and more limited dataset and included six acoustic classes. Although this model demonstrated that the engineered features could be used for acoustic classification, several classes contained overlapping characteristics and insufficient training examples. The final model therefore focused on three broader acoustic conditions and used an expanded dataset to improve generalization.

Following training and validation in Edge Impulse, the optimized model can be deployed to the Raspberry Pi 5 for local inference. This allows SDACS to automatically classify the acoustic

environment using measurements received from the distributed nodes. The resulting classification and confidence values can then be used by the higher-level SDACS analysis to provide additional information about the recording environment to the user.

#### 5.4. Enclosure Design

The enclosure was designed to be small, portable, and easily accessible for prototyping and testing purposes. Additionally, the enclosure was designed to be as light as possible, so the material used was PLA plastic via 3-D printing. The enclosure design is still under development, but the most current revision (Revision F) is shown below:



*Figure 36: SDACS Enclosure, Revision F*

The enclosure contains the LiPo battery, the MCU, and the custom PCB (mounted on MCU).

The enclosure consists of three main components:

1. Layer 1 - Battery Port
2. Layer 2 - MCU Housing
3. Layer 3 - Lid

#### 5.4.1. Battery Port

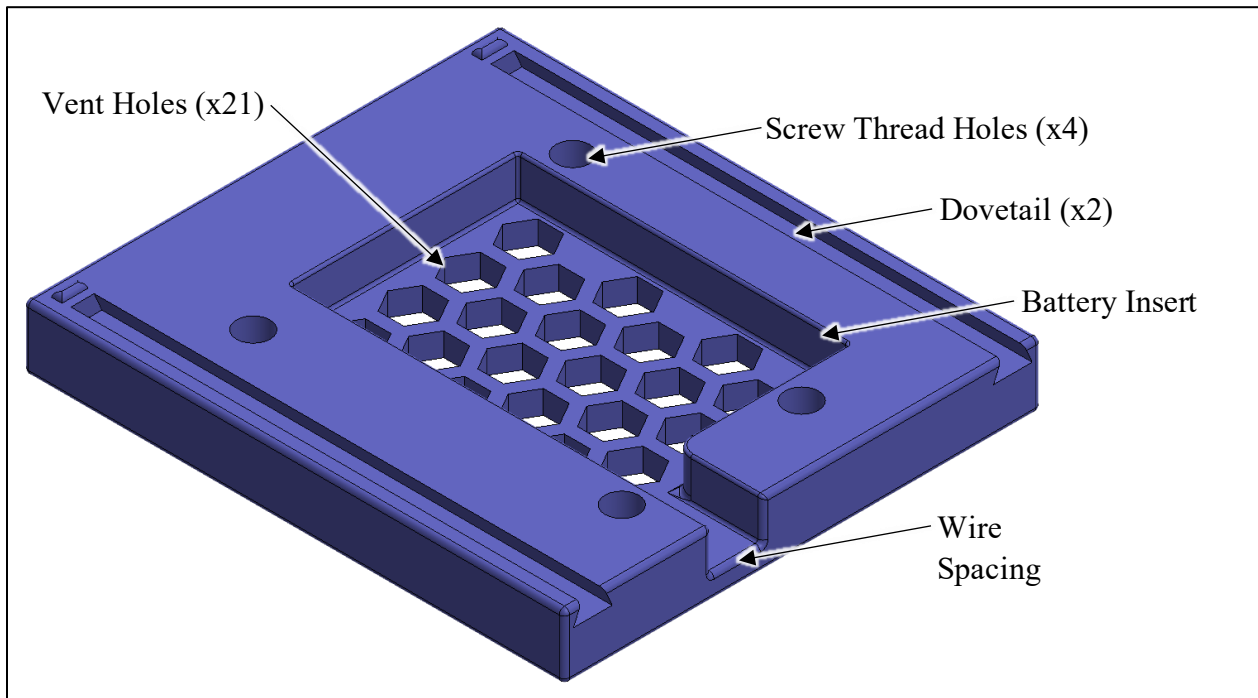


Figure 37: SDACS Battery Port, Revision F

The battery port provides housing for the LiPo battery, which powers the entire SDACS system. The battery insert has ventilation holes to prevent over-heating during operation, and a wire spacing port for the battery connector. The battery port additionally contains the screw thread holes for the standoffs, and a dovetail insert for connection to the MCU Housing layer.

### 5.4.2. MCU Housing

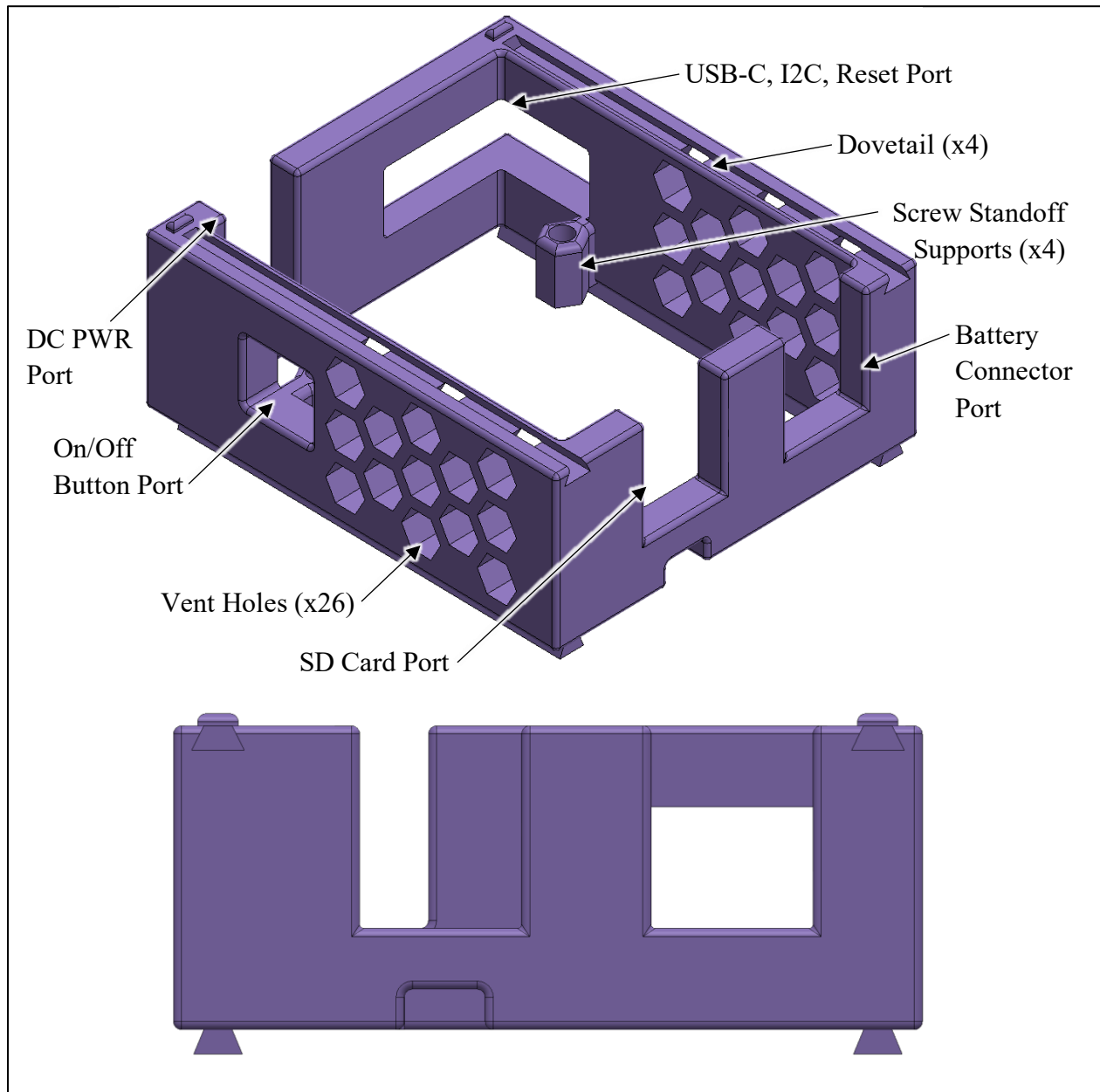
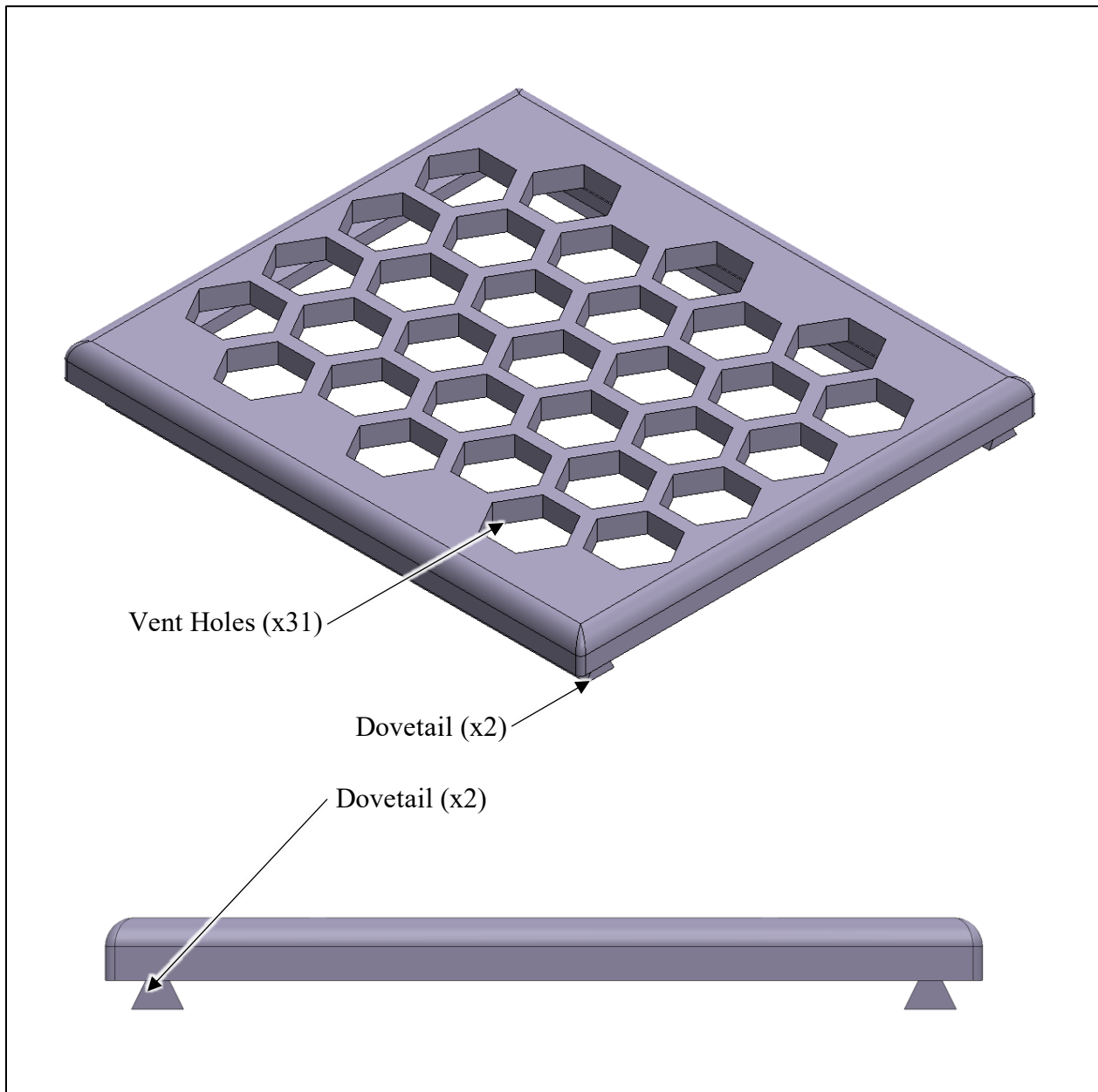


Figure 38: SDACS MCU Housing, Revision F

The MCU Housing uses screw standoff supports to mount the MCU inside the enclosure. There are various ports to allow external access by a connector, tool, or a finger (for buttons/switches). Additionally, the MCU has wire spacing for the battery connector, to allow easy plug-and-unplug of the battery as needed. The MCU Housing has two dovetail pairs: one to connect to the Battery Port layer, and the other to connect to the Lid layer.

#### 5.4.3. Lid



*Figure 39: SDACS Lid, Revision F*

The Lid provides housing for the custom PCB with vents allowing the microphone and temperature/humidity sensors to access the external environment. The Lid has two dovetails to connect to the MCU Housing Layer.

## 6. DOCUMENTARY EVIDENCE OF SYSTEM DEVELOPMENT AND TESTING

### 6.1 SDACS OSI Classification

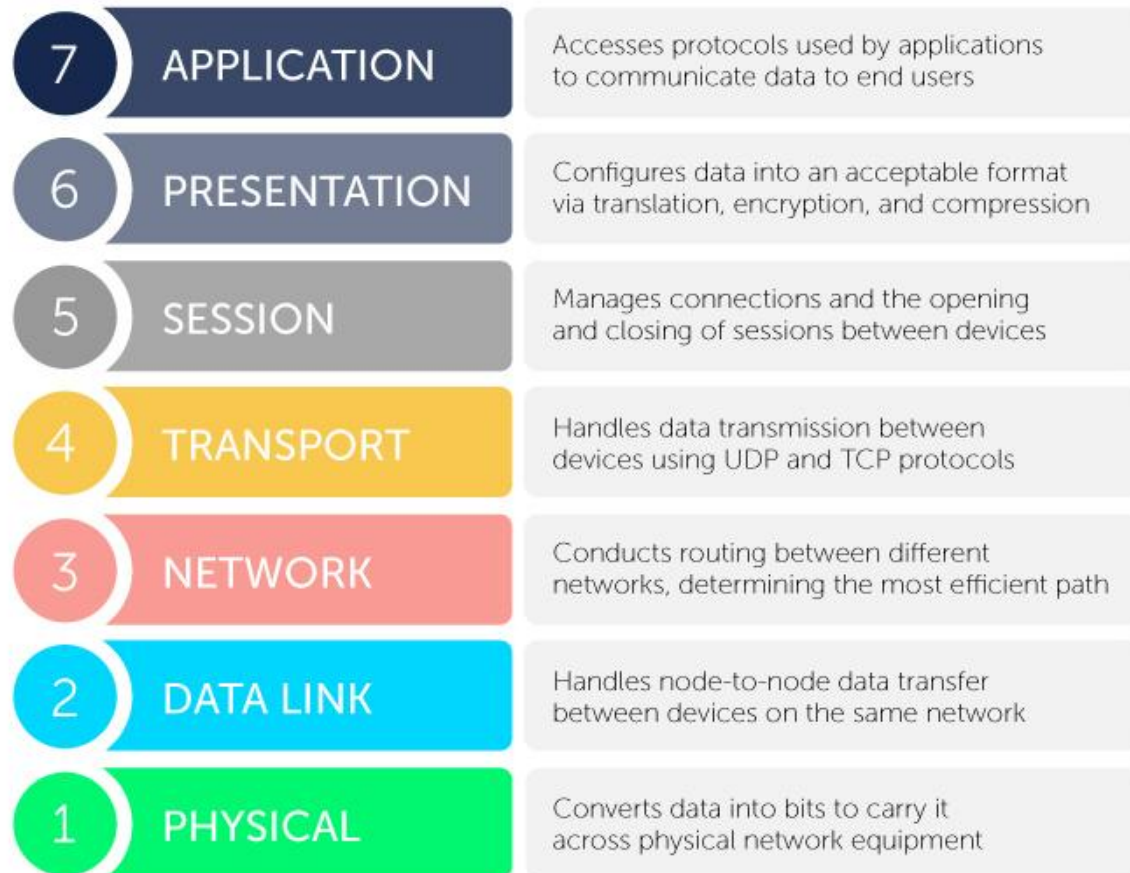


Figure 40: Visual Depiction of the OSI Model

Table 6: OSI Model Classification for SDACS Project

| OSI layer               | SDACS implementation                                                                                                               |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>7 – Application</b>  | MQTT topics and commands, FastAPI endpoints, Node-RED dashboard, Flutter GUI, Edge Impulse inference, MATLAB analysis, OTA control |
| <b>6 – Presentation</b> | JSON telemetry, CSV datasets, timestamps, labels, numeric feature formatting, AI confidence values                                 |
| <b>5 – Session</b>      | MQTT connection, subscriptions, keep-alive, reconnect handling, API communication sessions                                         |
| <b>4 – Transport</b>    | TCP for MQTT and HTTP/REST API communication                                                                                       |
| <b>3 – Network</b>      | IPv4 addressing, Raspberry Pi IP, ESP32 node IPs, routing, Tailscale virtual IP where used                                         |
| <b>2 – Data Link</b>    | Wi-Fi association and IEEE 802.11 frame communication; Ethernet where the Raspberry Pi uses a wired connection                     |
| <b>1 – Physical</b>     | ESP32 Wi-Fi radio and antenna, Raspberry Pi network interface, router/access point, Ethernet cabling                               |

The following figure presents the complete SDACS architecture and identifies the major functional units developed during Capstone II. The system consists of four distributed acoustic sensor nodes, a Wi-Fi and MQTT communication network, a Raspberry Pi backend, Node-RED data-collection tools, Edge Impulse classification, MATLAB room analysis, and a Flutter desktop GUI. These units operate together to capture, process, store, analyse, and display acoustic and environmental information.

This section should document both the development of the individual functional units and the testing of the fully integrated system. That structure is consistent with the required report progression from functional-unit design and test plans through system development, testing, results, and analysis.

# SDACS – System Functional Units Overview

Complete End-to-End System Architecture

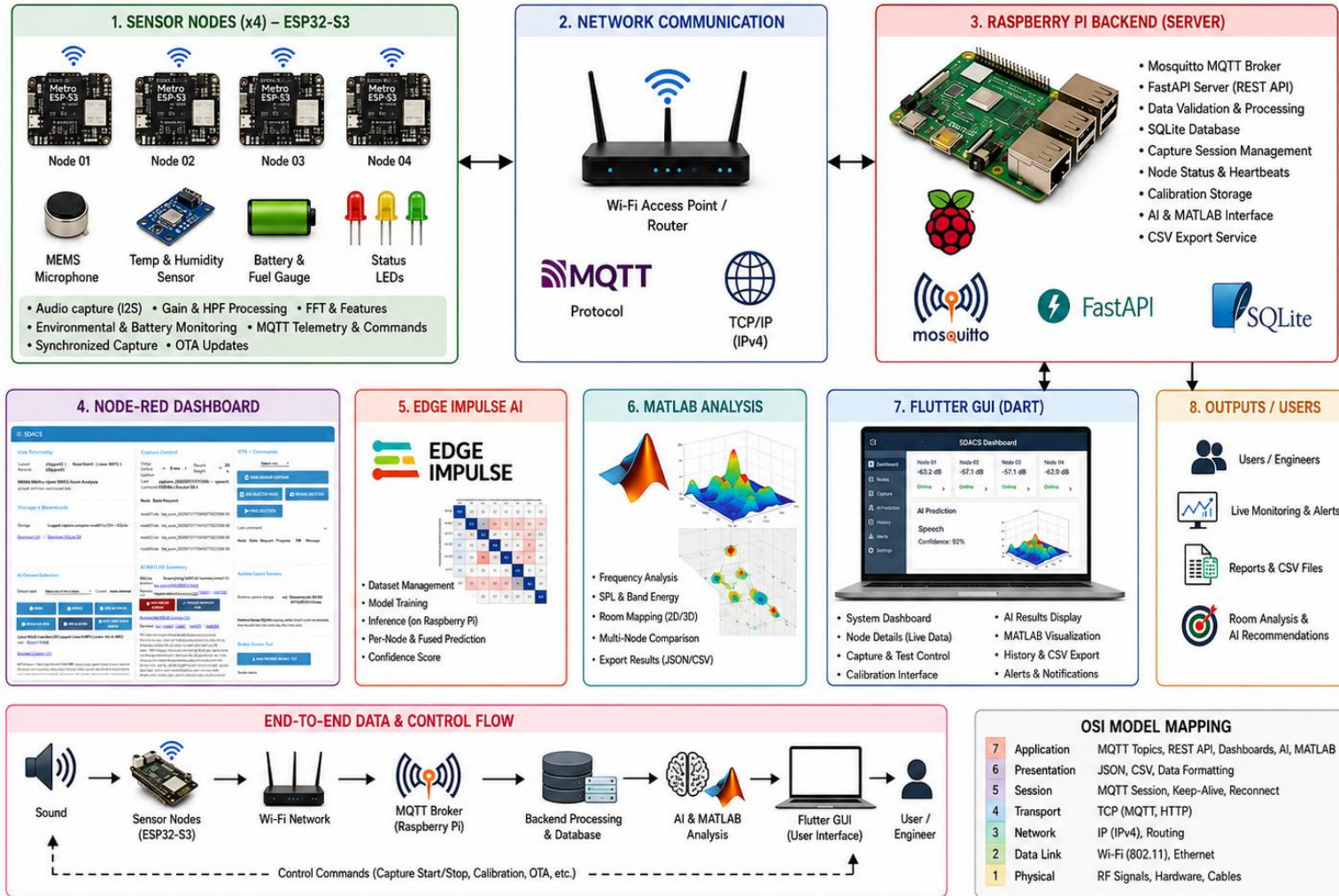


Figure 41:SDACS functional units and end-to-end system flo

## 6.2. Hardware

### 6.2.1. *Embedded hardware and PCB*

#### 6.2.1.1. Functional units

- ESP32-S3 microcontroller
- MEMS microphone interface
- Temperature and humidity sensor
- Battery and fuel-gauge circuitry
- Battery-status LEDs
- Power regulation
- PCB and electrical interconnections
- Local I<sup>2</sup>S, I<sup>2</sup>C, ADC, and GPIO interfaces

#### 6.2.1.2. OSI classification

Primarily supports **Layer 1 – Physical**, although the sensors and local buses are outside the network OSI stack.

### 6.2.1.3. Development and Testing evidence

- Photograph of the assembled PCB showing TH and MEMs mic

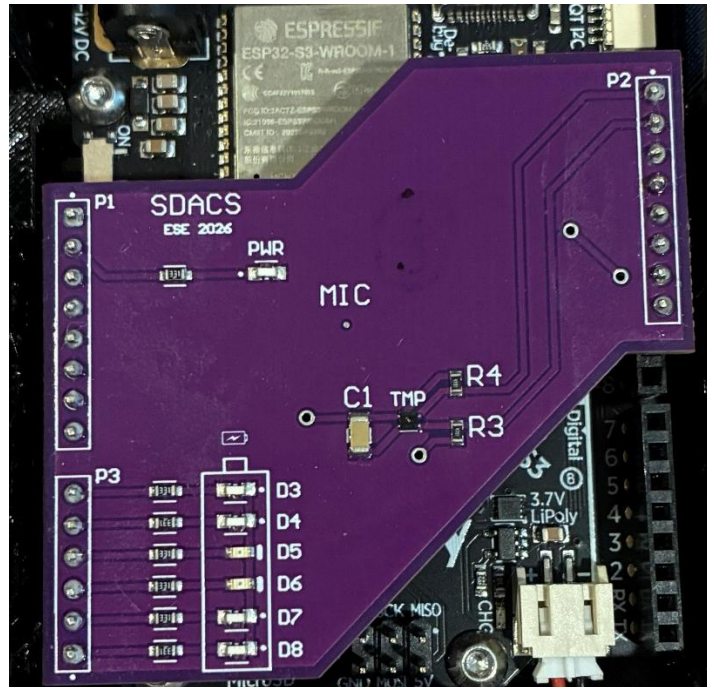


Figure 42: Custom Manufactured PCB (within enclosure)

- Battery (yellow light showing charge and LED array lit)



#### 6.2.1.4. System Integration and Performance evidence

- Photograph of all four assembled nodes

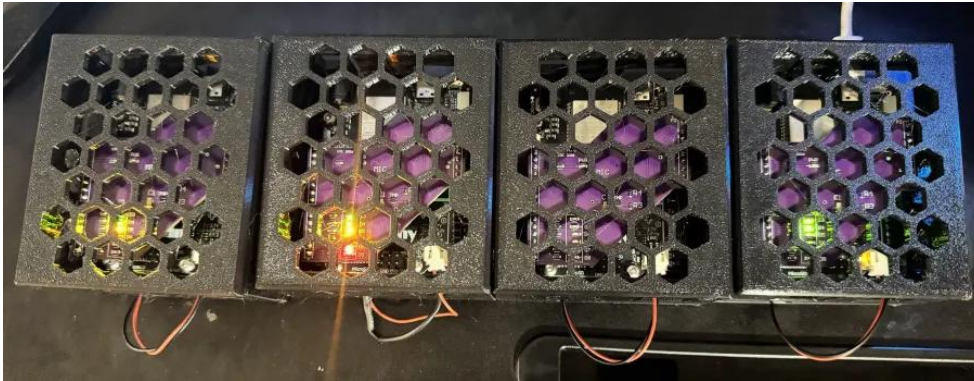


Figure 46: 4 Assembled Nodes

- Photograph showing nodes powered and operating simultaneously

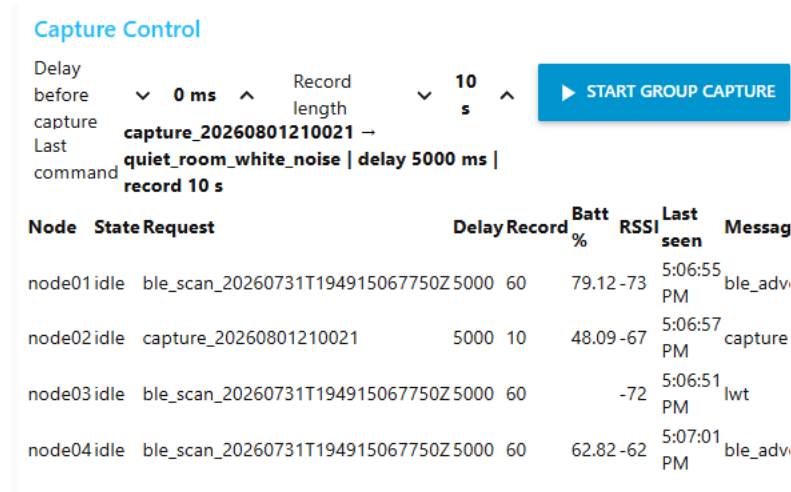


Figure 47: Group Capture Dashboard Module

- Evidence that all four nodes operate without power or communication failures (See GUI Functional Unit Testing Proof)

### 6.2.2. Mechanical Enclosure

#### 6.2.2.1. Functional units

- Sensor-node enclosure
- Microphone opening
- Ventilation openings
- PCB mounting
- Battery mounting
- Access to USB, reset, power, and status LEDs

#### 6.2.2.2. OSI classification

The enclosure is **outside the OSI model**, but it physically supports Layer 1 hardware.

#### 6.2.2.3. Development and Testing evidence

- 3D model

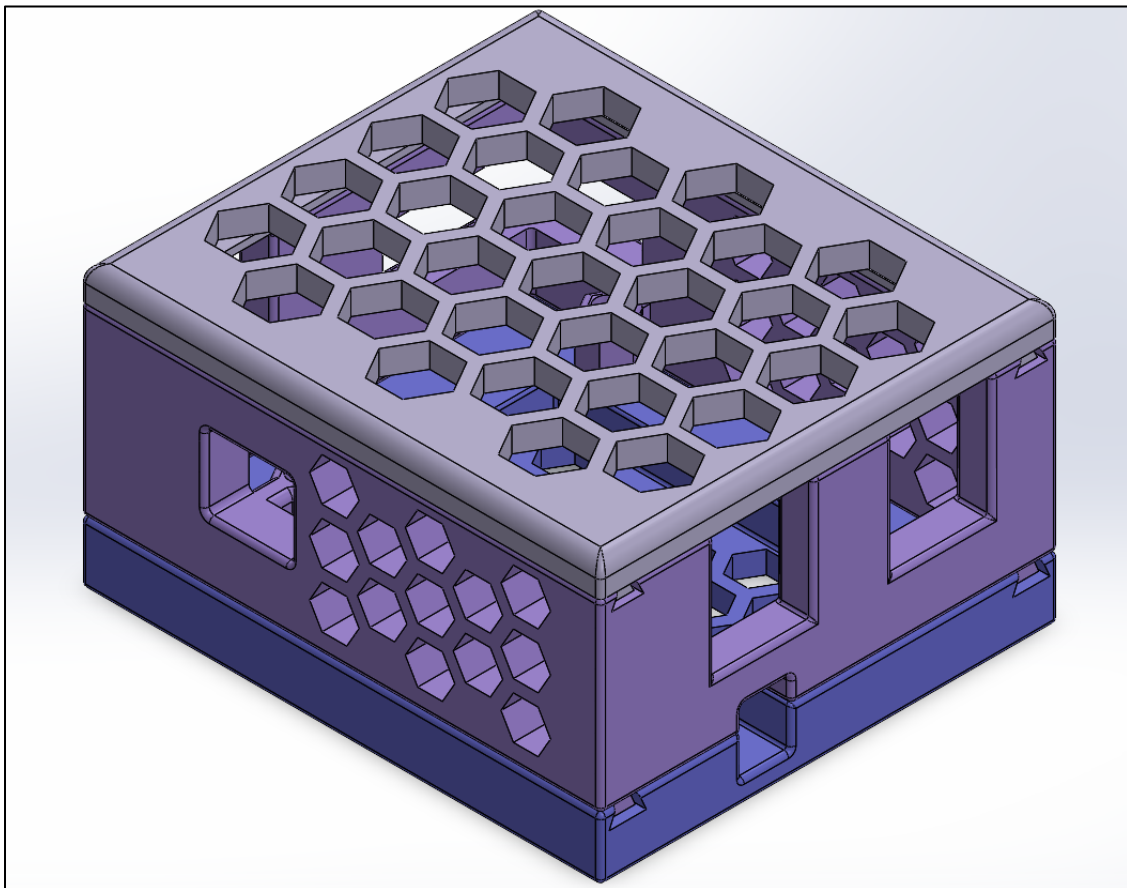
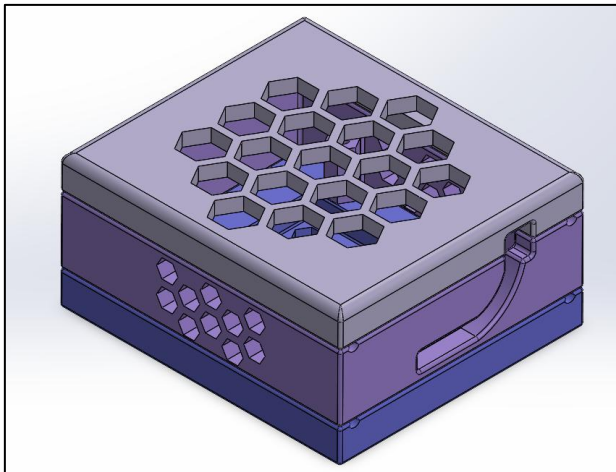


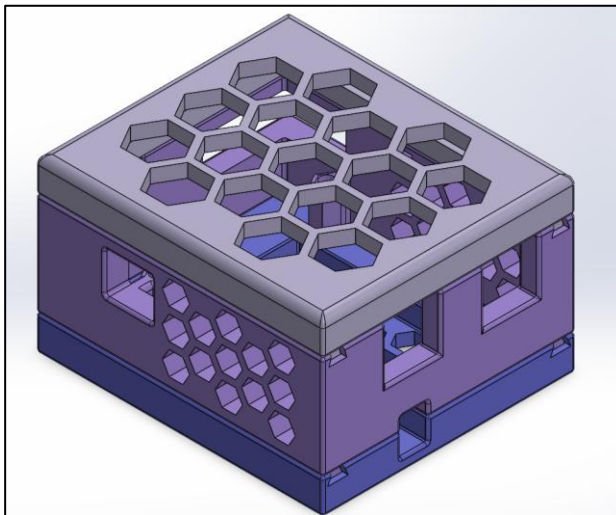
Figure 48: Custom Enclosure CAD Model

- Photographs of enclosure prototypes



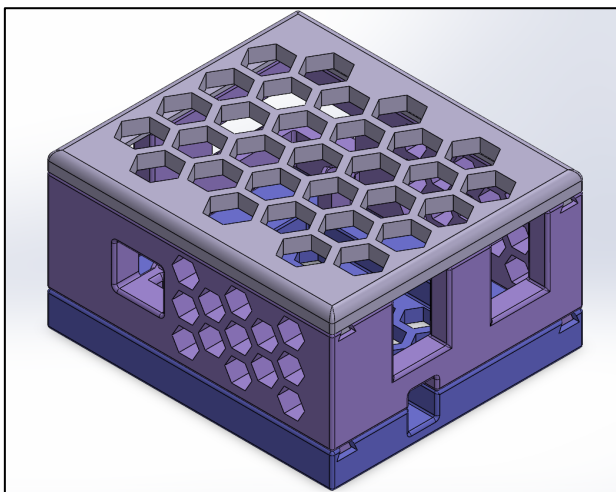
#### REV B: 1<sup>ST</sup> FULL ENCLOSURE

- Minimal ventilation to limit complexity
- Curved wire connector for the battery
- Thick enclosure walls
- Small vent port on lid



#### REV D: MID-DEVELOPMENT

- Larger access ports to allow button pushes, etc.
- Two-port wire connector for the battery because curve was having issues being printed
- Increased vent port on lid to minimize reflections for the mic



#### REV F: FINAL ENCLOSURE

- Lid was decreased in size to minimize enclosure gaps
- Shorter box size to reduce size and weight
- Light, minimal design for prototyping and testing

Figure 49: Custom Enclosure Progression (Rev B to Rev F)

#### 6.2.2.4. System Integration and Performance evidence

- Photograph of the final enclosed nodes positioned in the test room



*Figure 50: SDACS Lab Environment w/ Nodes Installed*

- Evidence that microphone and environmental-sensor openings remain functional



Figure 51: Custom Manufactured Enclosure, Fully Assembled Node

### 6.3. ESP-IDF Firmware and Embedded Signal Processing

#### 6.3.1. Functional units

- System startup
- Peripheral initialization
- I<sup>2</sup>S microphone capture
- Programmable software gain
- High-pass filtering
- RMS and dBFS calculation
- Estimated SPL
- FFT processing
- Peak-frequency detection
- Low-, mid-, and high-band energy calculation
- Temperature and humidity acquisition
- Battery monitoring
- Capture-state management
- Synchronized capture
- MQTT telemetry publication
- MQTT command reception
- Heartbeat and node status
- OTA firmware update support

#### 6.3.2. OSI classification

Table 7: Firmware OSI Classification Layers

| Firmware function                               | OSI layer  |
|-------------------------------------------------|------------|
| Wi-Fi driver and radio initialization           | Layers 1–2 |
| IP configuration                                | Layer 3    |
| TCP communication                               | Layer 4    |
| MQTT connection and reconnect logic             | Layer 5    |
| JSON payload creation                           | Layer 6    |
| MQTT topics, capture commands and OTA behaviour | Layer 7    |

### 6.3.3. Development and Testing evidence

- Firmware process-flow diagram

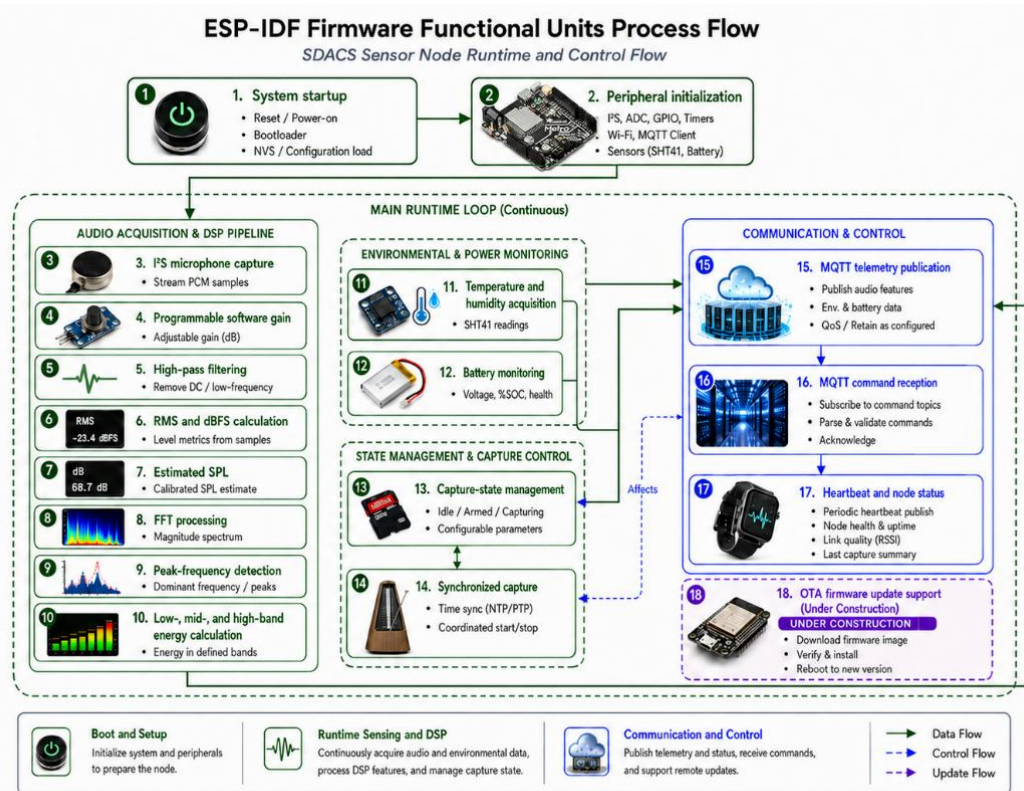


Figure 52: Firmware Process Flow Diagram

- Code reference to the ESP-IDF firmware branch

[https://github.com/AmyWentzel/CapstoneI\\_SDACS.git](https://github.com/AmyWentzel/CapstoneI_SDACS.git)

- Before-and-after results for software gain + before-and-after results for the high-pass filter

The programmable software gain increased PCM sample amplitude by a factor of eight, corresponding to an expected 18.06 dB increase in RMS and dBFS when clipping did not occur. Signed 24-bit saturation and clipping diagnostics were included to prevent numerical overflow. The fourth-order 150 Hz Butterworth high-pass filter reduced low-frequency interference by approximately 40.4 dB at 46.875 Hz and 16.4 dB at 93.75 Hz. Attenuation was approximately 3 dB at the cutoff frequency, while signals at 250 Hz and 1 kHz remained effectively unchanged.

Table 8: Before and After Addition of Gain

| Measurement          | Before gain    | After 8× gain                   |
|----------------------|----------------|---------------------------------|
| PCM sample amplitude | 1×             | 8×                              |
| Peak amplitude       | P              | approximately 8P                |
| RMS amplitude        | R              | approximately 8R                |
| dBFS                 | Original value | Original value + 18.06 dB       |
| FFT magnitude        | Original       | approximately 8×                |
| Clipping risk        | Lower          | Higher; protected by saturation |

Table 9: Before and After High Pass Filter

| Frequency  | Before HPF     | After HPF  | Reduction             |
|------------|----------------|------------|-----------------------|
| 46.875 Hz  | 0 dB reference | −40.431 dB | Very strong reduction |
| 93.75 Hz   | 0 dB reference | −16.415 dB | Strong reduction      |
| 140.625 Hz | 0 dB reference | −4.276 dB  | Moderate reduction    |
| 150 Hz     | 0 dB reference | −3.010 dB  | Cutoff point          |
| 250 Hz     | 0 dB reference | −0.072 dB  | Almost unchanged      |
| 1 kHz      | 0 dB reference | 0.000 dB   | Unchanged             |

- FFT tone-test results

Incremental single-node FFT testing was completed to evaluate the accuracy of the firmware's peak-frequency detection across the intended acoustic range. Controlled sinusoidal tones were applied at **100 Hz, 500 Hz, 2 kHz, 5 kHz, 10 kHz, and 15 kHz**. Each tone was recorded for approximately 20 seconds using a 48 kHz sampling rate, and the node's reported peak-frequency output was compared with the known input frequency.

Table 10: Observed Performance of Applied Tones

| Applied tone | Observed performance |
|--------------|----------------------|
|--------------|----------------------|

|               |                                                                                                                      |
|---------------|----------------------------------------------------------------------------------------------------------------------|
| <b>100 Hz</b> | Peak-frequency detection was unstable and could deviate noticeably from the applied tone.                            |
| <b>500 Hz</b> | Detection improved compared with 100 Hz but still showed measurable variation.                                       |
| <b>2 kHz</b>  | This was the least consistent test point, with the reported peak occasionally shifting away from the expected value. |
| <b>5 kHz</b>  | The detected peak closely followed the applied tone and was considered reliable.                                     |
| <b>10 kHz</b> | Strong agreement was observed between the applied and detected frequencies.                                          |
| <b>15 kHz</b> | The detected peak closely matched the applied tone and remained stable.                                              |

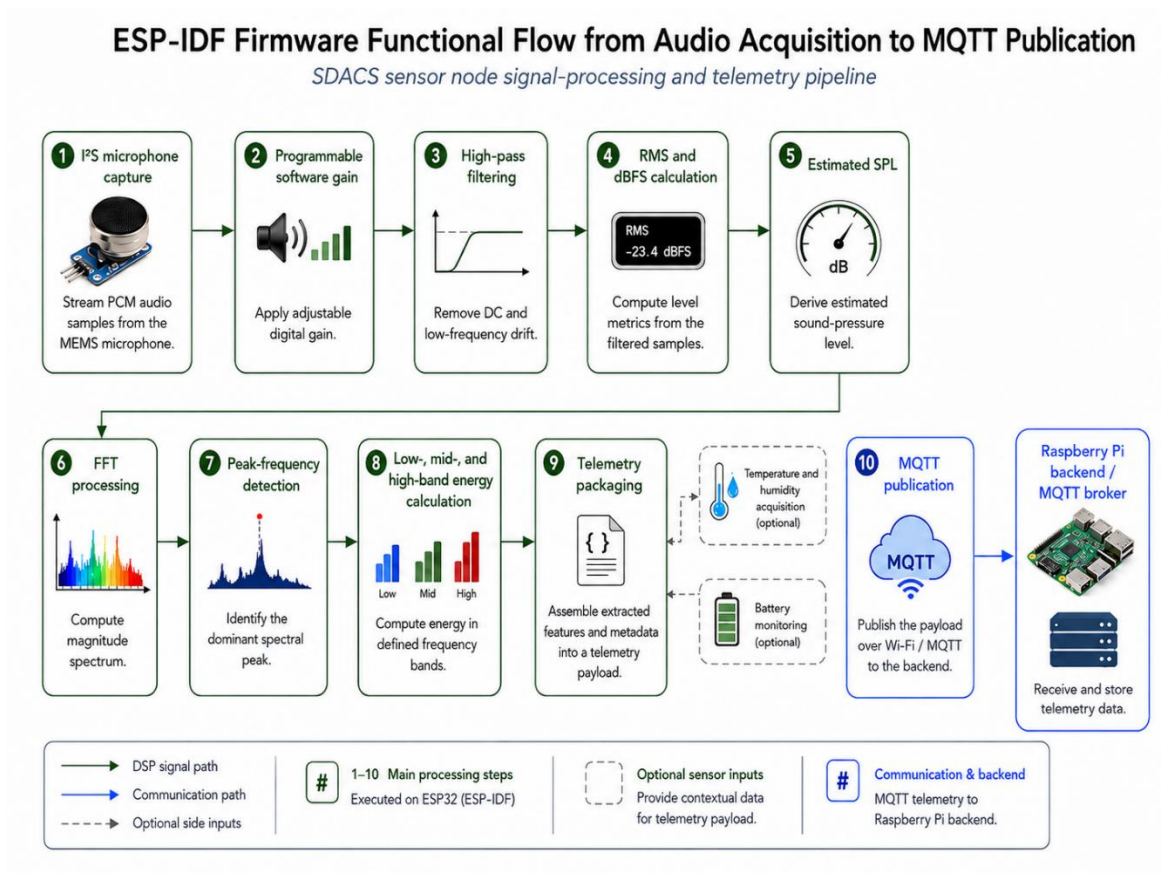


Figure 54: ESP-IDF firmware functional flow from audio acquisition to MQTT publication.

#### 6.3.4. System Integration and Performance evidence

Include:

- Simultaneous telemetry from all four nodes (Node-RED Capture Control)

**Node Health**

| Node   | State     | FW              | up SPL | Peak Hz | Acoustic Peak | Dominant Band | Ratio    | 1k Ratio | 1k Local | Con dB |
|--------|-----------|-----------------|--------|---------|---------------|---------------|----------|----------|----------|--------|
| node01 | capturing | mic-gain-hpf-v2 | 92.33  | 468.8   | 468.8         | low_mid       | 0.416787 | 0.018309 | 0.133202 | -8.75  |
| node02 | capturing | mic-gain-hpf-v2 | 92.97  | 46.9    | 328.1         | bass          | 0.461508 | 0.079532 | 0.294693 | -5.31  |
| node03 | capturing | mic-gain-hpf-v2 | 93.45  | 46.9    | 375           | bass          | 0.351741 | 0.095064 | 0.314713 | -5.02  |
| node04 | capturing | mic-gain-hpf-v2 | 92.7   | 46.9    | 234.4         | bass          | 0.381037 | 0.129982 | 0.904064 | -0.44  |

Figure 55: Node-RED Dashboard "Node Health" Module showing 4 Nodes Capturing

- Capture start and completion messages using the same request ID

**Capture Control**

Delay before capture: 0 ms | Record length: 5 s | **START GROUP CAPTURE**

Last command: **capture\_20260801220335 → quiet\_room\_white\_noise | delay 5000 ms | record 5 s**

| Node   | State     | Request                | Delay | Record | Batt % | RSSI | Last seen  | Message                           |
|--------|-----------|------------------------|-------|--------|--------|------|------------|-----------------------------------|
| node01 | capturing | capture_20260801220335 | 5000  | 5      | 65.71  | -71  | 6:03:42 PM | capture start MQTT/feat only mode |
| node02 | capturing | capture_20260801220335 | 5000  | 5      | 62.96  | -64  | 6:03:43 PM | capture start MQTT/feat only mode |
| node03 | capturing | capture_20260801220335 | 5000  | 5      |        | -70  | 6:03:42 PM | capture start MQTT/feat only mode |
| node04 | capturing | capture_20260801220335 | 5000  | 5      | 45.28  | -62  | 6:03:42 PM | capture start MQTT/feat only mode |

**Capture Control**

Delay before capture: 0 ms | Record length: 5 s | **START GROUP CAPTURE**

Last command: **capture\_20260801220335 → quiet\_room\_white\_noise | delay 5000 ms | record 5 s**

Requires one of the six dataset labels, then publishes start\_capture to all four nodes.

| Node   | State | Request                | Delay | Record | Batt % | RSSI | Last seen  | Message                  |
|--------|-------|------------------------|-------|--------|--------|------|------------|--------------------------|
| node01 | armed | capture_20260801220335 | 5000  | 5      | 65.71  | -71  | 6:03:36 PM | capture command accepted |
| node02 | armed | capture_20260801220335 | 5000  | 5      | 63.11  | -65  | 6:03:36 PM | capture command accepted |
| node03 | armed | capture_20260801220335 | 5000  | 5      |        | -70  | 6:03:37 PM | capture command accepted |
| node04 | armed | capture_20260801220335 | 5000  | 5      | 45.28  | -62  | 6:03:36 PM | capture command accepted |

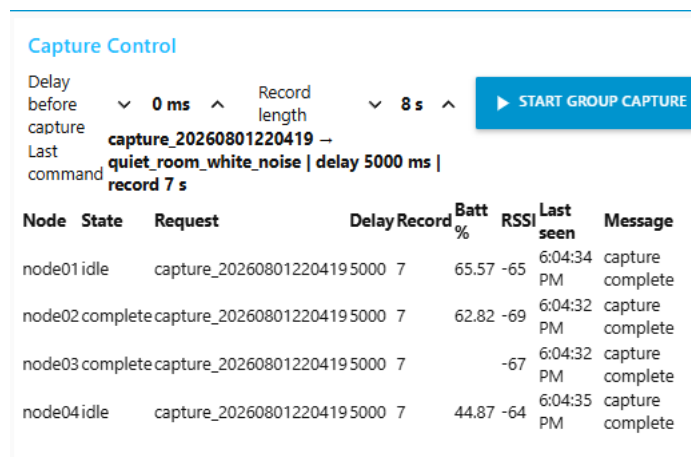


Figure 56: Node-RED Dashboard "Capture Control" Module showing 4 Nodes Capturing, Armed and Complete.

- Calibration results across all four nodes (Reference Item 9 Flutter/Dart GUI + Video and Show Screen-shots of UI)
- Known limitations, such as low-frequency accuracy or node-to-node microphone variation (Show warning from UI)

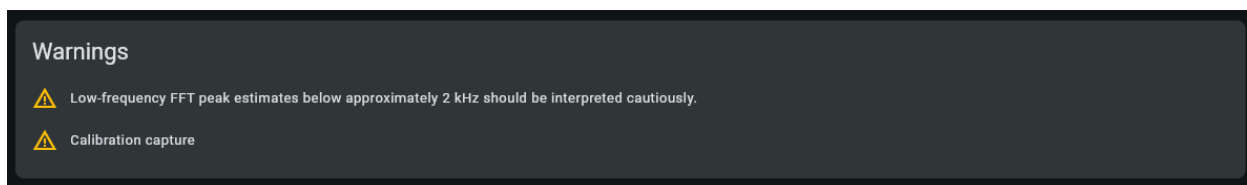


Figure 57: Warnings in SDACS Results Page

**NOTE:** See [Node Frequency Detection Analysis](#) in OneDrive [46].

## 6.4. Network and Communication

### 6.4.1. Functional units

- Wi-Fi network connection
- MQTT telemetry
- MQTT command distribution
- IPv4 addressing
- Raspberry Pi broker connection
- HTTP/REST API access
- Tailscale remote access, where used

### 6.4.2. OSI classification

This section covers all seven layers but mainly Layers 1–5.

| Communication item                     | OSI layer |
|----------------------------------------|-----------|
| Wi-Fi radio signal                     | Layer 1   |
| IEEE 802.11 connection                 | Layer 2   |
| ESP32 and Raspberry Pi IP addresses    | Layer 3   |
| MQTT and HTTP over TCP                 | Layer 4   |
| MQTT session, keep-alive and reconnect | Layer 5   |
| JSON message format                    | Layer 6   |
| MQTT topics and REST endpoints         | Layer 7   |

### 6.4.3. Development and Testing evidence

- Network architecture diagram

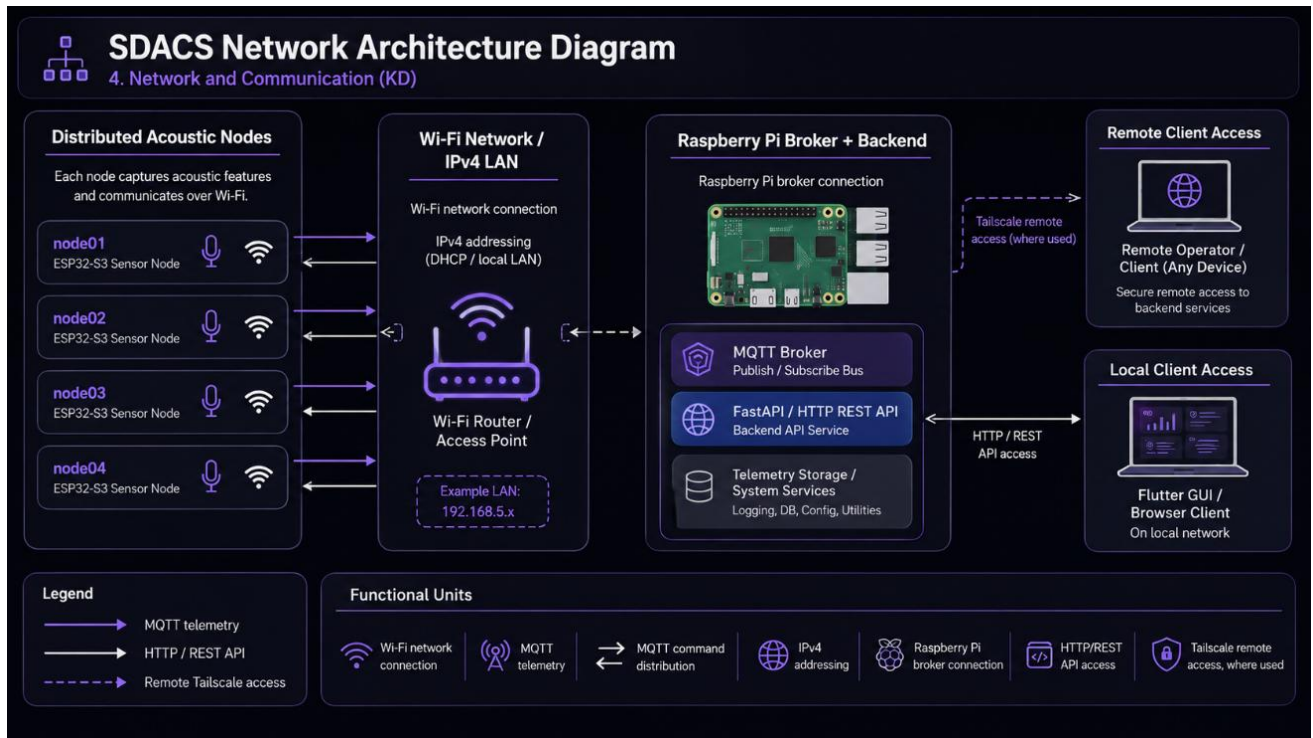


Figure 58: SDACS Network Architecture Diagram

- List of MQTT topics

Table 11: Core Node Topics

| MQTT topic                                               | Direction             | Purpose                                                                                                                        |
|----------------------------------------------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>sdacs/node/&lt;node_id&gt;/features</code>         | Node → broker/backend | Main acoustic telemetry: RMS, dBFS, estimated SPL, FFT peak frequency, band energies, gain/HPF diagnostics, and capture state. |
| <code>sdacs/node/&lt;node_id&gt;/temp_humidity</code>    | Node → broker/backend | Temperature and relative-humidity measurements.                                                                                |
| <code>sdacs/node/&lt;node_id&gt;/fuel_gauge</code>       | Node → broker/backend | Battery state of charge, voltage, validity, and charging information.                                                          |
| <code>sdacs/node/&lt;node_id&gt;/status/heartbeat</code> | Node → broker/backend | Periodic online status, Wi-Fi/MQTT condition, firmware version, uptime, and last-will offline status.                          |
| <code>sdacs/node/&lt;node_id&gt;/status</code>           | Node → broker/backend | Capture-state messages, command responses, acknowledgements, errors,                                                           |

|                                                    |                          |                                                                                      |
|----------------------------------------------------|--------------------------|--------------------------------------------------------------------------------------|
|                                                    |                          | and calibration-offset confirmation.                                                 |
| <b>sdacs/node/&lt;node_id&gt;/capture_complete</b> | Node → broker/backend    | Indicates that a synchronized capture has finished and provides capture metadata.    |
| <b>sdacs/node/&lt;node_id&gt;/ota/status</b>       | Node → broker/backend    | OTA update progress such as starting, downloading, verifying, restarting, or failed. |
| <b>sdacs/node/&lt;node_id&gt;/debug/i2s_diag</b>   | Node → diagnostic client | Detailed I <sup>2</sup> S microphone diagnostic results requested through a command. |

Table 12: Command Topics

| MQTT topic                            | Direction                           | Purpose                                                    |
|---------------------------------------|-------------------------------------|------------------------------------------------------------|
| <b>sdacs/node/&lt;node_id&gt;/cmd</b> | <b>Backend/Node-RED → one node</b>  | <b>Sends a command to one selected node.</b>               |
| <b>sdacs/group/all/cmd</b>            | <b>Backend/Node-RED → all nodes</b> | <b>Broadcasts synchronized commands to all four nodes.</b> |

Table 13: AI and BLE-related topics

| MQTT topic                                    | Purpose                                                                                                                        |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>sdacs/node/&lt;node_id&gt;/acoustic_ai</b> | Carries acoustic AI classifications or inference results associated with a node. The FastAPI backend subscribes to this topic. |
| <b>sdacs/node/&lt;node_id&gt;/presence</b>    | Reports BLE-derived node presence, RSSI, or proximity information.                                                             |
| <b>sdacs/ble/gateway/rpi5/cmd</b>             | Sends commands such as scan_now to the Raspberry Pi BLE gateway.                                                               |
| <b>sdacs/ble/gateway/rpi5/status</b>          | Reports BLE gateway state and scan information.                                                                                |
| <b>sdacs/site/topology</b>                    | Publishes the combined node-presence and proximity topology generated by the BLE gateway.                                      |

## Topic summary

The complete active and optional topic set is:

sdacs/node/<node\_id>/features

sdacs/node/<node\_id>/temp\_humidity

sdacs/node/<node\_id>/fuel\_gauge

sdacs/node/<node\_id>/status/heartbeat  
sdacs/node/<node\_id>/status  
sdacs/node/<node\_id>/capture\_complete  
sdacs/node/<node\_id>/ota/status  
sdacs/node/<node\_id>/debug/i2s\_diag  
sdacs/node/<node\_id>/cmd  
sdacs/node/<node\_id>/presence  
sdacs/node/<node\_id>/acoustic\_ai  
sdacs/group/all/cmd  
sdacs/ble/gateway/rpi5/cmd  
sdacs/ble/gateway/rpi5/status  
sdacs/site/topology

- Example MQTT payloads

```

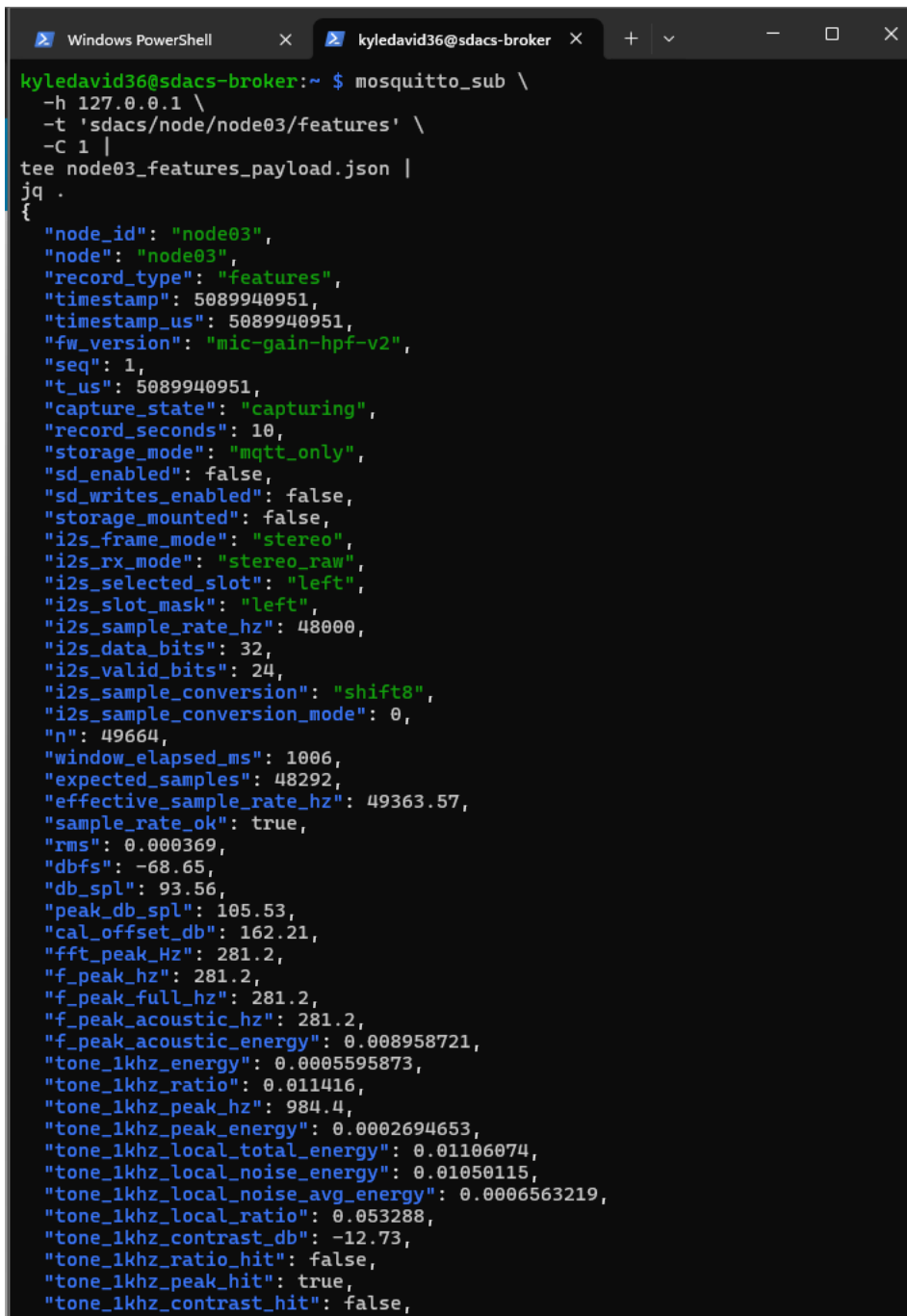
Windows PowerShell
kyledavid36@sdacs-broker

kyledavid36@sdacs-broker:~ $ mosquitto_sub \
  -h 127.0.0.1 \
  -t 'sdacs/node/node03/features' \
  -C 1
{"node_id":"node03","node":"node03","record_type":"features","timestamp":49
97870958,"timestamp_us":4997870958,"fw_version":"mic-gain-hpf-v2","seq":1,"
t_us":4997870958,"capture_state":"capturing","record_seconds":10,"storage_m
ode":"mqtt_only","sd_enabled":false,"sd_writes_enabled":false,"storage_moun
ted":false,"i2s_frame_mode":"stereo","i2s_rx_mode":"stereo_raw","i2s_select
ed_slot":"left","i2s_slot_mask":"left","i2s_sample_rate_hz":48000,"i2s_data
_bits":32,"i2s_valid_bits":24,"i2s_sample_conversion":"shift8","i2s_sample_
conversion_mode":0,"n":49664,"window_elapsed_ms":1005,"expected_samples":48
287,"effective_sample_rate_hz":49368.73,"sample_rate_ok":true,"rms":0.00036
8,"dbfs":-68.69,"db_spl":93.52,"peak_db_spl":106.13,"cal_offset_db":162.21,
"fft_peak_hz":93.8,"f_peak_hz":93.8,"f_peak_full_hz":93.8,"f_peak_acoustic_
hz":468.8,"f_peak_acoustic_energy":1.433569e-03,"tone_1khz_energy":3.149888
e-03,"tone_1khz_ratio":0.123554,"tone_1khz_peak_hz":937.5,"tone_1khz_peak_e
nergy":9.335628e-04,"tone_1khz_local_total_energy":7.499369e-03,"tone_1khz_
local_noise_energy":4.349482e-03,"tone_1khz_local_noise_avg_energy":2.71842
6e-04,"tone_1khz_local_ratio":0.724198,"tone_1khz_contrast_db":-1.40,"tone_
1khz_ratio_hit":false,"tone_1khz_peak_hit":true,"tone_1khz_contrast_hit":fa
lse,"tone_1khz_detected":false,"tone_1khz_local_noise_bins":16,"tone_1khz_b
and_bins":6,"acoustic_band_energy":2.549403e-02,"room_band_total_energy":3.
645195e-02,"low_rumble_energy":1.095792e-02,"low_rumble_ratio":0.331481,"ba
nd_sub_energy":0.000000e+00,"band_bass_energy":9.379448e-03,"band_low_mid_e
nergy":5.484819e-03,"band_mid_energy":9.357798e-03,"band_presence_energy":5
.794785e-03,"band_high_energy":6.435105e-03,"band_sub_ratio":0.000000,"band
_bass_ratio":0.257310,"band_low_mid_ratio":0.150467,"band_mid_ratio":0.2567
16,"band_presence_ratio":0.158971,"band_high_ratio":0.176537,"band_sub_peak
_hz":0.0,"band_bass_peak_hz":93.8,"band_low_mid_peak_hz":187.5,"band_mid_pe
ak_hz":562.5,"band_presence_peak_hz":3703.1,"band_high_peak_hz":8625.0,"ban
d_sub_peak_energy":0.000000e+00,"band_bass_peak_energy":7.037619e-03,"band_
low_mid_peak_energy":1.578474e-03,"band_mid_peak_energy":1.017578e-03,"band
_presence_peak_energy":6.213540e-04,"band_high_peak_energy":3.001548e-04,"d
ominant_band_name":"bass","dominant_band_ratio":0.257310,"dominant_band_pea
k_hz":93.8,"fft_low_ratio":0.338836,"fft_mid_ratio":0.393889,"fft_high_rati
o":0.267275,"fft_total_energy":3.305750e-02,"p2p_raw":25120,"zeros":65,"tem
p_c":29.01,"rh_percent":50.74,"batt_soc_percent":null,"batt_voltage_v":null
,"batt_charge_rate_pct_per_hr":null,"batt_valid":false,"storage_error":"","
storage_error_detail":"","audio_error":"i2s_timeouts","audio_read_errors":0
,"audio_read_timeouts":98,"consecutive_timeouts":1,"total_i2s_reads":292,"s
uccessful_i2s_reads":194,"err":1,"scene_metrics_valid":true,"scene_hpf_enab
led":true,"scene_hpf_cutoff_hz":150.0,"scene_hpf_order":4,"scene_software_g
ain":16.000,"scene_rms":0.000542,"scene_dbfs":-65.32,"scene_db_spl":96.89,"
scene_peak_db_spl":109.11,"scene_f_peak_hz":187.5,"scene_f_peak_acoustic_hz
":468.8,"scene_low_rumble_ratio":0.163759,"scene_band_bass_ratio":0.068155,
"scene_band_low_mid_ratio":0.207937,"scene_band_mid_ratio":0.310908,"scene_
band_presence_ratio":0.195639,"scene_band_high_ratio":0.217361,"scene_band_
bass_peak_hz":140.6,"scene_band_low_mid_peak_hz":187.5,"scene_band_mid_peak
_hz":562.5,"scene_band_presence_peak_hz":3703.1,"scene_band_high_peak_hz":8
625.0,"scene_dominant_band_name":"mid","scene_dominant_band_ratio":0.310908

```

Figure 59:MQTT Payload

Or properly formatted as a json:



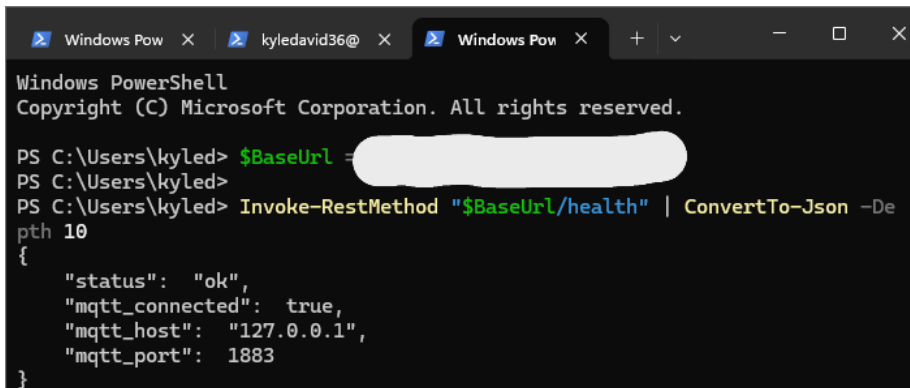
```

Windows PowerShell
kyledavid36@sdacs-broker:~ $ mosquitto_sub \
-h 127.0.0.1 \
-t 'sdacs/node/node03/features' \
-C 1 |
tee node03_features_payload.json |
jq .
{
  "node_id": "node03",
  "node": "node03",
  "record_type": "features",
  "timestamp": 5089940951,
  "timestamp_us": 5089940951,
  "fw_version": "mic-gain-hpf-v2",
  "seq": 1,
  "t_us": 5089940951,
  "capture_state": "capturing",
  "record_seconds": 10,
  "storage_mode": "mqtt_only",
  "sd_enabled": false,
  "sd_writes_enabled": false,
  "storage_mounted": false,
  "i2s_frame_mode": "stereo",
  "i2s_rx_mode": "stereo_raw",
  "i2s_selected_slot": "left",
  "i2s_slot_mask": "left",
  "i2s_sample_rate_hz": 48000,
  "i2s_data_bits": 32,
  "i2s_valid_bits": 24,
  "i2s_sample_conversion": "shift8",
  "i2s_sample_conversion_mode": 0,
  "n": 49664,
  "window_elapsed_ms": 1006,
  "expected_samples": 48292,
  "effective_sample_rate_hz": 49363.57,
  "sample_rate_ok": true,
  "rms": 0.000369,
  "dbfs": -68.65,
  "db_spl": 93.56,
  "peak_db_spl": 105.53,
  "cal_offset_db": 162.21,
  "fft_peak_Hz": 281.2,
  "f_peak_hz": 281.2,
  "f_peak_full_hz": 281.2,
  "f_peak_acoustic_hz": 281.2,
  "f_peak_acoustic_energy": 0.008958721,
  "tone_1khz_energy": 0.0005595873,
  "tone_1khz_ratio": 0.011416,
  "tone_1khz_peak_hz": 984.4,
  "tone_1khz_peak_energy": 0.0002694653,
  "tone_1khz_local_total_energy": 0.01106074,
  "tone_1khz_local_noise_energy": 0.01050115,
  "tone_1khz_local_noise_avg_energy": 0.0006563219,
  "tone_1khz_local_ratio": 0.053288,
  "tone_1khz_contrast_db": -12.73,
  "tone_1khz_ratio_hit": false,
  "tone_1khz_peak_hit": true,
  "tone_1khz_contrast_hit": false,
}

```

Figure 60: MQTT Payload Formatted as a JSON

- API endpoint tests using PowerShell or a browser.
  1. Confirm the backend is reachable:



```

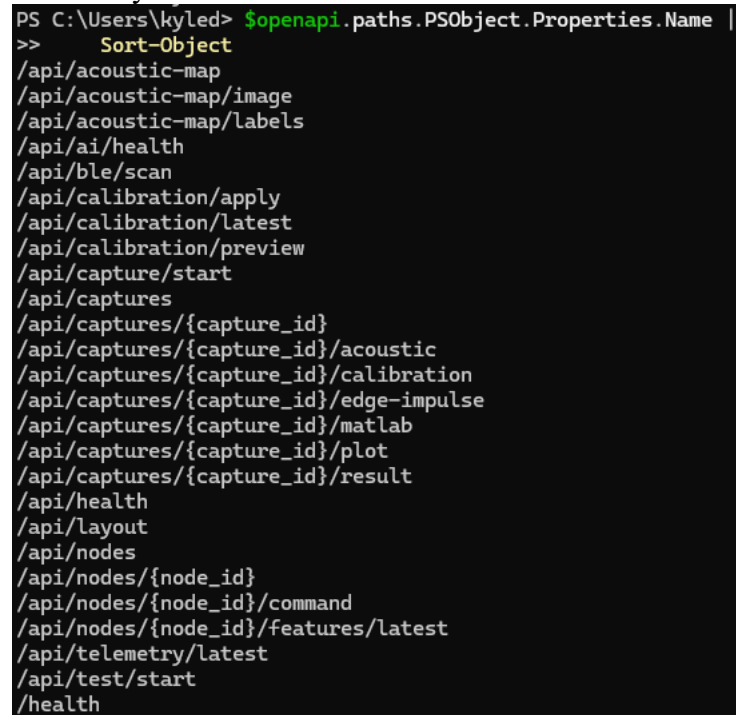
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\kyled> $BaseUrl = [REDACTED]
PS C:\Users\kyled>
PS C:\Users\kyled> Invoke-RestMethod "$BaseUrl/health" | ConvertTo-Json -Depth 10
{
  "status": "ok",
  "mqtt_connected": true,
  "mqtt_host": "127.0.0.1",
  "mqtt_port": 1883
}

```

Figure 61: Confirms Backend Endpoints Can be Reached at <BaseUrl>

## 2. List Every API Path



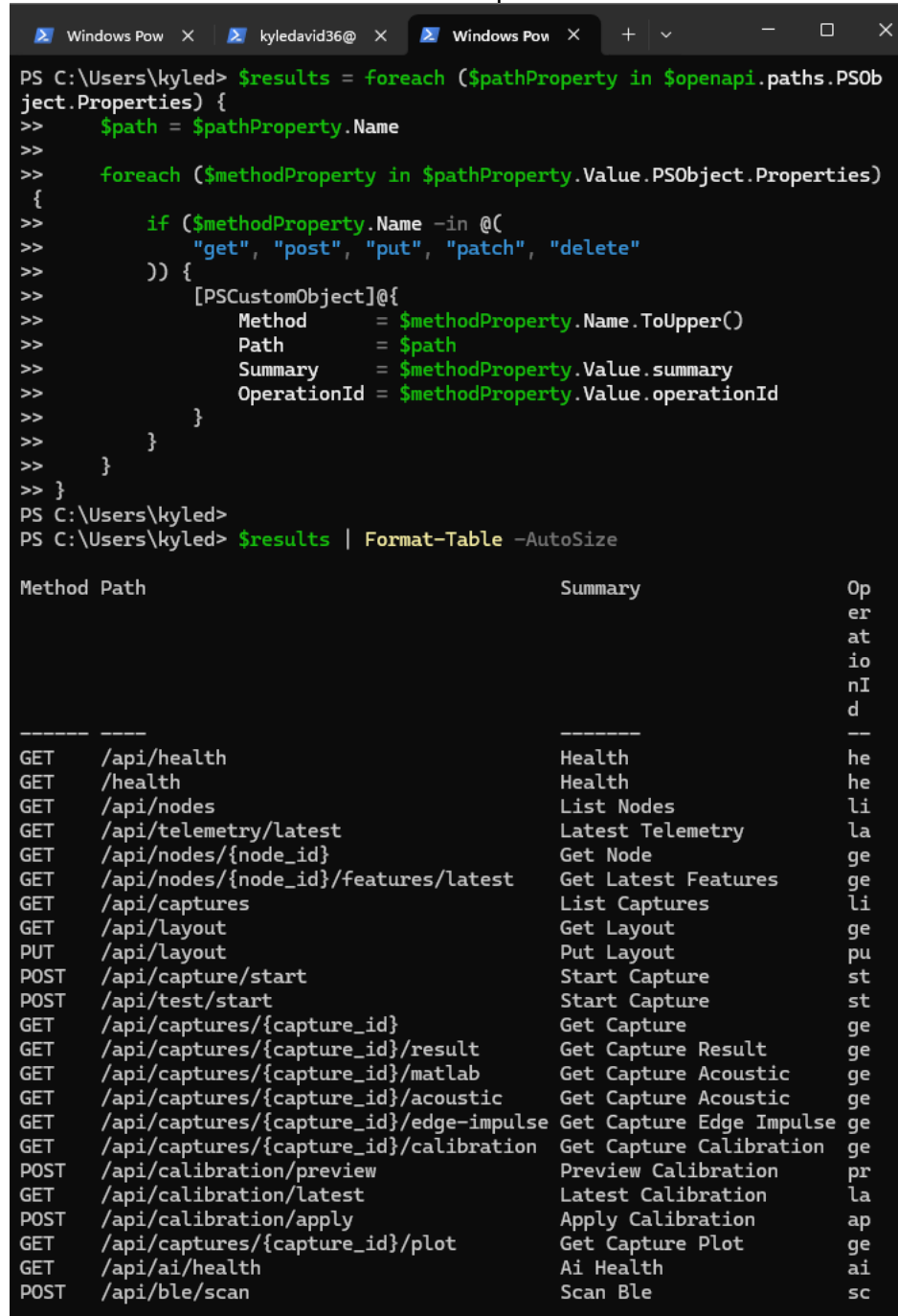
```

PS C:\Users\kyled> $openapi.paths.PSObject.Properties.Name |
>> Sort-Object
/api/acoustic-map
/api/acoustic-map/image
/api/acoustic-map/labels
/api/ai/health
/api/ble/scan
/api/calibration/apply
/api/calibration/latest
/api/calibration/preview
/api/capture/start
/api/captures
/api/captures/{capture_id}
/api/captures/{capture_id}/acoustic
/api/captures/{capture_id}/calibration
/api/captures/{capture_id}/edge-impulse
/api/captures/{capture_id}/matlab
/api/captures/{capture_id}/plot
/api/captures/{capture_id}/result
/api/health
/api/layout
/api/nodes
/api/nodes/{node_id}
/api/nodes/{node_id}/command
/api/nodes/{node_id}/features/latest
/api/telemetry/latest
/api/test/start
/health

```

Figure 62: List of API Paths

## 3. Show HTTP Methods Beside Each Endpoint:



```

PS C:\Users\kyled> $results = foreach ($pathProperty in $openapi.paths.PSObject.Properties) {
>>     $path = $pathProperty.Name
>>
>>     foreach ($methodProperty in $pathProperty.Value.PSObject.Properties)
>>     {
>>         if ($methodProperty.Name -in @(
>>             "get", "post", "put", "patch", "delete"
>>         )) {
>>             [PSCustomObject]@{
>>                 Method      = $methodProperty.Name.ToUpper()
>>                 Path        = $path
>>                 Summary     = $methodProperty.Value.summary
>>                 OperationId = $methodProperty.Value.operationId
>>             }
>>         }
>>     }
>> }
PS C:\Users\kyled>
PS C:\Users\kyled> $results | Format-Table -AutoSize

```

| Method | Path                                    | Summary                  | Operation ID          |
|--------|-----------------------------------------|--------------------------|-----------------------|
| GET    | /api/health                             | Health                   | health                |
| GET    | /health                                 | Health                   | health                |
| GET    | /api/nodes                              | List Nodes               | listNodes             |
| GET    | /api/telemetry/latest                   | Latest Telemetry         | latestTelemetry       |
| GET    | /api/nodes/{node_id}                    | Get Node                 | getNode               |
| GET    | /api/nodes/{node_id}/features/latest    | Get Latest Features      | getLatestFeatures     |
| GET    | /api/captures                           | List Captures            | listCaptures          |
| GET    | /api/layout                             | Get Layout               | getLayout             |
| PUT    | /api/layout                             | Put Layout               | putLayout             |
| POST   | /api/capture/start                      | Start Capture            | startCapture          |
| POST   | /api/test/start                         | Start Capture            | startCapture          |
| GET    | /api/captures/{capture_id}              | Get Capture              | getCapture            |
| GET    | /api/captures/{capture_id}/result       | Get Capture Result       | getCaptureResult      |
| GET    | /api/captures/{capture_id}/matlab       | Get Capture Acoustic     | getCaptureAcoustic    |
| GET    | /api/captures/{capture_id}/acoustic     | Get Capture Acoustic     | getCaptureAcoustic    |
| GET    | /api/captures/{capture_id}/edge-impulse | Get Capture Edge Impulse | getCaptureEdgeImpulse |
| GET    | /api/captures/{capture_id}/calibration  | Get Capture Calibration  | getCaptureCalibration |
| POST   | /api/calibration/preview                | Preview Calibration      | previewCalibration    |
| GET    | /api/calibration/latest                 | Latest Calibration       | latestCalibration     |
| POST   | /api/calibration/apply                  | Apply Calibration        | applyCalibration      |
| GET    | /api/captures/{capture_id}/plot         | Get Capture Plot         | getCapturePlot        |
| GET    | /api/ai/health                          | Ai Health                | aiHealth              |
| POST   | /api/ble/scan                           | Scan Ble                 | scanBle               |

Figure 63: HTTP Methods

## 4. Calibration Endpoints:

```

>> $results |
>> Where-Object { $_.Path -like "*calibration*" } |
>> Format-Table -AutoSize

```

| Method | Path                                   | Summary                 | OperationId |
|--------|----------------------------------------|-------------------------|-------------|
| GET    | /api/captures/{capture_id}/calibration | Get Capture Calibration | g...        |
| POST   | /api/calibration/preview               | Preview Calibration     | p...        |
| GET    | /api/calibration/latest                | Latest Calibration      | l...        |
| POST   | /api/calibration/apply                 | Apply Calibration       | a...        |

Figure 64: Calibration Endpoints

## 5. Capture Endpoints:

```

PS C:\Users\kyled> $results |
>> Where-Object { $_.Path -like "*capture*" } |
>> Format-Table -AutoSize

```

| Method | Path                                    | Summary                  | OperationId |
|--------|-----------------------------------------|--------------------------|-------------|
| GET    | /api/captures                           | List Captures            | li          |
| POST   | /api/capture/start                      | Start Capture            | st          |
| GET    | /api/captures/{capture_id}              | Get Capture              | ge          |
| GET    | /api/captures/{capture_id}/result       | Get Capture Result       | ge          |
| GET    | /api/captures/{capture_id}/matlab       | Get Capture Acoustic     | ge          |
| GET    | /api/captures/{capture_id}/acoustic     | Get Capture Acoustic     | ge          |
| GET    | /api/captures/{capture_id}/edge-impulse | Get Capture Edge Impulse | ge          |
| GET    | /api/captures/{capture_id}/calibration  | Get Capture Calibration  | ge          |
| GET    | /api/captures/{capture_id}/plot         | Get Capture Plot         | ge          |

Figure 65: Capture Endpoints

## 6.

## 7. Node Endpoints:

```

PS C:\Users\kyled> $results |
>> Where-Object { $_.Path -like "*nodes*" } |
>> Format-Table -AutoSize

```

| Method | Path                                 | Summary             | OperationId |
|--------|--------------------------------------|---------------------|-------------|
| GET    | /api/nodes                           | List Nodes          | list_no...  |
| GET    | /api/nodes/{node_id}                 | Get Node            | get_nod...  |
| GET    | /api/nodes/{node_id}/features/latest | Get Latest Features | get_lat...  |
| POST   | /api/nodes/{node_id}/command         | Node Command        | node_co...  |

Figure 66: Node Endpoints



#### 6.4.4. System Integration and Performance evidence

- Four nodes connected to the broker simultaneously (Node-RED)

Node Health

| Node   | State | FW              | dB SPL | Peak Hz | Acoustic Peak | Dominant Band | Band 1k Ratio | 1k Ratio | Contrast Local | 1k? dB | Batt % | RSSI |
|--------|-------|-----------------|--------|---------|---------------|---------------|---------------|----------|----------------|--------|--------|------|
| node01 | idle  | mic-gain-hpf-v2 |        |         |               |               |               |          |                |        | 64.41  | -68  |
| node02 | idle  | mic-gain-hpf-v2 |        |         |               |               |               |          |                |        | 62.1   | -67  |
| node03 | idle  | mic-gain-hpf-v2 |        |         |               |               |               |          |                |        |        | -65  |
| node04 | idle  | mic-gain-       |        |         |               |               |               |          |                |        | 42.82  | -65  |

Figure 67: Shows 4 Nodes Connected to Node-RED at Once

- Screenshot of MQTT traffic from all four node IDs (Backend Listening Script)

```

PS C:\Users\kyled> ssh kyledavid36@sdacs-broker
kyledavid36@sdacs-broker's password:
Linux sdacs-broker 6.18.34+rpt-rpi-2712 #1 SMP PREEMPT Debian 1:6.18.34-1+rpt1 (2026-06-09) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Jul 31 10:21:43 2026 from fe80::7666:ca61:d0f2:b49c%wlan0
kyledavid36@sdacs-broker:~$ mosquitto_sub -h localhost -v -t 'sdacs/#'
sdacs/node/node03/status {"node_id":"node03","record_type":"capture_status",
"timestamp":3566581163,"fw_version":"mic-gain-hpf-v2","request_id":"capture_20260801220419","state":"complete","delay_ms":5000,"record_seconds":7,"call_offset_db":162.21,"storage_mode":"mqtt_only","sd_enabled":false,"sd_write_s_enabled":false,"message":"capture complete"}
sdacs/node/node03/status/heartbeat {"node_id":"node03","record_type":"heart
beat","timestamp":4411736115,"status":"online","fw_version":"mic-gain-hpf-v2","capture_state":"idle","uptime_s":4396,"rssi_dbm":-70,"free_heap":116460,"wifi_connected":true,"mqtt_connected":true,"ota_ready":true,"ota_in_progress":false,"batt_soc_percent":null,"batt_voltage_v":null,"batt_charge_rate_pct_per_hr":null,"batt_sample_count":2200,"batt_error_count":2200,"batt_last_sample_time_us":4410696128,"batt_valid":false,"storage_mounted":false,"storage_last_error":"SD_DISABLED","storage_error_detail":"disabled by build c

```

Figure 68: MQTT Subscribe Listening Script Showing 4 Nodes are Active

- Evidence that one disconnected node does not prevent the other nodes from operating (GUI showing missing node 3 + Capture)

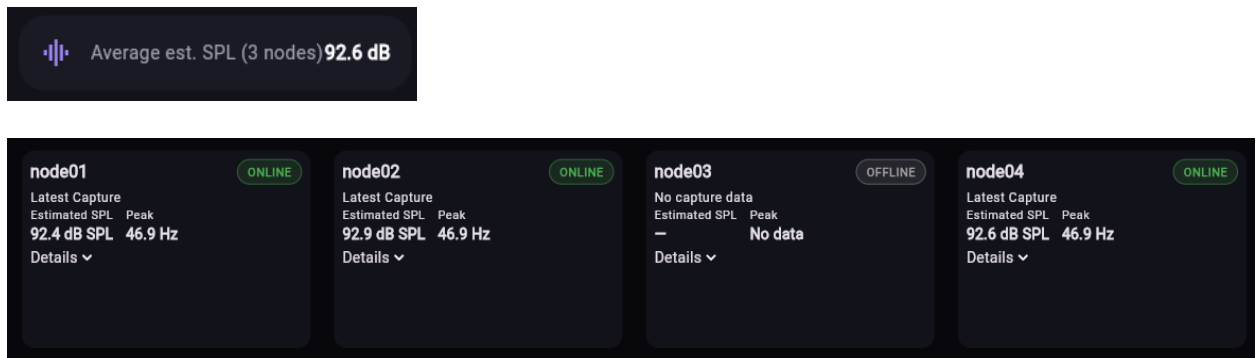


Figure 69: Shows 1 Missing Node Offline (3 / 4 Nodes)

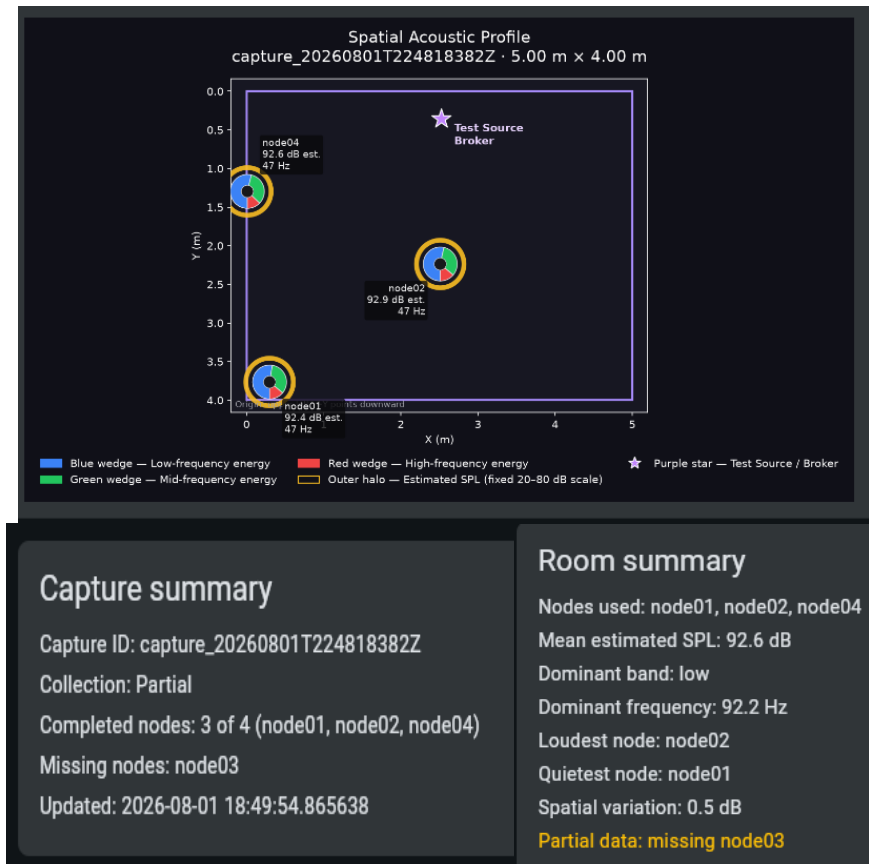


Figure 70: Shows 3 / 4 Nodes Captured

## 6.5. Middleware and Raspberry Pi Backend

### 6.5.1. Functional units

- Mosquitto MQTT broker
- FastAPI server
- MQTT client
- Telemetry validation
- SQLite database
- Capture-session management
- Node-status tracking
- Calibration storage
- AI inference interface
- MATLAB-result interface
- CSV export
- System service management

### *6.5.2. OSI classification*

Primarily:

- **Layer 5:** communication-session management
- **Layer 6:** JSON and database-field representation
- **Layer 7:** MQTT broker, FastAPI, storage and processing services

### *6.5.3. Development and Testing evidence*

- Backend architecture diagram
- Screenshot of the FastAPI service running (restart API script + /docs page)
- systemctl or journalctl output
- MQTT subscription logs
- Example API JSON response
- SQLite table or query output
- CSV creation evidence (Node-RED AI CSV)
- Reference to backend Python files

#### 6.5.4. System Integration and Performance Evidence



```

kyledavid36@sdacs-broker:~ $ mosquitto_sub \
  -h 127.0.0.1 \
  -t 'sdacs/node/node03/features' \
  -C 1
{"node_id":"node03","node":"node03","record_type":"features","timestamp":49
97870958,"timestamp_us":4997870958,"fw_version":"mic-gain-hpf-v2","seq":1,"
t_us":4997870958,"capture_state":"capturing","record_seconds":10,"storage_m
ode":"mqtt_only","sd_enabled":false,"sd_writes_enabled":false,"storage_moun
ted":false,"i2s_frame_mode":"stereo","i2s_rx_mode":"stereo_raw","i2s_select
ed_slot":"left","i2s_slot_mask":"left","i2s_sample_rate_hz":48000,"i2s_data
_bits":32,"i2s_valid_bits":24,"i2s_sample_conversion":"shift8","i2s_sample_
conversion_mode":0,"n":49664,"window_elapsed_ms":1005,"expected_samples":48
287,"effective_sample_rate_hz":49368.73,"sample_rate_ok":true,"rms":0.00036
8,"dbfs":-68.69,"db_spl":93.52,"peak_db_spl":106.13,"cal_offset_db":162.21,
"fft_peak_hz":93.8,"f_peak_hz":93.8,"f_peak_full_hz":93.8,"f_peak_acoustic_
hz":468.8,"f_peak_acoustic_energy":1.433569e-03,"tone_1khz_energy":3.149888
e-03,"tone_1khz_ratio":0.123554,"tone_1khz_peak_hz":937.5,"tone_1khz_peak_e
nergy":9.335628e-04,"tone_1khz_local_total_energy":7.499369e-03,"tone_1khz_
local_noise_energy":4.349482e-03,"tone_1khz_local_noise_avg_energy":2.71842
6e-04,"tone_1khz_local_ratio":0.724198,"tone_1khz_contrast_db":-1.40,"tone_
1khz_ratio_hit":false,"tone_1khz_peak_hit":true,"tone_1khz_contrast_hit":fa
lse,"tone_1khz_detected":false,"tone_1khz_local_noise_bins":16,"tone_1khz_b
and_bins":6,"acoustic_band_energy":2.549403e-02,"room_band_total_energy":3.
645195e-02,"low_rumble_energy":1.095792e-02,"low_rumble_ratio":0.331481,"ba
nd_sub_energy":0.000000e+00,"band_bass_energy":9.379448e-03,"band_low_mid_e
nergy":5.484819e-03,"band_mid_energy":9.357798e-03,"band_presence_energy":5
.794785e-03,"band_high_energy":6.435105e-03,"band_sub_ratio":0.000000,"band
_bass_ratio":0.257310,"band_low_mid_ratio":0.150467,"band_mid_ratio":0.2567
16,"band_presence_ratio":0.158971,"band_high_ratio":0.176537,"band_sub_peak
_hz":0.0,"band_bass_peak_hz":93.8,"band_low_mid_peak_hz":187.5,"band_mid_pea
k_hz":562.5,"band_presence_peak_hz":3703.1,"band_high_peak_hz":8625.0,"band
_sub_peak_energy":0.000000e+00,"band_bass_peak_energy":7.037619e-03,"band_
low_mid_peak_energy":1.578474e-03,"band_mid_peak_energy":1.017578e-03,"band
_presence_peak_energy":6.213540e-04,"band_high_peak_energy":3.001548e-04,"d
ominant_band_name":"bass","dominant_band_ratio":0.257310,"dominant_band_pea
k_hz":93.8,"fft_low_ratio":0.338836,"fft_mid_ratio":0.393889,"fft_high_rati
o":0.267275,"fft_total_energy":3.305750e-02,"p2p_raw":25120,"zeros":65,"tem
p_c":29.01,"rh_percent":50.74,"batt_soc_percent":null,"batt_voltage_v":null
,"batt_charge_rate_pct_per_hr":null,"batt_valid":false,"storage_error":"","
storage_error_detail":"","audio_error":"i2s_timeouts","audio_read_errors":0
,"audio_read_timeouts":98,"consecutive_timeouts":1,"total_i2s_reads":292,"s
uccessful_i2s_reads":194,"err":1,"scene_metrics_valid":true,"scene_hpf_enab
led":true,"scene_hpf_cutoff_hz":150.0,"scene_hpf_order":4,"scene_software_g
ain":16.000,"scene_rms":0.000542,"scene_dbfs":-65.32,"scene_db_spl":96.89,"
scene_peak_db_spl":109.11,"scene_f_peak_hz":187.5,"scene_f_peak_acoustic_hz
":468.8,"scene_low_rumble_ratio":0.163759,"scene_band_bass_ratio":0.068155,
"scene_band_low_mid_ratio":0.207937,"scene_band_mid_ratio":0.310908,"scene_
band_presence_ratio":0.195639,"scene_band_high_ratio":0.217361,"scene_band_
bass_peak_hz":140.6,"scene_band_low_mid_peak_hz":187.5,"scene_band_mid_peak
_hz":562.5,"scene_band_presence_peak_hz":3703.1,"scene_band_high_peak_hz":8
625.0,"scene_dominant_band_name":"mid","scene_dominant_band_ratio":0.310908

```

Figure 71: Raspberry Pi backend receiving four-node MQTT telemetry: `mosquitto_sub -h localhost -t "sdacs/node/+/features" -v`

```

PS C:\Users\kyled> "HTTP Status: $($r.StatusCode)"
HTTP Status: 200
PS C:\Users\kyled> ($r.Content | ConvertFrom-Json) | ConvertTo-Json -Depth 10
{
  "node_id": "node04",
  "record_type": "features",
  "timestamp_iso": "2026-08-07T16:00:36.597582+00:00",
  "timestamp_us": 1949192271,
  "uptime_ms": 1949192271,
  "raw_timestamp": 1949192271,
  "status": null,
  "capture_state": "capturing",
  "mode": null,
  "state": null,
  "cmd": null,
  "result": null,
  "reason": null,
  "fw_version": "mic-gain-hpf-v2",
  "seq": 59,
  "n": 48128,
  "rms": 0.000388,
  "dbfs": -68.22,
  "db_spl": 80.27,
  "peak_db_spl": 93.06,
  "cal_offset_db": 148.49,
  "tone_1khz_peak_hz": 1031.2,
  "tone_1khz_ratio": 0.060648,
  "tone_1khz_local_ratio": 0.25134,
  "tone_1khz_contrast_db": -6.0,

```

Figure 72: FastAPI endpoint returning the latest node feature record

```

kyledavid36@sdacs-broker:~ $ sqlite3 -header -column "$DB" "
SELECT
    capture_id,
    COUNT(DISTINCT node_id) AS contributing_nodes,
    COUNT(*) AS total_records
FROM capture_telemetry
WHERE capture_id = '$CAPTURE_ID'
GROUP BY capture_id;
"

```

| capture_id                  | contributing_nodes | total_records |
|-----------------------------|--------------------|---------------|
| capture_20260803T222244048Z | 4                  | 260           |

```

kyledavid36@sdacs-broker:~ $ sqlite3 -header -column "$DB" "
SELECT
    node_id,
    COUNT(*) AS stored_rows
FROM capture_telemetry
WHERE capture_id = '$CAPTURE_ID'
GROUP BY node_id
ORDER BY node_id;
"

```

| node_id | stored_rows |
|---------|-------------|
| node01  | 65          |
| node02  | 65          |
| node03  | 65          |
| node04  | 65          |

```

kyledavid36@sdacs-broker:~ $ sqlite3 -header -column "$DB" "
SELECT
    node_id,
    COUNT(*) AS rows_stored,
    MIN(timestamp_iso) AS first_record,
    MAX(timestamp_iso) AS last_record
FROM capture_telemetry
WHERE capture_id = '$CAPTURE_ID'
GROUP BY node_id
ORDER BY node_id;
"

```

| node_id | rows_stored | first_record                     | last_record                      |
|---------|-------------|----------------------------------|----------------------------------|
| node01  | 65          | 2026-08-03T22:22:42.990384+00:00 | 2026-08-03T22:23:49.434022+00:00 |
| node02  | 65          | 2026-08-03T22:22:42.967039+00:00 | 2026-08-03T22:23:49.454052+00:00 |
| node03  | 65          | 2026-08-03T22:22:43.010077+00:00 | 2026-08-03T22:23:49.444039+00:00 |
| node04  | 65          | 2026-08-03T22:22:42.944159+00:00 | 2026-08-03T22:23:49.472675+00:00 |

Figure 73: SQLite records produced during a synchronized capture.

## 6.6. Node-RED Data Collection Middleware

### 6.6.1. Functional units

- MQTT input
- Live monitoring
- Capture-label selection
- Capture-duration selection
- Start-capture command
- Progress display
- CSV logging
- Dataset generation
- Debug and diagnostic nodes

### 6.6.2. OSI classification

Primarily:

- **Layer 6:** JSON and CSV formatting
- **Layer 7:** MQTT processing, dashboard controls and data export

### 6.6.3. Development and Testing evidence

- Node-RED flow export (= export download or copy screenshot)
- Flow-chart screenshot + dashboard screenshot (ref 5.1.3)
  - Capture-label control
  - Start-capture command
  - Debug-node output
  - CSV output
  - Example labelled dataset
  - Reference to flows.json

### 6.6.4. System Integration and Performance evidence

- Four-node labelled capture with completed capture session

## Final Dataset Tracker

PREVIOUS COMPLETED  
CAPTURE

### Speech

capture\_20260805032258  
Aug 4, 2026, 11:23:20 p.m.

NEXT RECOMMENDED LABEL

### Quiet Room

Current selection: Speech  
In progress:  
capture\_20260804155146 —  
1/4 nodes

FINAL DATASET PROGRESS

**35 / 30**

Counts advance only after  
node01–node04 complete.

Quiet 11 / 10

Speech 13 / 10

Noisy 11 / 10

### Layout C

**node01**  
Centre

**node02**  
Rear Right

**Sound source**  
Front Centre

**node03**  
Front Right

**node04**  
Rear Centre

Arrangement name  
Layout C

node01

Centre ▼

node02

Rear Ri... ▼

node03

Front R... ▼

node04

Rear C... ▼

Sound source

Front... ▼

 **SAVE ARRANGEMENT**

 **RESET TRACKER**

## AI Dataset Collection

Dataset label Select one of the six labels Current **Speech**

|                                                                                                      |                                                                                                   |                                                                                                              |
|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
|  NOISY              |  SPEECH          |  LOW (40–500 HZ)          |
|  MID (400 HZ–4 KHZ) |  HIGH (4–20 KHZ) |  QUIET ROOM (WHITE NOISE) |

Latest **VALID | node03 seq 14 | speech | raw -68.74 dBFS | scene -65.22 dBFS |**  
 row **clipped 0 | ready; warning=i2s\_timeouts**

[Download AI Dataset CSV](#)

**AI CSV header — raw spectral path + 150 Hz HPF scene path:**

timestamp\_iso,run\_id,node\_id,seq,sample\_rate\_ok,effective\_sample\_rate\_hz,audio\_error,target\_freq\_hz,target\_band,suggested\_z\_metric,rms,dbfs,db\_spl,f\_peak\_hz,f\_peak\_full\_hz,f\_peak\_acoustic\_hz,tone\_1khz\_ratio,tone\_1khz\_peak\_hz,tone\_1khz\_ratio\_hit,tone\_1khz\_peak\_hit,tone\_1khz\_detected,tone\_1khz\_local\_total\_energy,tone\_1khz\_local\_noise\_energy,tone\_1khz\_local\_noise\_avg\_energy,tone\_1khz\_local\_ratio,tone\_1khz\_contrast\_db,tone\_1khz\_contrast\_hit,low\_rumble\_ratio,room\_band\_total\_energy,band\_sub\_ratio,band\_bass\_ratio,band\_low\_mid\_ratio,band\_mid\_ratio,band\_pres

 **FIX CSV HEADER**

 **RESET CSV**

CSV status **AI CSV header replaced with the dual-path HPF schema. Existing legacy rows were retained but should not be mixed with new training rows.**

**Labelled training dataset only — dual-path schema.** Select exactly one ground-truth label before each 60 s Node-RED dataset capture. Only a start\_capture command containing one of the six valid labels estimates learning to ... Some fields remain available for ...

- CSV containing records from all nodes (see final box dataset in appendix files) shows evidence that labels and request IDs remain synchronized over 4 nodes.

## 6.7. Edge Impulse AI Classification

### 6.7.1. Functional units

The Edge Impulse classification system consists of the following functional units:

- Dataset cleaning and validation
- Feature selection
- 31-feature acoustic input vector
- Class-label assignment
- Training and validation dataset separation
- Feature normalization
- Feedforward ANN classifier
- Three-class Softmax output
- Confusion-matrix evaluation
- Accuracy and loss evaluation
- Model optimization and deployment
- Raspberry Pi inference
- Classification confidence generation

These functional units form the complete machine-learning pipeline between the acoustic measurements generated by SDACS and the final acoustic-scene classification. The classification system operates on engineered acoustic measurements rather than raw microphone recordings, reducing model complexity and allowing the same measurements used elsewhere in SDACS to also support machine-learning inference.

### 6.7.2. OSI classification

The Edge Impulse classifier primarily operates at Layer 7 – Application, where the acoustic feature vectors are processed by the trained neural network and converted into acoustic-scene classifications. The representation and formatting of the input feature vectors, class labels, and confidence values also relate to Layer 6 – Presentation.

### *6.7.3. Development and Testing evidence*

Development of the Edge Impulse classifier was completed through multiple iterations. The initial model was developed using the first available SDACS acoustic dataset and six acoustic classes. This model achieved a validation accuracy of **71.4%**, demonstrating that the engineered acoustic measurements contained sufficient information for the neural network to identify differences between acoustic environments. However, analysis of the confusion matrix showed considerable overlap between several classes, particularly those with similar frequency characteristics.

The initial results identified dataset size and class similarity as major limitations. Additional acoustic measurements were therefore collected through the SDACS Node-RED dataset-collection workflow. During dataset preparation, fields that could reveal the correct class directly rather than describe the acoustic signal were excluded from the model inputs to prevent label leakage. The remaining measurements were used to construct the engineered acoustic feature vectors used during training.

The final model reduced the classification problem to three broader acoustic classes: Noisy, Quiet Room with Noise, and Speech. This produced a more practical classification problem while retaining classes that could be distinguished using the measurements generated by the SDACS nodes.

The final model achieved a validation accuracy of **82.4%** with a validation loss of **0.42**. The weighted average precision was approximately **0.83**, weighted recall was **0.82**, and weighted F1 score was **0.83**. The area under the ROC curve was approximately **0.94**. These results represent an improvement of **11.0 percentage points** over the initial 71.4% model.

The confusion matrix showed that Quiet Room with Noise produced the strongest classification performance at approximately **89.7%**, followed by **Noisy** at approximately **80.2%** and Speech at approximately **75.6%**. The remaining errors primarily occurred between Speech and Noisy conditions, indicating that these environments can contain overlapping spectral and signal-level characteristics.

The final model was also evaluated for embedded deployment using the Edge Impulse EON Compiler. The optimized model required approximately 19.0 kB of flash, 1.6 kB of peak RAM, and an estimated 2 ms inference time. These requirements demonstrate that the classifier is lightweight relative to the processing resources available on the Raspberry Pi 5 and support future investigation into deployment closer to the embedded sensing nodes.

Overall, the progression from the initial model to the final classifier demonstrated that increasing the quantity and quality of the dataset, removing potentially misleading fields, and reducing overlapping classes significantly improved classification performance.



Figure 74: First Iteration of AI Model

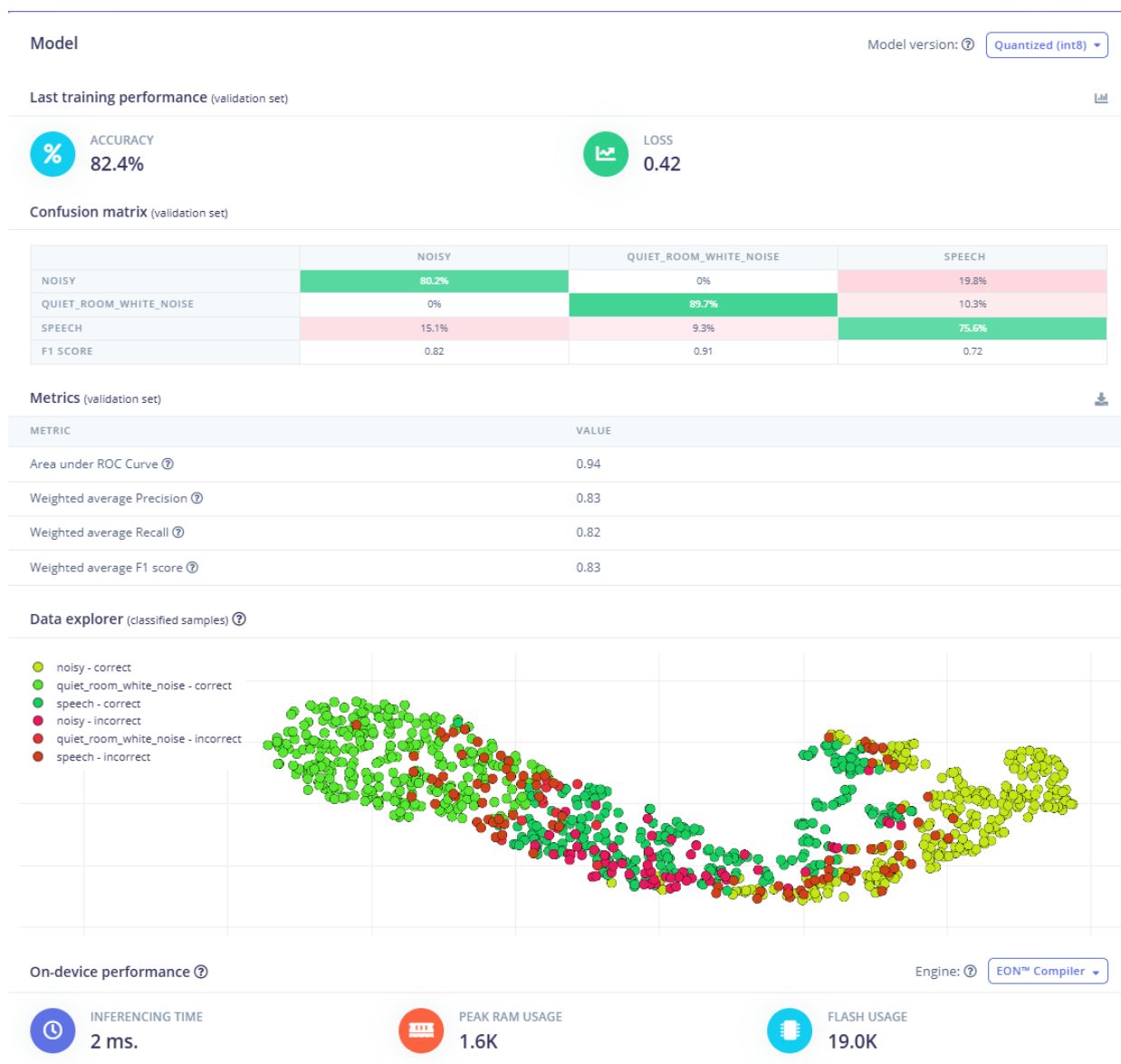


Figure 75: Final Iteration of AI Model

#### 6.7.4. System Integration and Performance evidence

The final Edge Impulse model was integrated into the SDACS processing workflow to demonstrate that acoustic measurements collected by the distributed nodes could be used for automated acoustic classification. Feature measurements generated by the ESP32-S3 nodes are transmitted through MQTT to the Raspberry Pi, where they are validated and stored by the SDACS middleware. These same feature vectors provide the input required by the trained classifier.

The integration demonstrates the complete processing path:

ESP32-S3 Nodes → MQTT → Raspberry Pi / Node-RED → Acoustic Feature Vector → Edge Impulse Classifier → Classification and Confidence

A video demonstration, [SDACS\\_Edge-Impulse.mp4](#), provides evidence of the completed Edge Impulse model and classification workflow.

- Video demonstration  
Link: [SDACS\\_Edge-Impulse.mp4](#)

## 6.8. MATLAB Analysis

### 6.8.1. Functional units

- CSV or JSON import
- Four-node comparison
- Frequency-band analysis
- SPL comparison
- Room-position mapping
- 2D or 3D visualization
- Summary generation
- Export to the backend or Flutter GUI

### 6.8.2. OSI classification

- **Layer 6:** CSV/JSON data representation
- **Layer 7:** MATLAB analysis and visualization

### 6.8.3. Development and Testing evidence

- Development Video

Link: [AW\\_MATLAB\\_Demo.mp4](#)

#### 6.8.4. System Integration and Performance evidence

- MATLAB analysis using data generated by the four-node system

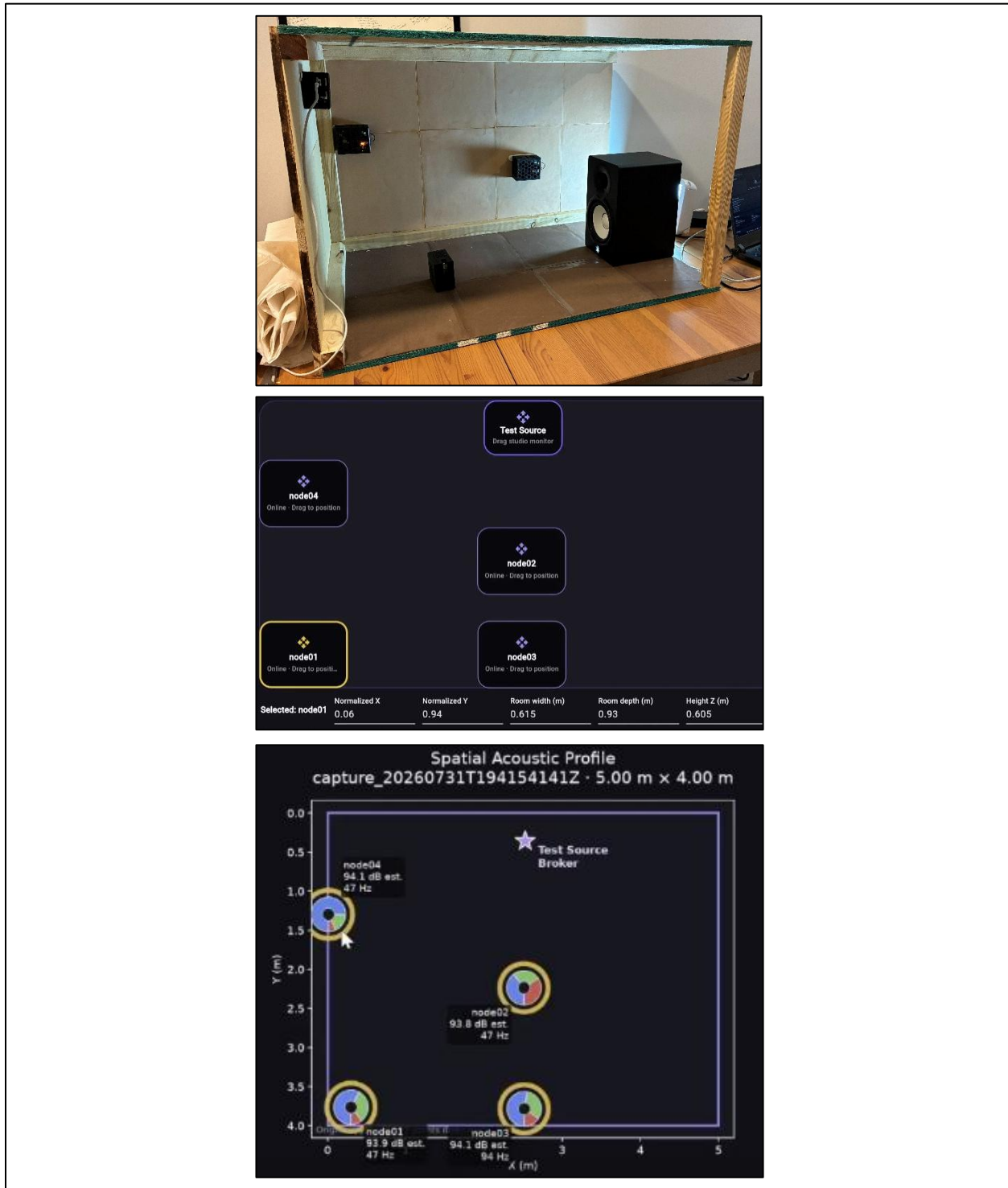


Figure 76: SDACS Node Layout, Actual (Real) vs. GUI (Setup) vs. Spatial Acoustic Profile (Results)

## 6.9. Flutter/Dart GUI

### 6.9.1. Functional units

- Backend connection
- Dashboard
- Four-node status cards
- Node-detail display
- Live telemetry
- Capture and test controls
- Calibration controls
- AI prediction display
- MATLAB room-analysis display
- History and CSV export
- Error and offline-node messages

### 6.9.2. OSI classification

Primarily:

- **Layer 6:** JSON parsing and display formatting
- **Layer 7:** REST API requests, user controls and visualization

### 6.9.3. Development and Testing evidence

The Flutter/Dart graphical user interface was developed as the primary operator interface for SDACS. Development focused on providing a single interface through which the user could monitor the four distributed nodes, configure the recording environment, initiate acoustic captures and calibration procedures, and review the resulting acoustic analysis.

The GUI was developed using a modular page structure connected to the Raspberry Pi FastAPI backend through REST API requests. Individual screens were developed and tested for system overview, node status, room setup, calibration, capture control, acoustic results, and system diagnostics. The final interface maintains a consistent layout and visual design across these functions while separating operator controls from measurement results.

Frontend testing verified the following functionality:

- Navigation between the major application pages
- Backend connectivity and REST API communication

- Four-node status and telemetry display
- Room and node configuration
- Capture and calibration controls
- Node-result comparison
- MATLAB spatial-acoustic-profile display
- AI classification result display
- Offline-node and warning handling
- Responsive interface behaviour

The GUI was also tested under partial system availability. If one node becomes unavailable, the application identifies the node as offline while continuing to display information from the remaining active nodes. This prevents a single node failure from making the complete interface unusable and provides the operator with a clear indication of the system state.

The results page was designed to combine the outputs from the different SDACS analysis stages. Individual node measurements are displayed alongside warnings for potentially unreliable measurements, while the MATLAB spatial acoustic profile provides a visual representation of the room. The interface therefore acts as the final presentation layer between the distributed sensing system and the user.

Figures 75 and the following GUI screenshots provide development and testing evidence for the completed Flutter interface. Additional system-level operation is demonstrated in the GUI demonstration videos.

- Flutter process-flow chart

# Flutter Operator Interface — System Block Diagram

System overview, node monitoring, capture and calibration workflows, and results visualization.

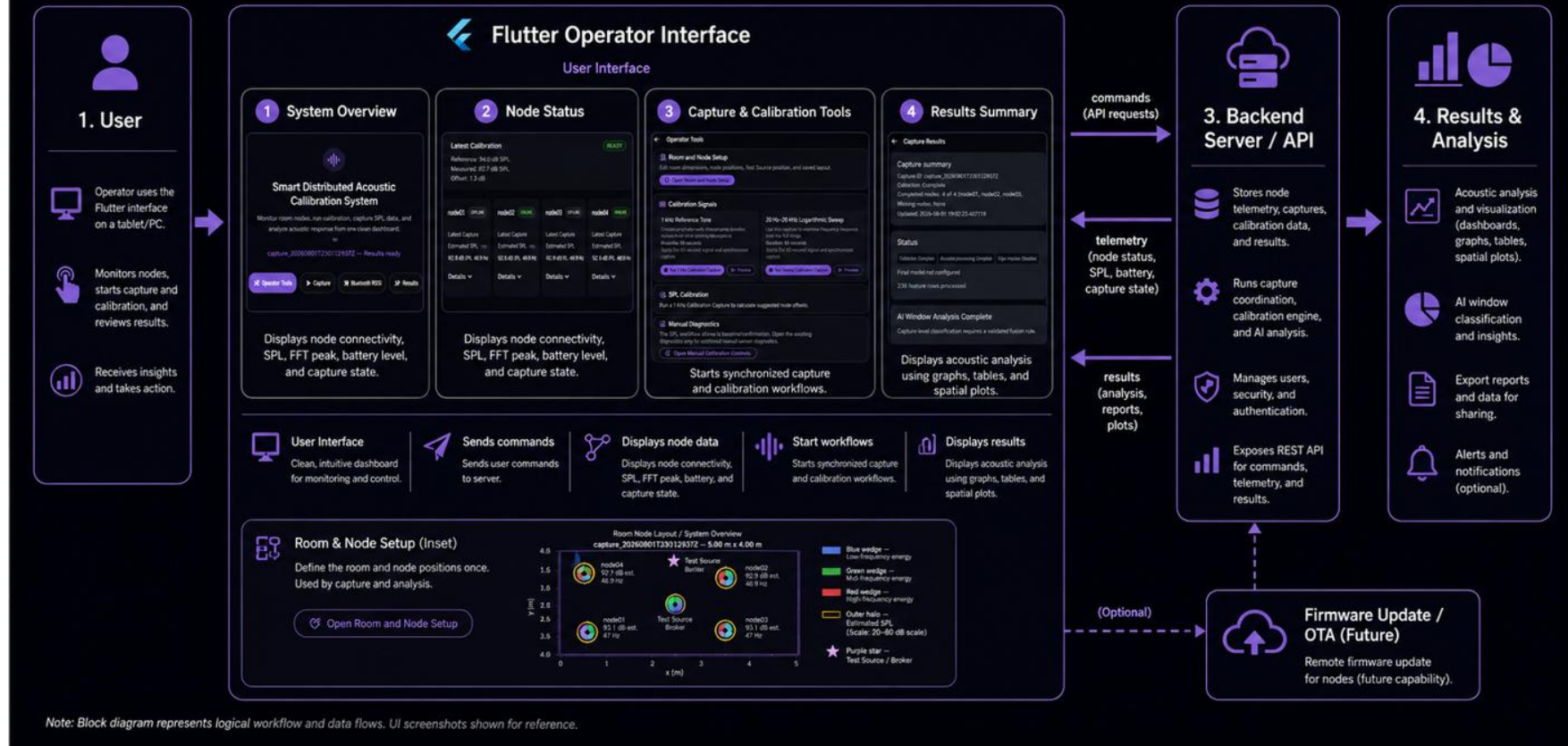
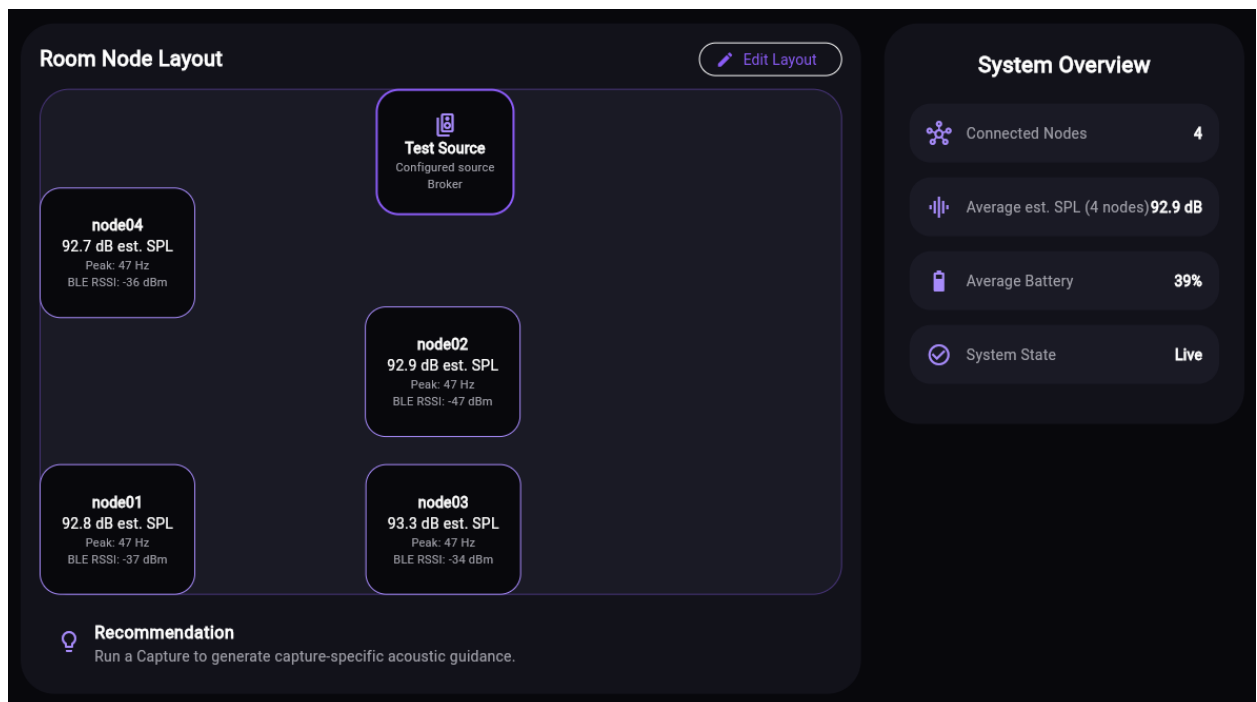
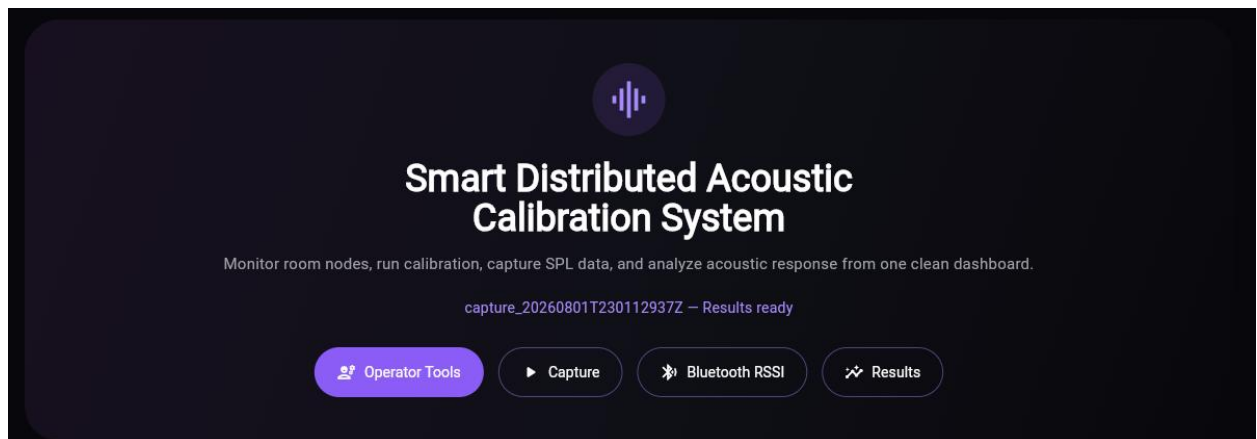


Figure 77: Flutter Operation Interface

- Screenshots of each major page



**Latest Calibration** READY

Reference: 94.0 dB SPL  
Measured: 92.7 dB SPL  
Offset: 1.3 dB

**node01** ONLINE  
Latest Capture  
Estimated SPL Peak  
**92.8 dB SPL 46.9 Hz**  
Details ▾

**node02** ONLINE  
Latest Capture  
Estimated SPL Peak  
**92.9 dB SPL 46.9 Hz**  
Details ▾

**node03** ONLINE  
Latest Capture  
Estimated SPL Peak  
**93.3 dB SPL 46.9 Hz**  
Details ▾

**node04** ONLINE  
Latest Capture  
Estimated SPL Peak  
**92.7 dB SPL 46.9 Hz**  
Details ▾

← **Operator Tools**

 **Room and Node Setup**  
Edit room dimensions, node positions, Test Source position, and saved layout.  
[✓ Open Room and Node Setup](#)

 **Calibration Signals**

**1 kHz Reference Tone**  
Co-locate all four node microphones beside a reference sound-level meter during this capture.  
**Duration: 60 seconds**  
Starts the 60-second signal and synchronized four-node capture.  
[● Run 1 kHz Calibration Capture](#) [▶ Preview](#)

**20 Hz–20 kHz Logarithmic Sweep**  
Use this capture to examine frequency response across the room.  
**Duration: 60 seconds**  
Starts the 60-second signal and synchronized four-node capture.  
[● Run Sweep Calibration Capture](#) [▶ Preview](#)

 **SPL Calibration**  
Run a 1 kHz Calibration Capture to calculate suggested node offsets.

 **Manual Diagnostics**  
The SPL workflow above is backend-authoritative. Open the existing controls only for additional manual sensor diagnostics.  
[△ Open Manual Calibration Controls](#)

← Setup

Backend and Nodes

Backend IP  
192.168.5.61

Backend port  
8000

Number of nodes  
4

Room

Room length (m)  
6.0

Room width (m)  
4.0

Room height (m)  
2.5

Wall material  
Drywall

Wi-Fi

Wi-Fi SSID

Wi-Fi password

Save Setup

← Calibration

Delay (ms)  
0

Duration (s)  
30

Start Calibration

Stop Calibration

Calibration Status

No calibration has been run in this session.

READY

←

Capture Results

Capture summary

Capture ID: capture\_20260801T230112937Z  
Collection: Complete  
Completed nodes: 4 of 4 (node01, node02, node03, node04)  
Missing nodes: None  
Updated: 2026-08-01 19:02:23.437119

Status

Collection: Complete

Acoustic processing: Complete

Edge Impulse: Disabled

Final model not configured  
236 feature rows processed

AI Window Analysis Complete

Capture-level classification requires a validated fusion rule.

Spatial Acoustic Profile

↻

⌵

Node position from saved room layout. Band wedges show low/mid/high energy ratios. Halo represents Estimated SPL.

Spatial Acoustic Profile  
capture\_20260801T230112937Z · 5.00 m × 4.00 m

node04  
92.7 dB est.  
47 Hz

Test Source  
Broker

node02  
92.9 dB est.  
47 Hz

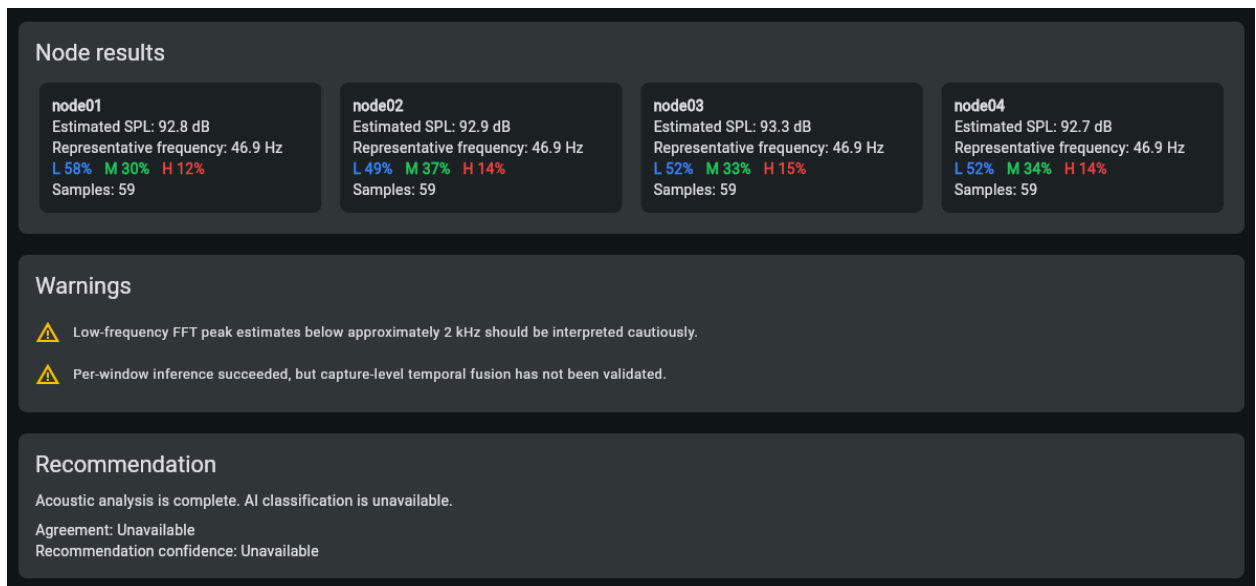
node01  
92.8 dB est.  
47 Hz

node03  
93.3 dB est.  
47 Hz

Blue wedge — Low-frequency energy  
Green wedge — Mid-frequency energy  
Red wedge — High-frequency energy  
Outer halo — Estimated SPL (fixed 20-80 dB scale)  
Purple star — Test Source / Broker

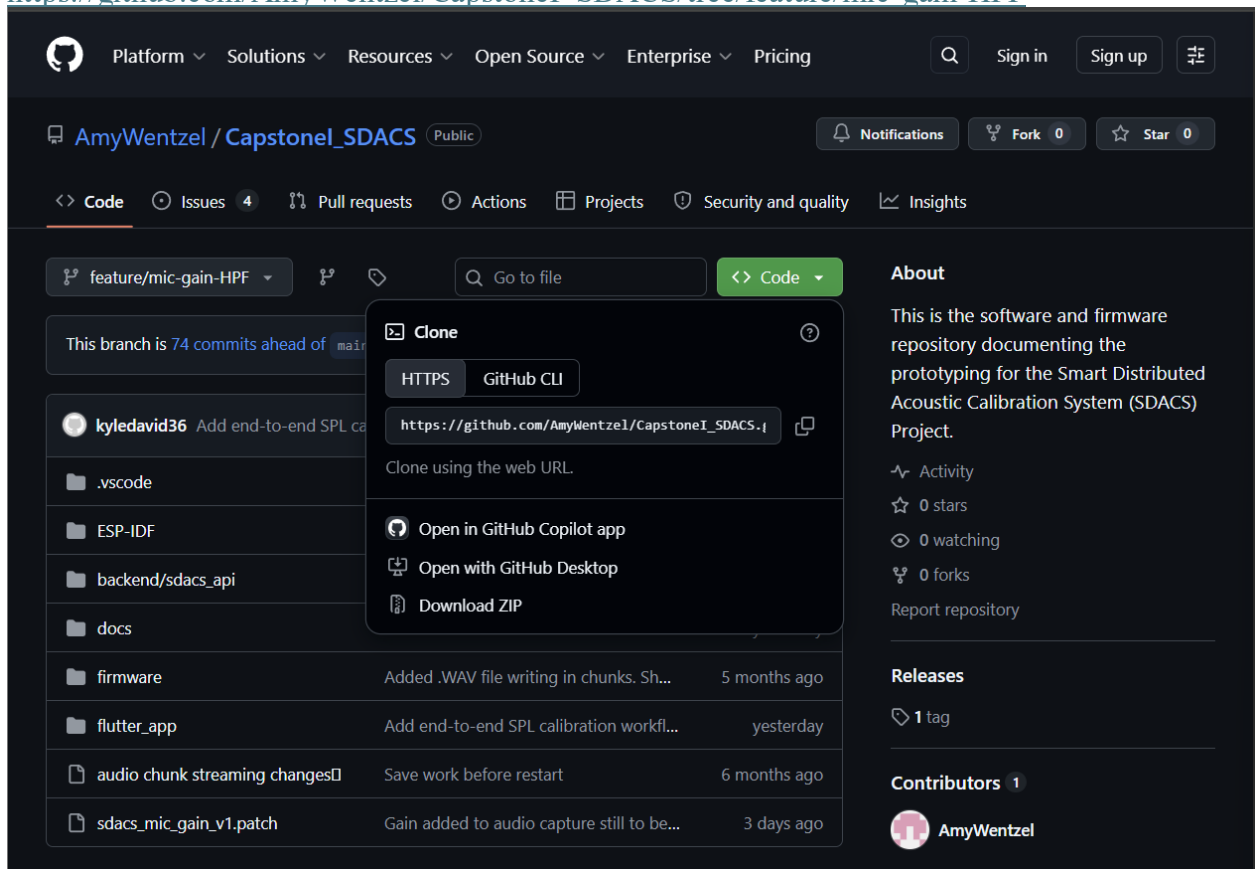
Room summary

Nodes used: node01, node02, node03, node04  
Mean estimated SPL: 92.9 dB  
Dominant band: low  
Dominant frequency: 99.9 Hz  
Loudest node: node03  
Quietest node: node04  
Spatial variation: 0.6 dB



- Node-card data comparison
- Button-function tests
- Offline-node handling (what if missing node03)
- Calibration-form testing
- AI-result display
- Reference to Dart source files (Link latest mic-gain-HPF repo branch .zipped)

- [https://github.com/AmyWentzel/CapstoneI\\_SDACS/tree/feature/mic-gain-HPF](https://github.com/AmyWentzel/CapstoneI_SDACS/tree/feature/mic-gain-HPF)



→ Download zip and add to project files

#### 6.9.4. System Integration and Performance evidence

- Videos of the complete GUI demonstration
  1. [SDACS\\_Test\\_SPL-Calibration.mp4](#)
  2. [SDACS\\_Demo\\_RSSI.mp4](#)
  3. [SDACS\\_Test\\_Frequency-Sweep.mp4](#)

## 7. RESULTS AND ANALYSIS

### 7.1. Node Frequency Detection Accuracy

The SDACS nodes use the ICS-43432 MEMS microphone to record audio. The nodes were tested at various frequencies in the audible range (up to 20kHz) to determine the accuracy of the frequency detection from the audio captures.

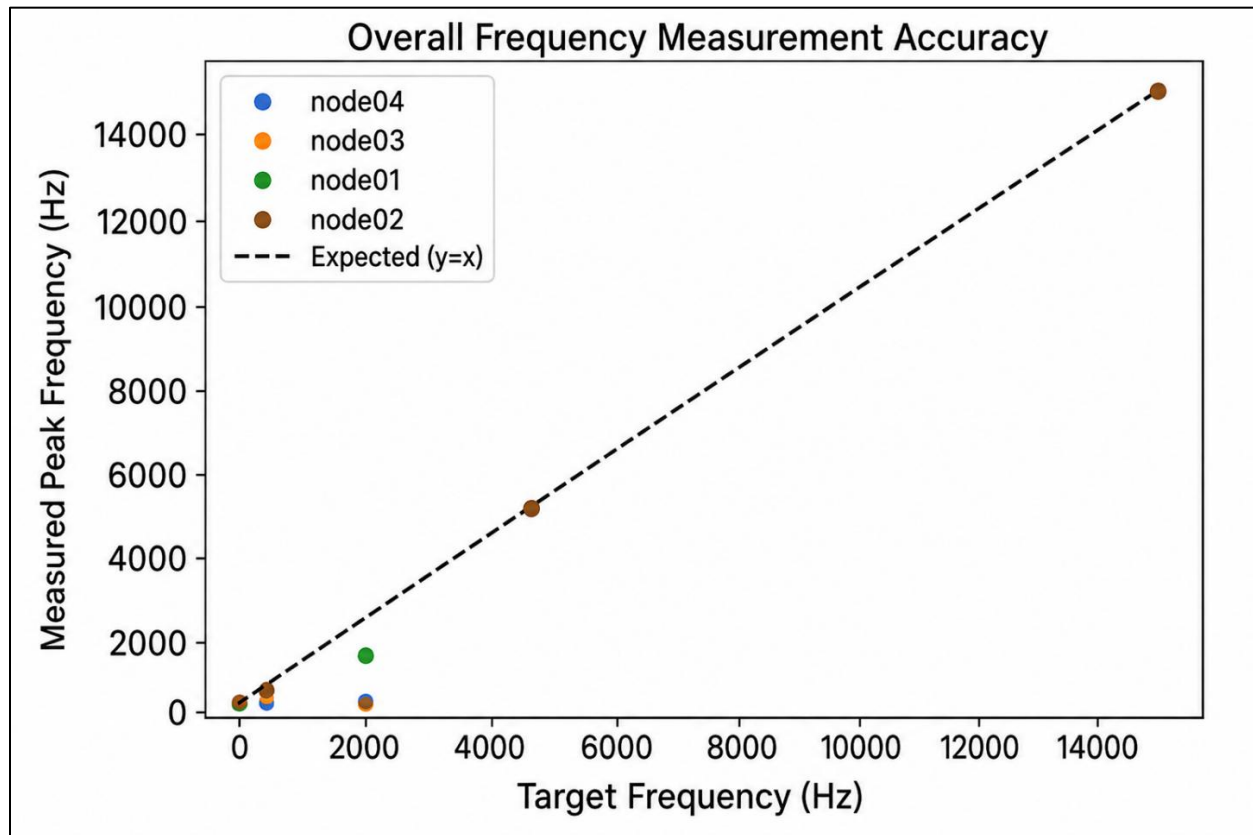


Figure 78: Summary of All Nodes, Target Frequency (Hz) vs. Measured Peak Frequency (Hz)

The results show a significant drop in accuracy for the mid-to-low frequency range ( $< 3\text{kHz}$ ), which presents a data quality issue for the MATLAB and TinyML models. Based on the results from testing, an ideal lab environment was constructed to optimize the frequency detection in the lower range. The purpose of the lab environment was to minimize distance between nodes, remove obstructing objects, and isolate the nodes from any external factors.



*Figure 79: Ideal Lab Environment for Maximum Audio Data Quality*

The lab environment significantly increased accuracy for the spatial acoustic results and analyses. The volume of the enclosure was 2ft. by 3ft. by 2ft, and when insulated, provides the ideal profile results for each node based on their position in the “room”. This suggests that a larger number of recording nodes are needed to accurately pick up on lower frequencies and analyze frequency content within a larger recording space.

### *7.2. TinyML Classifier Accuracy*

The final Edge Impulse TinyML classifier achieved a validation accuracy of 82.4%, compared with 71.4% for the initial model. This represents an improvement of 11.0 percentage points following additional dataset collection, feature validation, and simplification of the classification problem from six classes to three broader acoustic conditions.

The final confusion matrix demonstrated different performance levels across the three classes. Quiet Room with Noise achieved the highest classification rate at approximately 89.7%, followed by Noisy at approximately 80.2% and Speech at approximately 75.6%. The weighted average precision, recall, and F1 score were approximately 0.83, 0.82, and 0.83, respectively, while the area under the ROC curve was approximately 0.94.

The remaining classification errors were primarily associated with overlap between noisy and speech-containing environments. This is expected because speech may occur alongside background noise and can produce acoustic features that overlap with the broader noisy class. The model therefore performs best when the acoustic characteristics of a capture are clearly representative of one of the three trained environments.

The increase from 71.4% to 82.4% demonstrates that the classification performance is strongly dependent on dataset quality and class definition. Although the final classifier does not provide perfect classification, the results demonstrate that the acoustic features generated directly by the SDACS nodes can successfully support automated acoustic-scene recognition without requiring the neural network to process raw audio.

### *7.3. Spatial Acoustic Profile Accuracy*

The overall SDACS acoustic analysis depends on two primary stages: accurate frequency measurement at the distributed nodes and interpretation of those measurements through the MATLAB spatial model and TinyML classifier. Testing demonstrated that the complete system is capable of collecting measurements from multiple locations and using them to generate a spatial representation of the recording environment. However, the reliability of this analysis is directly dependent on the quality of the acoustic measurements produced by each node.

Frequency testing identified reduced measurement accuracy below approximately 3 kHz, which affects both the MATLAB spatial acoustic profile and the engineered features provided to the TinyML classifier. When the nodes were moved into the controlled 2 ft × 3 ft × 2 ft insulated test environment, the spatial analysis became significantly more consistent. Reducing node-to-source

distance, external noise, and physical obstructions improved the quality of the measurements used to construct the acoustic profile.

These results indicate that the SDACS processing and visualization architecture functions as intended, but spatial accuracy decreases as the size and acoustic complexity of the recording environment increases. With only four measurement nodes, each node represents a relatively large region of the room. Additional nodes would increase spatial sampling density and provide the MATLAB model with more measurement points from which to estimate the room's acoustic characteristics.

The TinyML results provide additional evidence that useful acoustic information can be extracted from the node measurements. The final classifier achieved 82.4% validation accuracy, demonstrating that the generated acoustic features contain identifiable patterns even with the known limitations of the microphone measurements. Together, the MATLAB spatial analysis and TinyML classification results validate the overall SDACS concept while also identifying measurement density and low-frequency detection as the primary areas requiring future improvement.

Overall, SDACS successfully demonstrates the feasibility of using a distributed network of low-cost embedded acoustic nodes to collect, analyze, classify, and visualize room acoustic information. Increasing the number of nodes, improving low-frequency measurement accuracy, and expanding the machine-learning dataset would be expected to improve both spatial-profile reliability and acoustic classification accuracy.

## 8. CONCLUSION AND RECOMMENDATIONS

### *8.1. Conclusion*

The SDACS project is a distributed acoustic calibration framework capable of multi-node data acquisition, wireless aggregation, and spatial visualization. Testing and evaluation verified that the system can reliably capture digital audio, perform frequency-domain analysis, monitor environmental and battery parameters, and stream metrics to a centralized server. While initial evaluations revealed limitations in low-frequency ( $< 3$  kHz) detection accuracy due to environmental noise and microphone constraints, controlled lab testing demonstrated that structured positioning and insulation significantly improve spatial profile reliability. Ultimately, SDACS provides an affordable, user-friendly alternative to commercial studio analyzers, bridging the gap between professional acoustic evaluation and accessible home-studio implementation.

### *8.2. Future Recommendations*

#### *8.2.1. Expand the Network*

Scale the deployment to support 8 or more distributed sensor nodes simultaneously, which will significantly increase spatial resolution, reduce interpolation errors, and produce higher-accuracy acoustic profiles across larger studio spaces.

#### *8.2.1. Detailed FFT Analysis*

Expand the Edge Impulse TinyML dataset by capturing a wider diversity of room volumes, shapes, and acoustic treatment types to improve classification confidence. Implement advanced frequency-domain processing to enable automated instrument profile recommendations and precise harmonic identification for complex sound sources.

#### *8.2.2. Minimize and Improve Hardware*

Redesign the physical architecture by integrating the system onto a single System-on-Chip (SoC) board, incorporating specialized probe or lobe microphones, and shrinking the overall product footprint by 50% for enhanced portability and discreet placement.

## APPENDICES

Github Link:

[https://github.com/AmyWentzel/CapstoneI\\_SDACS.git](https://github.com/AmyWentzel/CapstoneI_SDACS.git)

See attached documents that support this section included in the project drop box.

## REFERENCES

- [1] Ableton, “Music production with Live and Push | Ableton,” *Ableton.com*, 2018. <https://www.ableton.com/> (accessed Jan. 26, 2026).
- [2] Apple, “Logic Pro X,” *Apple*, 2019. <https://www.apple.com/logic-pro/> (accessed Jan. 26, 2026).
- [3] Audacity, “Audacity | free, open source, cross-platform audio software for multi-track recording and editing.,” *Audacityteam.org*, May 13, 2019. <https://www.audacityteam.org/> (accessed Jan. 26, 2026).
- [4] “Why Acoustics Matter The Science Behind a Studio Recording Studio,” *Finchley.co.uk*, Nov. 24, 2024. <https://www.finchley.co.uk/finchley-learning/why-acoustics-matter-the-science-behind-a-studio-recording-studio> (accessed Jan. 26, 2026).
- [5] J. Mulcahy, “REW - Room EQ Wizard Room Acoustics Software,” *Roomeqwizard.com*, 2018. <https://www.roomeqwizard.com/> (accessed Jan. 26, 2026).
- [6] “ARC Studio,” *Ikmultimedia.com*, 2026. <https://www.ikmultimedia.com/products/arstudio/index.php?p=versions#view> (accessed Jan. 26, 2026).
- [7] “SoundID™ - Audio Personalization App,” *Sonarworks*, 2025. <https://www.sonarworks.com/soundid> (accessed Jan. 26, 2026).
- [8] M. R. Bai, Y.-H. Chiang, and P.-C. Wu, “Analysis and synthesis of three-dimensional sound fields using a hybrid microphone array and a 3-layered loudspeaker array,” *Journal of Sound and Vibration*, vol. 443, pp. 666–682, Mar. 2019, doi: <https://doi.org/10.1016/j.jsv.2018.12.018>.
- [9] L. Carneiro and A. Berry, “Three-dimensional sound source diagnostic using a spherical microphone array from multiple capture positions,” *Mechanical Systems and Signal Processing*, vol. 199, p. 110455, Jun. 2023, doi: <https://doi.org/10.1016/j.ymssp.2023.110455>.
- [10] Bluetooth SIG, *Enhancing Bluetooth Location Services with Direction Finding*, Jan. 2019. Accessed: Jan. 27, 2026. [Online]. Available: [https://www.bluetooth.com/wp-content/uploads/2019/03/1901\\_Enhancing-Bluetooth-Location-Service\\_FINAL.pdf](https://www.bluetooth.com/wp-content/uploads/2019/03/1901_Enhancing-Bluetooth-Location-Service_FINAL.pdf)

- [11] *NUMERICAL RECIPES IN FORTRAN 77: THE ART OF SCIENTIFIC COMPUTING*. Cambridge University Press, 1992. Accessed: Jan. 27, 2026. [Online]. Available: <https://phys.uri.edu/nigh/NumRec/bookfpdf/fl2-4.pdf>
- [12] miniDSP Inc., “UMIK-1 USB Measurement Microphone,” miniDSP. Accessed: Jan. 19, 2026. [Online]. Available: <https://www.minidsp.com/products/acoustic-measurement/umik-1>
- [13] Sonarworks Ltd., “SoundID Reference – Pricing and Product Comparison,” Sonarworks. Accessed: Jan. 19, 2026. [Online]. Available: <https://www.sonarworks.com/soundid-reference/pricing>
- [14] Flutter Dev, “Learn Flutter,” Flutter. Accessed: Jan. 19, 2026. [Online]. Available: <https://flutter.dev/learn>
- [15] TDK InvenSense, *ICS-43434 High Performance Digital Microphone Datasheet*, San Jose, CA, USA, 2016.
- [16] F. Alton Everest and K. C. Pohlmann, *Master Handbook of Acoustics*, 6th ed. New York, NY, USA: McGraw-Hill, 2015.
- [17] Analog Devices, *MAX17048/MAX17049 ModelGauge Fuel Gauge IC Datasheet*, Wilmington, MA, USA, 2022.
- [18] Espressif Systems, *ESP32-S3 Series Datasheet*, Shanghai, China, 2023.
- [19] Raspberry Pi Ltd., *Raspberry Pi 5 Product Brief*, Cambridge, U.K., 2023.
- [20] Rational Acoustics, *Smaart v9 User Guide and Technical Overview*, Rational Acoustics LLC, Upper Saddle River, NJ, USA, 2023. [Online]. Available: <https://www.rationalacoustics.com>. Accessed: Jan. 27, 2026.
- [21] Dart Language, “Dart Programming Language,” Dart. Accessed: Jan. 19, 2026. [Online]. Available: <https://dart.dev/>
- [22] MQTT Org., “MQTT – The Standard for IoT Messaging,” MQTT. Accessed: Jan. 24, 2026. [Online]. Available: <https://mqtt.org/>
- [23] HowStuffWorks, “How Bluetooth Works,” HowStuffWorks – Electronics. Accessed: Jan. 24, 2026. [Online]. Available: <https://electronics.howstuffworks.com/bluetooth.htm>
- [24] espressif, “esp-dsp/modules/fft/fixed/dsps\_fft2r\_sc16\_ansi.c at master · espressif/esp-dsp,” GitHub, 2025. [https://github.com/espressif/esp-](https://github.com/espressif/esp-dsp)

- [dsp/blob/master/modules/fft/fft2r\\_sc16\\_ansi.c#L117](#) (accessed Feb. 02, 2026).
- [25] “2-D fast Fourier transform - MATLAB fft2,” [www.mathworks.com](#).  
<https://www.mathworks.com/help/matlab/ref/fft2.html> (accessed Mar. 31, 2026).
- [26] M. Riani, “FSDA - Flexible Statistics Data Analysis toolbox - File Exchange - MATLAB CentralFile Exchange - MATLAB Central,” [Mathworks.com](#), Feb. 15, 2026.  
<https://www.mathworks.com/matlabcentral/fileexchange/72999-fsda-flexible-statistics-data-analysis-toolbox/> (accessed Mar. 31, 2026).
- [27] M. Riani, D. Perrotta, and F. Torti, “FSDA: A MATLAB toolbox for robust analysis and interactive data exploration,” *Chemometrics and Intelligent Laboratory Systems*, vol. 116, pp. 17–32, Jul. 2012, doi: <https://doi.org/10.1016/j.chemolab.2012.03.017>.
- [28] Ajeet Kumar Gandhi, V. Mishra, Bhanu Vashistha, M. Rastogi, and Rajesh Vashistha, “Distribution,” Elsevier eBooks, pp. 131–133, Jan. 2023, doi: <https://doi.org/10.1016/b978-0-323-88423-5.00052-2>.
- [29] A. Wentzell, *Project Idea Document*, Conestoga College, 2026.
- [30] A. Wentzell, *Assessment 1 Work Summary*, Conestoga College, 2026.
- [31] A. Wentzell, *SDACS System Diagram*, Conestoga College, 2026.
- [32] A. Wentzell, *MATLAB Research for SDACS*, Conestoga College, 2026.
- [33] A. Wentzell, *Battery Port Design Rev. B*, Conestoga College, 2026.
- [34] K. Dick, *Capstone 1 Bill of Materials*, Conestoga College, 2026.
- [35] K. Dick, A. Wentzell, and D. Toma, *Capstone Project Description*, Conestoga College, 2026.
- [36] K. Dick, A. Wentzell, and D. Toma, *ESE Capstone Functional Block Diagram*, Conestoga College, 2026.
- [37] K. Dick, *Capstone Idea Document*, Conestoga College, 2026.
- [38] A. Wentzell, *MCU Housing Design Rev. B*, Conestoga College, 2026.
- [39] A. Wentzell, *SDACS Enclosure Rough Concept*, Conestoga College, 2026.
- [40] K. Dick, *SDACS Node Calibration and Validation Report*, Conestoga College, 2026.
- [41] K. Dick, A. Wentzell, and D. Toma, *SDACS Project Proposal*, Conestoga College, 2026.
- [42] A. Wentzell, *SDACS Test Report, March 2026*, Conestoga College, 2026.

- [43] K. Dick, A. Wentzell, and D. Toma, *SDACS User Requirements and Project Plan*, Conestoga College, 2026.
- [44] K. Dick, A. Wentzell, and D. Toma, *SMART Centre Git Notes and Process Flow*, Conestoga College, Sep. 4, 2025.
- [45] A. Wentzell, *Top Vent Design Rev. B*, Conestoga College, 2026.
- [46] A. Wentzell, *Node Frequency Detection Analysis*, Conestoga College, 2026.